Bayesian neural networks versus neural posterior estimation for simulation-based inference with little data

Sam Dowd
Seljak Group, UC Berkeley
November 2026

Abstract

We construct a toy problem for simulation-based inference (SBI) on cosmological fields, which in the simplest, Gaussian setting depend on their parameters only through the power spectrum: inferring the amplitude and length scale of a radial basis function (RBF) kernel, which set a Gaussian field's power spectrum, from the empirical covariance of ten noisy draws of the field, a task whose exact posterior is known. On it we compare Bayesian neural networks (BNNs) sampled with Microcanonical Langevin Ensembles (MILE) against neural posterior estimation (NPE) at small simulation budgets, $n = 50$ to 10,000. Three 12-chain BNNs, under MILE's default $\mathcal{N}(0, 1)$ prior, with a weight-prior scale chosen without training data (the data-free prior) and with a tempered likelihood, are compared with 12-member ensembles of NPE and of the BNN's own networks trained without sampling. Against single networks the BNN has the lower test negative log-likelihood (NLL) at every n, but at $n \leq 100$ ensembling the baselines matches this advantage. Against ensembles, the BNN does not beat NPE on average in the low-data regime: NPE is better at $n = 100$ by 0.18–0.29 nats, and at $n = 50$ only the BNN with the data-free prior is marginally better. The BNN beats NPE from $n = 500$ with tempering and from $n = 5{,}000$ without, by 0.11–0.20 nats. At small n the BNN is nonetheless the more reliable estimator: its test NLL varies 3–6 times less across training sets than NPE's, its worst training set is better, and it is never badly over-confident, which we argue matters more than the average when one estimator is trained once and applied to one observation. Under $\mathcal{N}(0, 1)$ the BNN is too wide at small n, almost entirely in its aleatoric term; against the exact posterior this is not irreducible noise, and we read it as epistemic uncertainty from a misspecified prior reported as aleatoric. The data-free prior removes most of the excess and helps most at the smallest n, where the scale of the prior is the dominant error; tempering helps most at intermediate n, where what remains is the isotropic Gaussian form of the prior.

Contents

1 Introduction **3**

2 Background **5**

2.1 Simulation-based inference and neural posterior estimation 5
2.2 Bayesian neural networks and deep ensembles 5
2.3 Sampling BNN posteriors with MILE 6
2.4 Tempering and the prior scale 7

3 The RBF-kernel inference task **8**

3.1 Simulator 8
3.2 Exact reference posterior 9
3.3 Datasets, training sizes and repeats 11
3.4 Why this task 11

4 Methods **13**
4.1 Common setup . . . 13
4.2 Gaussian predictor . . . 14
4.2.1 Single network . . . 14
4.2.2 12-network ensemble . . . 14
4.3 Neural posterior estimator . . . 15
4.3.1 Single flow . . . 15
4.3.2 12-flow ensemble . . . 15
4.4 Bayesian neural network . . . 15
4.4.1 Data-free prior scale . . . 16
4.4.2 Tempered likelihood . . . 16
4.4.3 Empirical Bayes . . . 17
4.5 Our MILE implementation . . . 17
4.6 Hyperparameter selection . . . 18
4.7 Software, compute and use of language models . . . 18

5 Evaluation protocol **19**
5.1 Metrics . . . 19
5.2 Convergence diagnostics . . . 19
5.3 Paired comparisons . . . 20

6 Experiments **20**
6.1 Sampling and ensembling together: the BNN against single networks . . . 21
6.2 Sampling alone: the BNN against 12-member ensembles . . . 22
6.3 Sampling and ensembling inside the BNN . . . 24
6.4 Adjusting the prior and the likelihood . . . 24

7 Discussion **25**
7.1 Why sampling, and not ensembling, buys consistency . . . 25
7.2 At small n the aleatoric term is not aleatoric . . . 25
7.3 The scale and the form of the prior . . . 26
7.4 What MCLMC does at large n, and how to judge convergence . . . 26
7.5 Relation to BNNs for simulation-based inference . . . 27
7.6 Limitations . . . 27
7.7 Next steps . . . 27

A Further results on the final runs **32**
A.1 Paired differences . . . 32
A.2 Inside the BNN: chains, draws and starting networks . . . 32

A.3 NPE head and encoder width . . . 36
A.4 Fairness of the comparison . . . 36
A.5 Width and coherence . . . 37
A.6 Moments and calibration . . . 38
A.7 Empirical Bayes and the re-tuning sampler . . . 39

B Reproducibility **46**

C Hyperparameters and compute **48**

D Implementation of the correlated `bde` model **51**

E Background details **52**
E.1 Microcanonical Langevin Monte Carlo . . . 52
E.2 Tempering . . . 52
E.3 Empirical Bayes and data-free calibration of the prior scale . . . 53

F Reference posterior and NLL details **54**

G Further prior and sampler variants **55**
G.1 Prior scale: data-free width and empirical Bayes . . . 55
G.2 Prior shape, and our claim on priors versus tempering . . . 55
G.3 Calibration of all variants . . . 57
G.4 Other variants . . . 58

H Convergence diagnostics in detail **59**
H.1 Role of convergence diagnostics for BNNs . . . 59
H.2 Pooled R-hat is not an acceptance test for BNNs . . . 60
H.3 Periodic re-tuning of MILE's step size and L . . . 62

I Negative results and pipeline bugs **62**

1 Introduction

Many sciences describe phenomena with simulators whose likelihood is intractable but from which data are easy to draw [1]. Simulation-based inference (SBI) turns this around and learns the map from data back to parameters from simulated pairs. In neural posterior estimation (NPE), its most widely used form, a conditional density estimator, typically a normalizing flow, is trained on pairs (θ, x) to approximate the posterior directly [2, 3]. Cosmology is one of the fields where this matters most. In the simplest, Gaussian setting a cosmological field depends on its parameters only through its power spectrum [4], and realistic field simulations are expensive enough that training sets are often small. Even CAMELS (Cosmology and Astrophysics with Machine Learning Simulations), the

largest suite of cosmological hydrodynamic simulations built for training machine-learning models, contains only a few thousand simulations [5].

Small training sets raise a problem that NPE does not address. An estimator that has seen few simulations is itself uncertain about the inverse map, and that uncertainty ought to widen the posterior it reports. A standard NPE has no way to express it, since it returns a single fitted density whatever the budget, and neural SBI algorithms are known to produce over-confident posteriors [6]. In scientific use this is the more harmful kind of error, because an over-confident posterior excludes parameter values that the data do not rule out. Ensembling several estimators is one remedy, and Hermans et al. [6] recommend it.

Bayesian neural networks (BNNs) offer a more direct one. By placing a posterior over the network's weights, a BNN adds to its predictive an epistemic term, the uncertainty about the network itself, which should be large when the training set is small and shrink as it grows [7]. Two recent studies have compared BNNs with NPE. Delaunoy et al. [8] make the NPE flow itself Bayesian and find it conservative down to very small budgets, where NPE is over-confident, although it becomes accurate more slowly than NPE as the budget grows. Poh et al. [9], working with several hundred thousand simulations, find NPE more accurate and better calibrated than their BNN, which is under-confident. In both studies the BNN is fitted by variational inference, and neither compares the results with a reference posterior. It therefore remains open how a BNN whose posterior is sampled rather than approximated would fare against NPE, and how close either comes to the true posterior.

Sampling BNN posteriors by Markov chain Monte Carlo (MCMC) has long been considered impractical, because posteriors over network weights are high-dimensional and multimodal [10, 11]. Microcanonical Langevin Ensembles (MILE) have recently made it practical at moderate cost [12]. MILE first trains a deep ensemble, a set of independently trained networks, and then starts a short Microcanonical Langevin Monte Carlo (MCLMC) chain [13] from each member. This construction affects how a MILE BNN should be evaluated. Deep ensembles, which average the predictions of their members, are themselves the standard non-Bayesian way to obtain epistemic uncertainty, and they can be read as an approximation to Bayesian model averaging [14, 15]. A MILE BNN is therefore both an ensemble and a sampler, and comparing it with a single network credits sampling with improvements that ensembling alone would provide. To isolate what sampling contributes, the baselines have to be ensembled to the same size, and the comparison needs a control made of the BNN's own networks without sampling.

Question. *Do BNNs sampled with MILE perform better than NPE when the simulation budget is small?* We study this question on a toy version of cosmological field inference for which the exact posterior is available (Section 3.4). The task is to infer the amplitude and length scale of a radial basis function (RBF) kernel, which together set the power spectrum of a Gaussian field, from the empirical covariance of ten noisy draws of the field. We train three methods on 50 to 10,000 simulations, each as a 12-member ensemble. The first is a BNN sampled with MILE, the second is NPE, and the third is a Gaussian predictor, which consists of the BNN's own networks trained to convergence and not sampled. Comparing the BNN with the Gaussian predictor isolates what sampling adds, and comparing it with NPE compares the two approaches at equal ensemble size. All methods are scored mainly by the test negative log-likelihood (NLL) of the true parameter, which does not require the posterior. We use the exact posterior in the appendices to check each method's mean, width and calibration, and to examine how the BNN divides its uncertainty into aleatoric (irreducible) and epistemic parts.

Contributions. Our first contribution is the comparison itself, in which every method is ensembled to the same size so that the effects of ensembling and of sampling can be told apart. Against a single network the BNN is better at every budget, but at small budgets ensembling the baselines removes most of that advantage (Sections 6.1 and 6.2). Second, we find that at small budgets the BNN does not beat an NPE ensemble on average, but it is the more reliable estimator. Its test NLL varies several times less from one training set to the next, and it is never badly over-confident. We argue that this reliability is what matters when an estimator is trained once and applied to a single observation (Section 7). At large budgets the BNN also wins on average, which is the opposite of what has been reported for variational BNNs. Third, under MILE's default prior the BNN is too wide at small budgets, and almost all of the excess lies in its aleatoric term. The exact posterior shows that this extra width is not irreducible noise, and we interpret it as epistemic uncertainty, caused by a misspecified prior, that the network reports as aleatoric (Sections 6.4 and 7.2). Finally, the scale and the form of the prior matter at different budgets. A prior scale chosen without training data removes most of the excess width and helps most at the smallest budgets, while tempering the likelihood helps most at intermediate ones. This pattern is consistent with a prior whose scale can be set correctly but whose isotropic Gaussian form is misspecified (Section 7.3).

2 Background

2.1 Simulation-based inference and neural posterior estimation

A simulator defines a prior $p(\theta)$ and a way to draw data $x \sim p(x \mid \theta)$ whose density is intractable; the goal is the posterior $p(\theta \mid x) \propto p(x \mid \theta)\, p(\theta)$ for an observed x [1]. Neural posterior estimation [2] fits a conditional density $q_\phi(\theta \mid x)$ to simulated pairs $(\theta_i, x_i) \sim p(\theta)\, p(x \mid \theta)$ by maximizing $\sum_i \log q_\phi(\theta_i \mid x_i)$. In expectation this objective equals the negative expected Kullback–Leibler (KL) divergence $-\mathbb{E}_{p(x)} \operatorname{KL}\big(p(\theta \mid x) \,\|\, q_\phi(\theta \mid x)\big)$ up to a constant, so a sufficiently flexible q_ϕ recovers the posterior, for every x at once (amortization). The usual family is a normalizing flow, e.g. a masked autoregressive flow, "a stack of autoregressive models, each modelling the random numbers of the next" [3].

With a finite budget the objective is a sample average over n pairs, and the fitted ϕ carries no uncertainty of its own: the estimator reports a single density whatever n is. Neural SBI algorithms "can yield overconfident posterior approximations", and ensembles of estimators are more reliable [6]. Evaluation matters as much as the algorithm, since "the choice of performance metric is critical" [16]. Without the true posterior, calibration is tested from samples: simulation-based calibration (SBC) checks the ranks of true parameters among posterior draws [17], and TARP (Tests of Accuracy with Random Points) is a coverage test that is necessary and sufficient for an accurate estimator [18]. Here the true posterior is available (Section 3.2). The main comparison does not need it, since the test NLL uses only the true parameter; the appendices use it to compare each estimator's moments, width and calibration with the posterior directly.

2.2 Bayesian neural networks and deep ensembles

A BNN places a prior $p(\mathbf{w})$ on the weights of a network defining $p(y \mid x, \mathbf{w})$, and predicts with the posterior predictive

$$p(y \mid x, \mathcal{D}) = \int p(y \mid x, \mathbf{w})\, p(\mathbf{w} \mid \mathcal{D})\, \mathrm{d}\mathbf{w}, \qquad p(\mathbf{w} \mid \mathcal{D}) \propto p(\mathcal{D} \mid \mathbf{w})\, p(\mathbf{w}) \tag{1}$$

[19, 20]. With a heteroskedastic likelihood, one whose predicted width depends on the input, the predictive variance splits into an aleatoric part, the expected predicted noise, and an epistemic part, the spread of the predicted means over the posterior (Eq. (10)); how the BNN's split behaves on this task is examined in Appendix A.5.

A non-Bayesian alternative is the deep ensemble: independently trained networks averaged at prediction, which is "simple to implement, readily parallelizable" and gives "well-calibrated uncertainty estimates which are as good or better than approximate Bayesian NNs" [14]; deep ensembles also "provide an effective mechanism for approximate Bayesian marginalization" [15]. MILE starts its chains from a deep ensemble (Section 2.3), and the baselines are also run as ensembles to separate the effect of sampling from that of ensembling.

2.3 Sampling BNN posteriors with MILE

MCMC targets $p(\mathbf{w} \mid \mathcal{D})$ exactly in the limit, but BNN posteriors are high-dimensional, multimodal and non-identifiable: permuting hidden units or rescaling between layers leaves the function unchanged. Papamarkou et al. [10] trace the resulting obstacles to a "lack of convergence to the parameter posterior", yet show that a non-converged chain "can yield via Bayesian marginalization a valuable posterior predictive distribution". Full-batch Hamiltonian Monte Carlo (HMC) chains [11] disagree in weight space but mostly agree in function space, and Sommer et al. [21] propose Bayesian deep ensembles, chains initialized at the members of a deep ensemble, since "multiple samples from multiple chains improve predictive performance and uncertainty quantification". Cost is the other obstacle: the No-U-Turn Sampler (NUTS), the adaptive form of HMC, "often requires more than 90k leapfrog steps/gradient evaluations for warmup in BNN tasks" [12].

MCLMC [13] samples with a unit-speed velocity whose dynamics leave the target invariant for any noise level. It is integrated with a step size ϵ, the velocity is partially refreshed over a decoherence length L, and there is no Metropolis–Hastings correction, so a finite ϵ biases the stationary distribution; the bias is controlled by tuning ϵ to a target energy-error variance per dimension (EEVPD; Appendix E). MILE [12] (Figure 1) turns MCLMC into a sampler for BNN posteriors. It decomposes the error of the Monte Carlo predictive

$$p(\mathbf{y}^* \mid \mathbf{x}^*, \mathcal{D}) \approx \frac{1}{KS} \sum_{k=1}^{K} \sum_{s=1}^{S} p\big(\mathbf{y}^* \mid \mathbf{x}^*, \mathbf{w}^{(k,s)}\big) \tag{2}$$

into initialization, discretization and Monte Carlo error. Against the first, K networks are optimized independently (a deep ensemble, 14; AdamW with early stopping) and chain k starts from member k, so the ensemble explores across modes and each chain explores its own. Against the second, the initial step size is the optimizer's learning rate and the EEVPD target is annealed linearly from 0.5 to 0.1 over the first tuning phase; L is then set from the chain's variance and its effective sample size (ESS; target 100). Against the third, a fixed budget per chain: 40,000/5,000/5,000 steps in the three tuning phases, then 10,000 sampling steps thinned by 10 to $S = 1{,}000$ draws. Defaults are $K = 12$ and an isotropic $\mathcal{N}(0, 1)$ prior on all weights. MILE drops the Metropolis correction deliberately and does not claim convergence: short chains, no correction and ensemble initialization "will induce a bias in the sampling distribution"; its authors report a chain-wise split $\widehat{R}$ (the potential scale reduction factor, which compares the variance between chains with that within them and is 1 for chains that agree; Section 5.2) on the weights above the 1.1 and 1.01 cutoffs and justify the method by predictive performance [12, App. A.3]. The Gaussian predictor used here as a control is MILE's first stage alone: the same $K = 12$ networks, with the same seeds, data

and hyperparameters, optimized but not sampled. Its networks are trained to convergence, whereas the BNN's chains start from networks pretrained under MILE's shorter budget (Section 4.5), so the BNN and the Gaussian predictor differ in the sampling stages and in how long the starting networks were optimized.

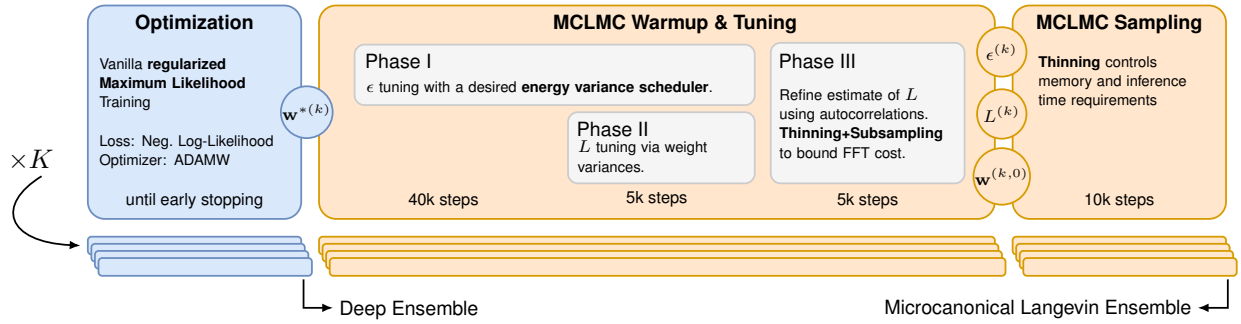


Figure 1: The MILE procedure: optimization of K ensemble members, MCLMC warmup and tuning (phases I–III, step counts annotated), and sampling. The quantities carried between stages are circled: the optimized weights $\mathbf{w}^{*(k)}$ of member k, and the step size $\epsilon^{(k)}$, decoherence length $L^{(k)}$ and initial state $\mathbf{w}^{(k,0)}$ of its chain. Adapted from Sommer et al. [12, Fig. 1] under CC BY 4.0, redrawn with this report's notation ($\mathbf{w}$ for network weights, θ for the inferred parameters).

2.4 Tempering and the prior scale

With the posterior energy $U(\mathbf{w}) = -\sum_{i=1}^{n} \log p(y_i \mid x_i, \mathbf{w}) - \log p(\mathbf{w})$, the Bayes posterior is $p(\mathbf{w} \mid \mathcal{D}) \propto \exp(-U(\mathbf{w}))$. Wenzel et al. [22, App. E] distinguish two tempered versions at temperature $T > 0$:

$$\text{fully tempered:} \quad p_F(\mathbf{w} \mid \mathcal{D}) \propto \exp\big(-U(\mathbf{w})/T\big) = p(\mathcal{D} \mid \mathbf{w})^{1/T}\, p(\mathbf{w})^{1/T}, \tag{3}$$

$$\text{partially tempered:} \quad p_P(\mathbf{w} \mid \mathcal{D}) \propto p(\mathcal{D} \mid \mathbf{w})^{1/T}\, p(\mathbf{w}) = \exp\Big(\log p(\mathbf{w}) + \tfrac{1}{T}\textstyle\sum_i \log p(y_i \mid x_i, \mathbf{w})\Big). \tag{4}$$

$T = 1$ is the Bayes posterior and $T < 1$ a "cold" posterior. A cold posterior sharpens the Bayes posterior, "overcounting the data by a factor of $1/T$" [22, Sec. 2.2]; under full tempering the prior is also rescaled to $p(\mathbf{w})^{1/T}$. Their experiments use full tempering. This project uses partial tempering, Eq. (4). Appendix E relates it to the KL-weighted evidence lower bound of variational BNNs, used by Delaunoy et al. [8], and shows that no prior reproduces it: the prior that would turn a partially tempered posterior into a $T = 1$ posterior depends on the training data.

Per-observation form. For the bivariate Gaussian head of Eq. (9), direct algebra gives

$$\mathcal{N}_2(\theta; \boldsymbol{\mu}, \Sigma)^{1/T} \;\propto\; \mathcal{N}_2(\theta; \boldsymbol{\mu}, T\Sigma) \cdot \det(\Sigma)^{-\frac{1}{2}\left(\frac{1}{T}-1\right)}, \tag{5}$$

with a constant independent of the weights. At $T < 1$ each training pair therefore enters the sampled posterior through a Gaussian of covariance $T\Sigma$, narrower than the network's own, together with a factor that grows as the predicted covariance shrinks. Tempering thus pulls the predicted mean towards the training targets and the predicted width towards zero; the prior and the remaining residual term resist the second pull. The temperature acts only in the MCLMC target, during

warmup and sampling; pretraining and prediction are untempered, so the test NLL of a tempered BNN is directly comparable with that of the other methods (Appendix E).

The cold posterior effect. Wenzel et al. [22] find that predictive performance of deep BNNs is best at $T < 1$ (for a convolutional and long short-term memory network, CNN-LSTM, on the IMDB movie-review sentiment task, roughly $0.01 < T < 0.2$), and state that they "are not aware of any published work demonstrating well-performing Bayesian deep learning at temperature $T = 1$". Proposed explanations include data augmentation, under which Izmailov et al. [11] find tempering "not needed for near-optimal performance", and prior misspecification: heavy-tailed weight priors remove the effect in fully connected classifiers on the MNIST and FashionMNIST image datasets, though not in convolutional or residual networks [23]. Noci et al. [24] report that the effect "becomes stronger for smaller dataset size n", since "the relative influence of the prior over the likelihood increases with decreasing dataset size". The task here has no data augmentation and is studied precisely at small n, where these accounts predict the effect to be strongest if the prior is the cause. Section 6.4 tests whether changing the prior can replace tempering.

Prior scale. The scale of an isotropic prior $\mathcal{N}(\mathbf{w}; 0, \tau^2 I_d)$ can be set from the training data or without it. Empirical Bayes (type-II maximum likelihood) maximizes the evidence $p(\mathcal{D} \mid \tau)$, whose stationary point is MacKay's condition $\hat{\tau}^2 = \|\mathbf{m}\|^2/\gamma$, with $\mathbf{m}$ the posterior mean and $\gamma \in [0, d]$ "the number of good parameter measurements" [20, 25]. MacKay himself uses separate scales for hidden weights, hidden biases and output weights, since forcing one decay rate on all of them is "inconsistent" [25]. With a sampled posterior the expectations are Monte Carlo estimates: stochastic-approximation updates with decreasing steps alternate with stochastic-gradient MCMC (SG-MCMC) in Deng et al. [26], and the same update, estimated from posterior samples, oscillates or diverges when applied per layer [27]. The marginal likelihood also "has a bias towards underfitting" that weakens as data grow [28], which matters most at the smallest n. A data-free alternative chooses the τ whose prior predictive covers the simulator's parameters with calibrated coverage, a single-scale version of the prior optimization of Delaunoy et al. [8]; it needs simulations outside the training budget, here 3,000: 1,000 pairs scored for coverage and 2,000 for standardizing the inputs. A condition that needs none follows from the law of total variance: any coherent posterior satisfies $\mathbb{E}_x[\mathrm{Var}(\theta \mid x)] + \mathrm{Var}_x(\mathbb{E}[\theta \mid x]) = \mathrm{Var}(\theta)$, the second moment of the self-consistency that simulation-based calibration tests [17], and requiring it of the untrained network on the n training inputs fixes τ from the prior's moments alone. All three fit the network's prior predictive to the simulator's joint distribution. For the first two, a well-specified model should select the same scale (derivation and caveats in Appendix E); on this task all three roughly agree, within about 12%, at n = 100–1,000 (Section 4.4.1, Appendix A.7). Agreement fixes the scale within the isotropic Gaussian family, so the scale is well specified relative to MILE's default $\mathcal{N}(0, 1)$; it does not test whether an isotropic Gaussian is the right form of prior (Section 7.3).

3 The RBF-kernel inference task

3.1 Simulator

The parameter $\theta = (\sigma, \rho)$ is the amplitude and length scale of a squared-exponential (RBF) kernel [29], with independent truncated-normal priors

$$\sigma \sim \mathcal{N}(1.25,\, 0.3^2)\big|_{\sigma>0}, \qquad \rho \sim \mathcal{N}(0.60,\, 0.2^2)\big|_{\rho>0} \tag{6}$$

(the truncation removes 1.5×10^{-5} and 1.3×10^{-3} of the prior mass). One simulated dataset is $N = 10$ independent draws of a zero-mean process observed at $D = 10$ equally spaced points $t_1, \ldots, t_D \in [0, 1]$ with Gaussian noise $\eta = 0.05$:

$$x_i \overset{\text{iid}}{\sim} \mathcal{N}(0,\, C_\theta), \qquad (C_\theta)_{jk} = \sigma^2 \exp\Big(- \frac{(t_j - t_k)^2}{2\rho^2} \Big) + \eta^2 \delta_{jk}, \qquad i = 1, \ldots, N. \tag{7}$$

Every method sees only the empirical covariance $S = X^\top X / N$ ($X \in \mathbb{R}^{N \times D}$), flattened to 100 numbers (55 distinct). Since the x_i are zero-mean Gaussian, S is sufficient for θ, so the summary loses no information. Figure 2 shows draws and S at three length scales.

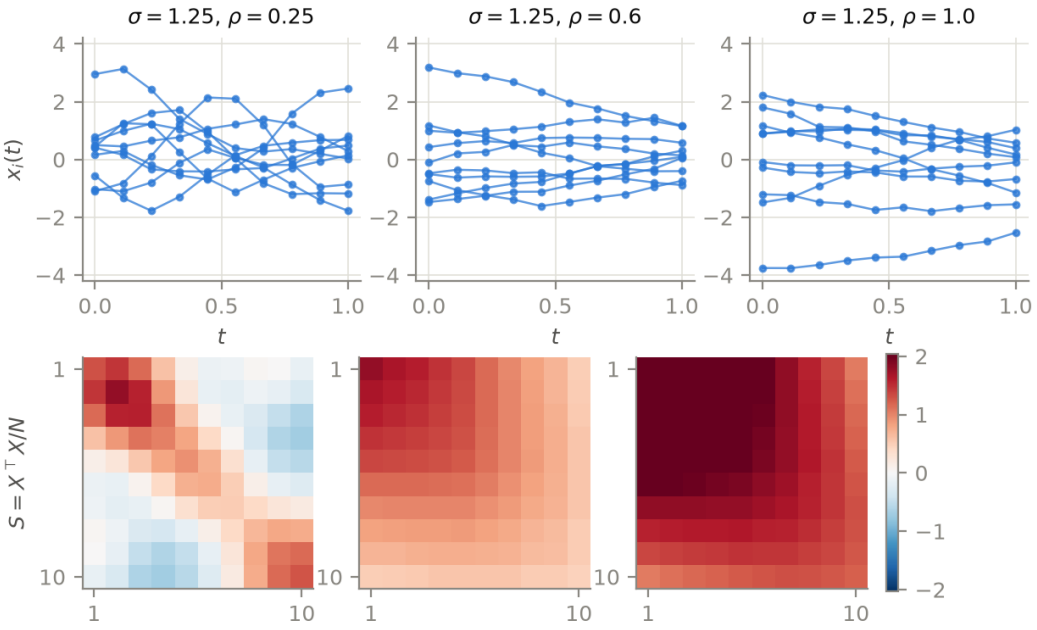


Figure 2: Top: the $N = 10$ draws of one dataset at $\sigma = 1.25$ and $\rho = 0.25$, 0.60, 1.00. Bottom: the summary S each method receives. Draws re-simulated from the model for illustration.

3.2 Exact reference posterior

With S sufficient, the log-likelihood is

$$\log p(S \mid \theta) = -\frac{N}{2} \Big[\log\det C_\theta + \operatorname{tr}\big(C_\theta^{-1} S\big) \Big] + \text{const}, \tag{8}$$

and the posterior $p(\theta \mid S) \propto p(S \mid \theta)\, p(\theta)$ is evaluated on a 240×240 grid, $\sigma \in [0.05, 2.9]$, $\rho \in [0.02, 1.6]$ (spacings 0.0119 and 0.0066), and normalized on the grid. Reference moments are grid sums; $\log p(\theta^* \mid S)$ at the true parameter is bilinearly interpolated; reference draws are sampled per cell with uniform jitter inside the cell.

On the 1,000 test datasets (Figures 3, 4) the posterior is informative and correlated. Its standard deviation is 0.153 for σ (RMS; 5–95% range 0.121–0.169) and 0.054 for ρ (0.015–0.090), a median variance reduction relative to the prior of 73% and 94%. The width grows with the parameter itself (correlation of posterior std with the true value 0.61 for σ, 0.91 for ρ). σ and ρ are positively

correlated a posteriori in every test dataset (median 0.56, 5–95% 0.44–0.65), so a predictive with independent marginals is misspecified here. The marginals are mildly skewed (median skewness 0.42 for σ, 0.14 for ρ). As a check, the reference is calibrated on the test set, as it must be when θ is drawn from the prior: the standardized error $(\mathbb{E}[\theta \mid S] - \theta^*)/\mathrm{sd}[\theta \mid S]$ has standard deviation 1.02 / 0.97, and 69% / 71% of truths lie within one posterior standard deviation. It also satisfies the law-of-total-variance condition of Section 2.4: the mean posterior variance plus the variance of the posterior mean is 0.0939 for σ and 0.0379 for ρ, against test-set variances of θ^* of 0.0935 and 0.0376.

A per-dataset 300×300 refinement of the grid changes none of these summaries; only the two lowest-ρ test datasets change materially (Appendix F).

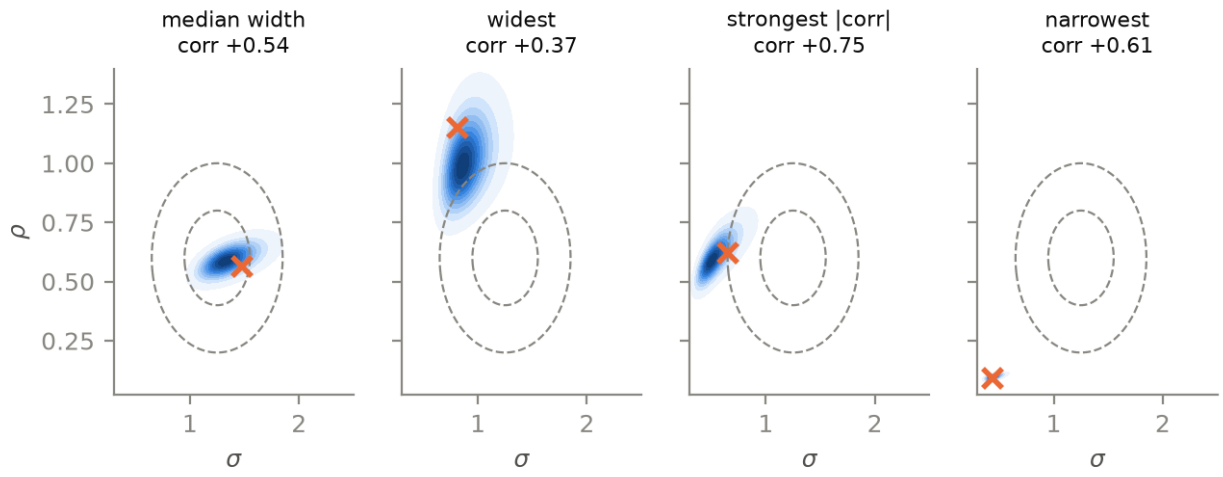


Figure 3: Exact posterior (blue, normalized to its maximum) for four test datasets: median joint width, widest, strongest correlation, narrowest. Dashed: prior at 1 and 2 standard deviations. Cross: true θ.

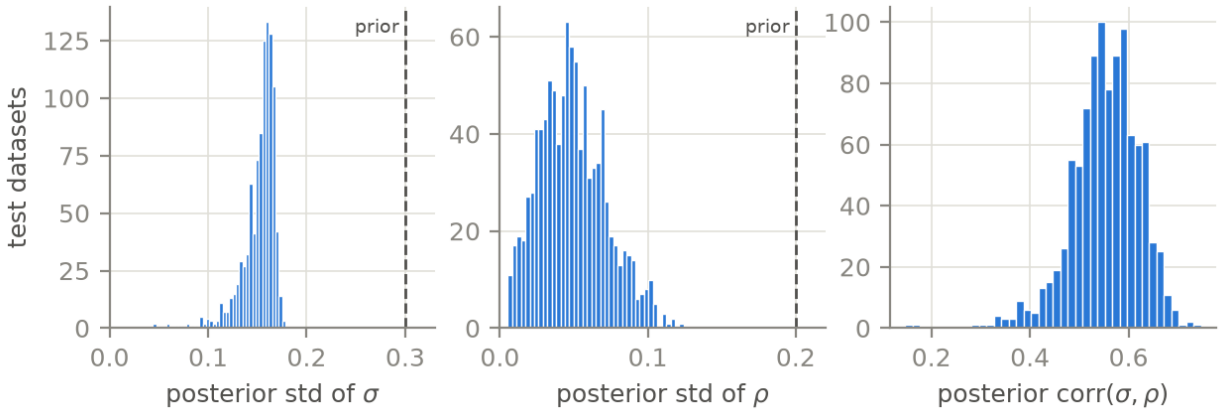


Figure 4: Across the 1,000 test datasets: posterior standard deviation of σ and ρ (dashed: prior standard deviation) and posterior correlation of (σ, ρ).

3.3 Datasets, training sizes and repeats

One canonical dataset is generated once (JAX key 0, split into 11,000 subkeys): a pool of 10,000 training pairs (S, θ) and 1,000 test pairs. The test set is shared by every method, size and repeat. Training sets of size $n \in \{50, 100, 500, 1{,}000, 5{,}000, 10{,}000\}$ are nested subsamples of the pool: repeat r takes the first n rows of one random permutation (NumPy generator seeded $123+r$), so the $n = 50$ set is contained in the $n = 100$ set of the same repeat. All methods at a given (n, r) train on identical rows, which makes every comparison paired. Each fit's own seed is $100\,j+r$, with j the index of n in (10,000, 5,000, 1,000, 500, 100, 50). Results use repeats $r = 0, \ldots, 19$ at $n \leq 100$ and $r = 0, \ldots, 4$ at larger n; the chain-wise diagnostics of Appendix H.2 use repeats 0–4 at every n. Because the pool has 10,000 rows, repeats are nearly disjoint at small n (two repeats share 1% of their rows at $n = 100$ and 10% at $n = 1{,}000$, on average) but share half their rows at $n = 5{,}000$ and all of them at $n = 10{,}000$, where repeats differ only in seeds and row order.

3.4 Why this task

The exact posterior is known for every test dataset. Because S is sufficient and $p(S \mid \theta)$ is Gaussian in closed form (Section 3.2), $p(\theta \mid S)$ itself is available on all 1,000 test datasets. The main comparison does not need it: methods are scored mainly on the test NLL, which uses only the simulated θ^* (Section 5.1). The exact posterior serves the appendices. Calibration tests that use only θ^* (TARP, SBC) are passed by the prior: a predictive equal to $p(\theta)$ has exact coverage when θ^* is drawn from it. They detect a predictive that is too narrow or too wide on average, but not how much wider than the posterior a calibrated predictive is. With the exact posterior, each method's mean and width are compared with the posterior's, and the width warranted by the method's own mean error is computable, and the BNN's split into aleatoric and epistemic uncertainty can be checked (Appendices A.5 and A.6). Of the two questions of this report, whether a BNN beats NPE at small n is answered by the test NLL; whether the width it adds there is warranted needs the exact posterior.

The posterior width varies across datasets. Over the central 90% of test datasets the posterior standard deviation ranges from 0.121 to 0.169 for σ and from 0.015 to 0.090 for ρ, and it grows with the parameter (correlation with the true value 0.61 and 0.91). A predictive with one width for all inputs is wrong for most datasets, so each method must learn a dataset-dependent width from n examples. In the BNN this is the role of the aleatoric head, while the epistemic term should vanish as n grows; the exact target lets this split be checked (Appendix A.5).

The posterior has dependence and mild skew. σ and ρ are positively correlated a posteriori in every test dataset (median 0.56), so a predictive with independent marginals is misspecified, and the marginal of σ is right-skewed (median skewness 0.42; 0.14 for ρ). The BNN and the Gaussian predictor have a bivariate Gaussian head, which represents the dependence but not the skew; NPE's flow can represent both. Both departures from an independent Gaussian are moderate, so no method is badly misspecified, and differences that come from the predictive family are visible against the exact moments rather than hidden by a gross misfit.

The summary is fixed and sufficient. Every method receives the same 100 numbers S, which lose no information about θ (Section 3.1). Differences between methods therefore come from the

inference model and its uncertainty, not from learning a summary of the $N = 10$ replicated draws, which is a separate difficulty for neural estimators [30].

Simulation and evaluation are cheap. One simulation is ten draws from a 10-dimensional Gaussian, and one exact posterior is a 240×240 grid evaluation. Training sets can therefore go down to $n = 50$, every fit can be repeated on nearly independent training sets at small n (Section 3.3), and every comparison is paired on the same 1,000 test datasets. The prior is informative but much wider than the posterior (median variance reduction 73% for σ and 94% for ρ), so a method that returns the prior is far from the target, and a method's width at small n measures how much of the information in S it has learned.

Precedent. Inferring the amplitude and length scale of a squared-exponential covariance from Gaussian-process data is a long-standing problem with a closed-form likelihood. Williams and Rasmussen [31] fit these hyperparameters by optimization or average over them with hybrid Monte Carlo, Neal [32] samples them with Markov chain methods, and Rasmussen and Williams [29] give the standard treatment. Murray and Adams [33] illustrate the posterior over the length scale of a squared-exponential kernel given draws from the prior, the closest classical analogue of this task. Bayesian inference of covariance parameters is equally established in spatial statistics, for the Matérn class [34]. Approximate Bayesian computation uses tractable Gaussian processes summarized by covariances as test problems: the moving-average model of order 2, MA(2), with autocovariance summaries, compared with exact-likelihood contours [35], and an Ornstein–Uhlenbeck process scored against its true posterior [36]. Recent amortized methods take the same targets: prior-fitted networks are checked against exact Gaussian-process (GP) posterior predictives with an RBF kernel [37], the Amortized Conditioning Engine (ACE) infers the length scale and output scale of a one-dimensional GP [38], Bitzer et al. [39] amortize type-II maximum likelihood for kernels built from squared-exponential components, and Zammit-Mangion et al. [40] compare amortized posterior methods, including a normalizing-flow NPE, with MCMC on a Gaussian-process length scale over 1,000 test datasets. None of the ten tasks of the standard SBI benchmark infers covariance parameters [16].

Relation to cosmological field inference. A stationary covariance function and its spectral density, or power spectrum, are Fourier duals (the Wiener–Khintchine theorem), and the spectral density of the squared-exponential kernel in d dimensions is $\sigma^2(2\pi\rho^2)^{d/2}\exp(-2\pi^2\rho^2 s^2)$ at frequency s [29, Sec. 4.2.1]. Each draw x_i is therefore a zero-mean Gaussian field whose parameters enter only through its power spectrum: σ sets the amplitude and ρ the width in frequency, and the empirical covariance S is the real-space counterpart of a measured power spectrum. This is the simplest setting of cosmological parameter inference from fields. Makinen et al. [4] infer the amplitude A and the scale dependence B of a power spectrum $P(k) = Ak^{-B}$ from two-dimensional Gaussian random fields, which "are relevant to early cosmological density fields such as the Cosmic Microwave Background", and compare simulation-based inference with the exact field-level posterior, as is done here. The present task differs in the shape of the spectrum, Gaussian rather than a power law in frequency, and in the data, ten noisy points of a one-dimensional field. The simulator extends without change to images and three-dimensional volumes by placing the same kernel on a two- or three-dimensional grid; the likelihood stays Gaussian, and on a periodic grid its covariance is diagonal in Fourier space [4], so the exact posterior remains available as a reference at larger scale. Beyond Gaussian fields, as for late-time galaxy clustering, the power spectrum is no longer sufficient [4], and a fixed sufficient summary such as S is no longer available.

Limitations. The tractable likelihood is what provides the exact reference and is the point of the task, not a limitation. The main limitation is that all results come from this single task (Section 7.6). Its dimension is also small: θ has two components and each dataset 55 distinct numbers, whereas analyses of cosmological fields infer more parameters from far larger data, and how the comparison changes with the dimension of θ and of the data is not tested. The summary is fixed and sufficient, so the methods are compared as posterior estimators only; on fields, a summary or encoder must also be learned [4], and how that interacts with sampling the network's weights is not addressed. The posteriors have moderate dependence and skew but are close to Gaussian. This does not favour the Gaussian heads of the BNN and the Gaussian predictor: NPE with a bivariate Gaussian head in place of its flow is worse than with the flow (Appendix A.3). Near-Gaussian posteriors are also the regime of well-constrained cosmological parameters, whose posteriors from Planck "can be well approximated as Gaussian for ΛCDM, as well as for many one- and two-parameter extensions" [41]; strongly non-Gaussian posteriors, which arise there for curvature, the dark-energy equation of state or parameters bounded at zero [41], are not covered. The design is otherwise fixed: one kernel family, fixed observation points and a fixed noise level. The training pool is 10,000 pairs, so repeats share training rows at large n, and the spread over repeats there reflects seeds more than training sets.

4 Methods

4.1 Common setup

Each method maps the summary $S \in \mathbb{R}^{100}$ to a predictive density $q(\theta \mid S)$ over $\theta = (\sigma, \rho)$, trained on n pairs (S_i, θ_i) and evaluated on the shared 1,000 test datasets. Every method is a 12-member ensemble, and at each (n, r) all methods receive the same n training rows (Section 3.3). All training is full-batch. Within those rows, 20% are held out for early stopping (10 of 50 at $n = 50$), and every member of an ensemble uses its method's split. The BNN and the Gaussian predictor share `bde`'s split (`train_test_split` with a fixed random state); NPE uses a permutation seeded with the fit's seed. The two validation sets overlap only as much as independent random splits would (on average 1.9 of 10 rows at $n = 50$ and 4.3 of 20 at $n = 100$), so the split adds noise to the paired comparisons but no systematic difference. The held-out rows serve only for early stopping: the Gaussian predictor and NPE are fitted to 80% of the training rows, whereas the BNN's sampling stage conditions on all n (Section 4.4), so the BNN uses 25% more rows in its final fit at every n. The BNN and the Gaussian predictor standardize inputs and targets by their mean and standard deviation over the n training rows, so that an $\mathcal{N}(0, 1)$ weight prior is on the right scale, and map predictions back; NPE sees raw S and θ.

Gaussian heads. The Gaussian predictor and the BNN output five numbers $(\mu_\sigma, \mu_\rho, a_\sigma, a_\rho, c)$ per input, defining a bivariate normal

$$q(\theta \mid S) = \mathcal{N}_2\big(\theta;\, \boldsymbol{\mu},\, \Sigma\big), \qquad \Sigma = \begin{pmatrix} s_\sigma^2 & r\, s_\sigma s_\rho \\ r\, s_\sigma s_\rho & s_\rho^2 \end{pmatrix}, \qquad s_\bullet = \mathrm{softplus}(a_\bullet) + 10^{-6}, \quad r = \tanh(c). \tag{9}$$

Both are trained or sampled with this heteroskedastic likelihood; the correlation is fitted, since the exact posterior is correlated in every test dataset (Section 3.2).

Uncertainty decomposition. A BNN prediction is the equal-weight mixture of the Gaussians of

its C posterior draws ($C = 12 \times 1{,}000$; Eq. 2). Per parameter, by the law of total variance,

$$\bar{\mu} = \tfrac{1}{C}\textstyle\sum_c \mu_c, \qquad \underbrace{\tfrac{1}{C}\textstyle\sum_c s_c^2}_{\text{aleatoric}^2} + \underbrace{\tfrac{1}{C}\textstyle\sum_c (\mu_c - \bar{\mu})^2}_{\text{epistemic}^2} = \text{total}^2, \tag{10}$$

following Kendall and Gal [7]. The Gaussian predictor is the same mixture over its 12 networks ($C = 12$), so its epistemic term is the spread of the networks' means; a single network ($C = 1$) has none. NPE has no such split. The NPE ensemble is the equal-weight mixture of its 12 flows: its log density is the log-mean-exp of the flows', its mean and standard deviation are the mixture's first two moments, and its calibration draws are taken in equal shares from each flow.

Predictive draws. Calibration metrics use 1,000 draws per test dataset: from the Gaussian mixture for the Gaussian predictor and the BNN, and from the flows for NPE (each flow's mean and standard deviation are taken from 5,000 of its draws). The log density at θ^* is exact for every method.

4.2 Gaussian predictor

The Gaussian predictor is the BNN's network without sampling: the network of Section 4.4 ($100 \to 16 \to 16 \to 16 \to 5$, soft-sign, the bivariate head of Eq. (9)), with the BNN's member seeds, training rows, early-stopping split, standardization, learning rate and weight decay. Each network is trained by AdamW [42] on the negative log of Eq. (9) to convergence. The validation NLL is evaluated every 100 steps, training stops after 100 evaluations without an improvement of 10^{-4} (after at least 500 steps), and the best parameters are kept. All 720 networks stopped early, none at the cap of 200,000 steps; the median last step is 11,900 at $n = 50$, rising to 33,400 at $n = 10{,}000$ (maximum 67,500).

The BNN's chains start from the same networks pretrained under MILE's budget (at most 2,000 steps, patience 5 evaluations; Section 4.5), and most of those stop at that cap: 148 of 240 networks at $n = 50$, none at $n = 100$, 37 of 60 at $n = 500$, all 60 at $n = 1{,}000$, and 48 of 60 at $n = 5{,}000$ and 10,000. This capped version is reported alongside the converged one. It isolates what sampling adds to the BNN's own starting point, while the converged predictor measures it against networks whose optimization has finished. Both versions are evaluated as a single network and as a 12-network ensemble.

4.2.1 Single network

The single network is member 0 of the 12, used alone ($C = 1$ in Eq. (10)). Its predictive is one bivariate Gaussian, so its width is aleatoric only, with no term for uncertainty about the network itself. It is compared with a single NPE flow at one fitted network each; the two differ in head, encoder, optimizer and hyperparameters, not only in head.

4.2.2 12-network ensemble

The ensemble is the equal-weight mixture of the 12 networks ($C = 12$), whose epistemic term is the spread of the networks' predicted means (Eq. (10)), and is a deep ensemble in the sense of Lakshminarayanan et al. [14]. It has the same ensemble size as the BNN and the NPE ensemble, so it is the comparison that separates the effect of sampling from the effect of ensembling (Section 1).

4.3 Neural posterior estimator

NPE is a conditional masked autoregressive flow [3] in `nflows` [43], trained as a neural posterior estimator [2]. A context encoder $g : \mathbb{R}^{100} \to \mathbb{R}^8$ (layers $100 \to 16 \to 16 \to 8$, ReLU) summarizes S. The flow maps a standard normal on $\mathbb{R}^2$ through two blocks, each a masked affine autoregressive transform (a masked autoencoder for distribution estimation, MADE, with 16 hidden units, one residual block, ReLU, conditioned on $g(S)$, scale softplus$(\cdot) + \varepsilon$) followed by reversing the variable order, so each parameter is transformed conditional on the other in one of the two blocks. It has 3,920 parameters in total (encoder 2,024, flow 1,896). Training maximizes $\sum_i \log q(\theta_i \mid S_i)$ with Adam [44] and coupled (L_2) weight decay at a per-size learning rate and weight decay (Section 4.6), for up to 20,000 steps, evaluating the validation NLL every 100 steps and stopping after 20 evaluations without an improvement of 10^{-4}; the best state is restored. Because training θ_i are prior draws, maximizing this objective targets the posterior [2]. The flow gives exact $\log q(\theta^* \mid S)$ and draws; it has no aleatoric/epistemic split. Like the Gaussian predictor, NPE is evaluated as a single flow and as a 12-flow ensemble. Two one-change variants, an encoder of the BNN's width ($100 \to 16 \to 16 \to 16$) and a bivariate Gaussian head in place of the flow, are in Appendix A.3.

4.3.1 Single flow

The single flow is one flow trained as above, with the fit's seed. It is member 0 of the ensemble below.

4.3.2 12-flow ensemble

The ensemble has twelve flows on the same training rows and early-stopping split, differing only in their initialization seeds ($s + 10{,}000\,k$ for member k, with s the fit's seed, so member 0 is the single flow). The ensemble is the equal-weight mixture of the 12 flows (Section 4.1), the deep-ensemble construction that Hermans et al. [6] find "provides more reliable approximations" and that Delaunoy et al. [8] use, with 5 members, as a baseline. It has the same ensemble size as the BNN and the Gaussian predictor ensemble.

Long-trained ensemble at $n = 10{,}000$. At the default budget the flows at $n = 10{,}000$ are under-trained: the one member whose stopping step was checked (repeat 0, member 0) ran to the 20,000-step cap with its best state at step 19,700 (the default-budget runs did not record stopping steps otherwise). A second ensemble at $n = 10{,}000$, repeats 0–4, is identical except for a cap of 200,000 steps and a patience of 100 evaluations. Every flow stopped early, with its best state at a median step of 30,000–51,000 depending on the repeat (at most 84,400 steps run). This long-trained ensemble is the NPE comparator at $n = 10{,}000$ wherever stated.

4.4 Bayesian neural network

Network. The network is a fully connected $f_{\mathbf{w}} : \mathbb{R}^{100} \to \mathbb{R}^5$ with three hidden layers of width 16, the soft-sign activation $x/(1 + |x|)$ on every hidden layer, and a linear output layer ($100 \to 16 \to 16 \to 16 \to 5$), with $d = 2{,}245$ weights and biases. Soft-sign is used because `bde` offers no tanh; it is smooth and saturating like tanh, with heavier tails. The five outputs parameterize the bivariate normal of Eq. (9) in standardized units.

Likelihood, prior, posterior. With $\tilde{S}_i, \tilde{\theta}_i$ the standardized training pairs,

$$p(\mathcal{D} \mid \mathbf{w}) = \prod_{i=1}^{n} \mathcal{N}_2\big(\tilde{\theta}_i;\, \boldsymbol{\mu}_{\mathbf{w}}(\tilde{S}_i),\, \Sigma_{\mathbf{w}}(\tilde{S}_i)\big), \qquad p(\mathbf{w}) = \prod_{j=1}^{d} \mathcal{N}(w_j; 0, 1), \qquad p(\mathbf{w} \mid \mathcal{D}) \propto p(\mathcal{D} \mid \mathbf{w})\, p(\mathbf{w}). \tag{11}$$

The $\mathcal{N}(0,1)$ prior on every weight and bias is MILE's default [12]. The variants below make one change each, everything else fixed: a prior scale chosen without training data, a tempered likelihood, and a scale chosen by empirical Bayes.

Prediction. The posterior is sampled with MILE (Section 2.3): K members, $S = 1{,}000$ kept draws each. The predictive for a test summary S^* is the equal-weight mixture over all KS draws (Eq. 2), mapped back to original units by $\boldsymbol{\mu} \mapsto \boldsymbol{\mu} \odot \mathbf{s}_y + \mathbf{m}_y$ and $s_\bullet \mapsto s_\bullet\, s_{y,\bullet}$ (training-set target mean $\mathbf{m}_y$ and standard deviation $\mathbf{s}_y$); the correlation r is unchanged, being invariant under a positive per-coordinate rescaling. Its first two moments split into aleatoric and epistemic parts as in Eq. (10). Mixing is assessed on these predictive quantities, never on raw weights, whose permutation and scaling symmetries make weight-space diagnostics uninterpretable [11, 21].

4.4.1 Data-free prior scale

The prior is $\mathcal{N}(0, \tau^2)$ on every weight and bias with $\tau = 0.3655$, chosen before any training data are seen. The prior predictive of the untrained network, with weights and biases drawn from $\mathcal{N}(0, \tau^2)$, is scored by TARP coverage [18] on 1,000 simulated pairs (θ, x) drawn independently of the canonical dataset (the same pairs in every trial), with 1,000 weight draws; inputs are standardized by the mean and standard deviation of 2,000 further simulated x, and outputs are mapped to θ with the prior mean and standard deviation. With $\mathrm{ECP}(\alpha)$ the expected coverage probability, the fraction of datasets whose true parameter lies inside TARP's credible region at credibility level α, which equals α for a calibrated estimator (Appendix A.6), τ minimizes the absolute area between the coverage curve and the diagonal, $\left|\int (\mathrm{ECP}(\alpha) - \alpha)\, \mathrm{d}\alpha\right|$, by Bayesian optimization (`scikit-optimize`, Gaussian-process surrogate, expected improvement, 40 trials, τ log-uniform on $[10^{-3}, 3]$). This is a single-scale version of the prior optimization of Delaunoy et al. [8]; it uses 3,000 simulations outside the training budget but no training pairs.

Simulation-free check. The same scale follows from the coherence condition of Section 2.4 without extra simulations. For each candidate τ (81 values, log-spaced on $[0.05, 2]$), the untrained network's predictive over 2,000 weight draws, standardized as in training, is evaluated on the n training inputs, and the ratio $\big(\overline{s_j^2} + \widehat{\mathrm{Var}}(\mu_j)\big)/V_j^*$ is computed for each parameter j, with the averages taken over weight draws and inputs and V_j^* the truncated-normal prior variance in standardized units. Each ratio crosses 1 exactly once; the selected scale minimizes the sum of the squared log ratios. It is $\tau = 0.377 \pm 0.053$ at $n = 50$, 0.364 ± 0.024 at $n = 100$, 0.353 ± 0.008 at $n = 500$ and 0.347 at $n = 1{,}000$ (mean $\pm$ sd over repeats), so the simulator-calibrated 0.3655 lies within the repeat spread at $n = 50$ and within 5% at larger n. The spread at $n = 50$ comes from standardizing the targets by the standard deviation of 50 draws: over repeats and parameters, the selected scale correlates at -0.998 with that standard deviation divided by the prior's. Under $\mathcal{N}(0,1)$ the untrained predictive implies about ten times the prior variance. The check sets the scale of no run.

4.4.2 Tempered likelihood

The likelihood is partially tempered (Eq. (4)): in warmup and sampling the MCLMC target is $\log p(\mathbf{w}) + \frac{1}{T} \log p(\mathcal{D} \mid \mathbf{w})$, with the $\mathcal{N}(0,1)$ prior unscaled; pretraining and prediction are untem-

pered. T is chosen per size on the grid $0.2, 0.3, \ldots, 1.0$, by bisection on the sign of the local slope and always including $T = 1$, minimizing the mean validation NLL of 3-member fits over the training sets of repeats 0–2 (Appendix C). The validation data are the 1,000 pool rows that follow the training rows in each repeat's permutation, and the reported T is the argmin over every evaluated point. The search uses the runs' own learning rate and weight decay. It gives $T = 0.7$, 0.7, 0.6 and 0.7 at $n = 50$, 100, 500 and 1,000. At $n \geq 5{,}000$ the best temperature is close to 1, so the $\mathcal{N}(0, 1)$ BNN is there both the untempered and, approximately, the optimally tempered BNN, and no separate tempered model is run. The grid search was not run at these sizes; the evidence is that the selected temperature rises with n (0.6–0.7 at $n = 500$–1,000), as does the coldest temperature at which the chains still mix (Appendix C), and that the only tempered runs at $n = 5{,}000$ and 10,000, earlier 3-member fits at $T = 0.305$ transferred from an $n = 100$ search, are worse than $T = 1$ under the same settings (test NLL -1.88 against -2.21 and -2.25 against -2.27) and do not mix (maximum $\widehat{R}$ 32–64).

4.4.3 Empirical Bayes

One scale τ for all 2,245 weights and biases is updated during sampling, on the re-tuning sampler of Appendix H.3: the 10,000 sampling steps are split into 10 segments of 1,000, and (ϵ, L) are re-tuned (200 and 50 steps) after each, so a chain runs 12,500 steps. τ starts at the root-mean-square of the member's pretrained weights, under which the warmup runs. After each segment, with m_i and v_i the mean and variance of weight i over that segment's steps,

$$\gamma = d - \sum_i v_i/\tau^2 \quad \text{(clamped to } [1, d]), \qquad \tau^2_{\text{target}} = \sum_i m_i^2/\gamma, \tag{12}$$

which is MacKay's condition (Section 2.4) with the posterior mean and covariance replaced by the segment's sample moments. $\log \tau$ then moves halfway towards $\log \tau_{\text{target}}$, by at most a factor of 2 per update, with a floor of 10^{-3}. The scale is per member, and all 1,000 kept draws per chain are used, including those drawn before the scale has settled. The damping, the step cap and the floor have no source. The fixed-scale control, $\mathcal{N}(0, 1)$ + retune, is the same sampler with $\tau = 1$.

4.5 Our MILE implementation

We use the `sklearn-contrib-bde` package (version 1.0.2, `blackjax` 1.2.2), which implements MILE. Its settings match the paper's defaults: $K = 12$ members per fit, 50,000 warmup steps split 0.8/0.1/0.1 into phases I–III (40,000/5,000/5,000), 10,000 sampling steps thinned by 10 to 1,000 draws per chain, EEVPD target annealed $0.5 \to 0.1$, ESS target 100, initial step size equal to the pretraining learning rate, no preconditioning, and an isotropic $\mathcal{N}(0, 1)$ prior on all 2,245 weights. Each member is pretrained by AdamW on the negative log of Eq. (9) for at most 2,000 steps, evaluating the validation NLL every 100 steps and stopping after 5 evaluations without an improvement of 10^{-4} (after at least 500 steps), on `bde`'s 80/20 split (Section 4.1); the best parameters are kept, and chain k starts from member k. The network and likelihood are those of Section 4.4. The per-draw outputs give the chain-wise diagnostics of Appendix H.2.

Departures from stock `bde` are subclass overrides in `src/rbf_sbi/bnn.py` (Appendix D): a correlated two-target head; a pretraining schedule that evaluates every 100 steps, with AdamW; replacement of members whose chains become non-finite; likelihood tempering in warmup and sampling; and, for the empirical-Bayes model and its control, periodic re-tuning of (ϵ, L) during sampling and the block update of the prior scale (Section 4.4.3).

4.6 Hyperparameter selection

Only the learning rate and weight decay are searched, per method and training size; MILE's sampler settings are its defaults. NPE's are chosen by Bayesian optimization [45]: `scikit-optimize` 0.10.2, a Gaussian-process surrogate with expected improvement, log-uniform search ranges lr $\in [10^{-5}, 10^{-1}]$ and wd $\in [10^{-7}, 10^{-2}]$, 10 random initial points, random state 123, and a wall-clock budget per size. The objective is the validation NLL on an 80/20 split of the repeat-0 training rows. Within its budget the search ran 3–20 trials per size and acted as a random search of 3–10 points (Appendix C). The BNN and the Gaussian predictor are not tuned by Bayesian optimization: they share one pair per size. It is chosen on a grid for the BNN's network as a standalone predictor, with one network pretrained exactly as in Section 4.5, without sampling, scored by its NLL on 1,000 held-out pool rows averaged over the training sets of repeats 0–2. The grid has 17 learning rates from 10^{-5} to 10^{-1} in quarter decades and 15 weight decays from 10^{-7} to 1 in half decades (the upper bound is raised from 10^{-2} because AdamW's decoupled decay is nearly inert below it at these learning rates), and the argmin is used at every size. At $n = 50$ the argmin is noise-driven, and at its learning rate two of the three grid fits stop at the 2,000-step cap. The converged Gaussian predictor keeps these values, which were chosen under the 2,000-step budget. Appendix C gives the search details, budgets and rationale, including the earlier BNN search on the full MILE fit that this grid supersedes.

	NPE		BNN and Gaussian predictor	
n	lr	wd	lr	wd
50	9.53×10^{-5}	2.98×10^{-4}	3.16×10^{-5}	3.16×10^{-1}
100	3.23×10^{-3}	4.18×10^{-6}	1.78×10^{-4}	3.16×10^{-1}
500	2.37×10^{-3}	1.44×10^{-4}	1.78×10^{-4}	1.00×10^{-2}
1,000	5.80×10^{-3}	3.94×10^{-4}	1.78×10^{-4}	3.16×10^{-1}
5,000	4.40×10^{-4}	7.92×10^{-5}	1.00×10^{-3}	3.16×10^{-1}
10,000	9.21×10^{-4}	7.95×10^{-4}	1.00×10^{-3}	3.16×10^{-1}

Table 1: Learning rate and weight decay used by every result in this document. The BNN values apply to all BNN variants and to the Gaussian predictor (capped and converged).

4.7 Software, compute and use of language models

All fits ran on the Perlmutter system of the National Energy Research Scientific Computing Center (NERSC), one NVIDIA A100 GPU (40 or 80 GB) per Slurm job. For one training set, a 12-chain BNN under stock MILE takes 36–72 s of wall time (fitting and prediction; the 12 chains run in parallel on the GPU), the re-tuning sampler 411–534 s, the converged 12-network Gaussian predictor 24–82 s, and the 12-flow NPE ensemble 157–238 s at $n \leq 1{,}000$ and 663–907 s at $n \geq 5{,}000$; Appendix C gives the settings per model and Appendix B the software versions; the code is at `https://github.com/sam-dowd/rbf-sbi-bnn`. The code, analysis scripts and text of this report were developed with the assistance of Claude models (Anthropic), most recently Claude Opus 5.5, used through Claude Code; all results were checked by the author.

5 Evaluation protocol

5.1 Metrics

All metrics are computed on the $M = 1{,}000$ test datasets (S_j, θ_j^*).

Test NLL. The primary score is

$$\mathrm{NLL} = -\frac{1}{M}\sum_{j=1}^{M} \log q(\theta_j^* \mid S_j) \quad \text{(nats, joint over } (\sigma, \rho)\text{)}, \tag{13}$$

in nats (natural-log units). It needs only the true parameter θ_j^*, not the exact posterior. It rewards accuracy and penalizes miscalibration in either direction, but over-confidence much more than conservatism: a width that is too narrow enters through the squared standardized error, one that is too wide only through its logarithm. That asymmetry suits scientific use, where a conservative posterior is the safer error (Section 1). Each method's q is a normalized density on θ in original units, evaluated exactly at θ_j^*: the mixture of the 12 networks' Gaussians for the Gaussian predictor (one Gaussian for a single network), the equal-weight mixture over all KS draws for the BNN, the flow's change-of-variables density for a single flow and the mixture of the 12 flows' densities for the NPE ensemble, and the grid density for the exact posterior (Appendix F), so all NLLs are directly comparable. The test NLL is the negative of the mean log pointwise predictive density (LPPD) at θ^*, the predictive score of Sommer et al. [21]. The exact posterior scores -2.390.

Badly over-confident datasets. With $\hat{\mu}_{j,p}$ and $\hat{s}_{j,p}$ the mean and total standard deviation of a method's predictive for parameter p (Section 4.1), $z_{j,p} = (\hat{\mu}_{j,p} - \theta_{j,p}^*)/\hat{s}_{j,p}$. The percentage of test datasets with $|z_{j,p}| > 3$ for σ or ρ counts the datasets on which the predictive is far too narrow for its own error. The exact posterior scores 0.9%, somewhat more than the 0.5% of a Gaussian, since its posteriors are not exactly Gaussian.

Metrics that use the exact posterior. Four appear in Section 6; each is defined in full where it is reported. The *TARP coverage area* is the integral over credibility levels α of $\mathrm{ECP}(\alpha) - \alpha$: 0 for a calibrated predictive, positive for a conservative one, negative for an over-confident one (Appendix A.6). The *width over warranted width* is the RMS predictive width divided by the RMS width that a calibrated predictive with the same mean errors would need; 1 is ideal (Appendix A.5). The *coherence ratio* is the mean predicted variance plus the variance of the predicted means across test datasets, divided by the variance of θ^* over the test set; by the law of total variance (Section 2.4) it is 1 for a coherent predictive (Appendix A.5). The *aleatoric share* is the fraction of a BNN's variance in excess of the exact posterior's and of its own mean error that lies in its aleatoric term (Eq. (10)). The appendices add the error of the predicted mean and of the predicted standard deviation against the exact posterior's and the spread of z (Appendix A.6); and the decomposition of the BNN's NLL into chains, draws and starting networks (Appendix A.2). Table 4 gives their values for the exact posterior.

5.2 Convergence diagnostics

Diagnostics are reported, never used to accept or discard a fit. The only members removed are those whose draws become non-finite. Under stock MILE (the $\mathcal{N}(0, 1)$, data-free and tempered BNNs) they are replaced by re-seeded refits on the same training rows, so every fit has 12 members (8 of 2,136 member fits failed). The re-tuning sampler (the $\mathcal{N}(0, 1)$ + re-tune and empirical-Bayes models, Appendix A.7) drops them without replacement, which leaves 9–11 members in 19 of 60

and 8 of 60 fits respectively. BNN weight posteriors are non-identifiable and multimodal, chains started in different modes need not reach a common function [21, Sec. 6.2], and MILE claims no convergence [12, App. A.3]; fits are judged by their test NLL (Section 5.1) and, in the appendices, by their agreement with the exact posterior. The pipeline diagnostic is the classic potential scale reduction factor (PSRF) $\widehat{R}$ [46] with the effective sample size, as implemented in BlackJAX 1.2.2, on the predicted mean for every (test dataset, parameter) pair with the K members as chains, reported as maximum and mean. Appendix H adds rank-normalized $\widehat{R}$, both pooled (all members' chains together, so it measures whether the members agree) and chain-wise (each chain split into sub-chains, so it measures whether each chain is stationary), on the mean, the width and the log density at θ^*, and the per-member expanding-window LPPD: the test LPPD of a member's predictive built from its first l kept draws, as a function of l, the per-chain convergence check of Sommer et al. [21, App. B.2].

5.3 Paired comparisons

All methods at a given (n, r) train on the same rows and are evaluated on the same test set (Section 3.3). A comparison between two methods A and B is therefore made per repeat, $\Delta_r = m_A(n, r) - m_B(n, r)$ for a metric m, which removes the variation due to which training set was drawn (at low n the largest source of variation). Differences are reported as

$$\bar{\Delta} \pm \mathrm{sd}(\Delta) \quad (k/R),$$

the mean and sample standard deviation of Δ_r over the R repeats ($R = 20$ at $n \leq 100$, $R = 5$ above) and the number k of repeats in which the difference has the stated sign; for NLL, negative means A is better. A difference is called a finding if a two-sided paired t-test on the Δ_r gives $p < 0.05$. A Wilcoxon signed-rank test, which does not assume normality, is reported as a check, and cells in which the two disagree are marked. A statement about a single fit requires a difference larger than the chain-to-chain spread within that fit. The spread of a metric over repeats is itself reported, as a method's consistency across training sets. At $n \leq 1{,}000$ repeats share at most 10% of their training rows; at $n = 5{,}000$ and 10,000 they share half or all of them (Section 3.3), so the spread there is mostly seed variance.

GPU runs are not bit-reproducible: identical code, seed and data give different chains (e.g. empirical-Bayes prior scales differing by 0.01–0.02). "Reproduces" therefore means agreement within the spread above. For example, re-running the 3-member $\mathcal{N}(0, 1)$ configuration used before the final runs at $n = 100$, $r = 0$ gave test NLL -0.759 against the saved -0.745, inside that configuration's repeat-to-repeat spread of 0.108, with the pooled predicted mean within the saved run's between-chain spread.

6 Experiments

This section reports test NLL, the primary score (Section 5.1), and in Section 6.4 also the width metrics that use the exact posterior. Three BNNs are compared, each a 12-chain MILE fit, under MILE's default $\mathcal{N}(0, 1)$ prior, with the data-free prior scale (Section 4.4.1) and with a tempered likelihood (Section 4.4.2); "the BNN" means any of the three. The tempered BNN is run only at $n \leq 1{,}000$, since at larger n the best temperature is close to 1 and the $\mathcal{N}(0, 1)$ BNN is also the tempered one. Section 6.1 compares the BNNs with a single network of each baseline, so that they differ from it in both sampling and ensembling, and Section 6.2 compares them with 12-member

ensembles of the same baselines, so that only sampling differs. Paired differences, the verdict of each BNN against each ensemble, the decomposition of the BNN's NLL, calibration and width are in Appendix A.

6.1 Sampling and ensembling together: the BNN against single networks

	$n = 50$	$n = 100$	$n = 500$	$n = 1{,}000$	$n = 5{,}000$	$n = 10{,}000$
Sampled BNN (12 chains)						
BNN, $\mathcal{N}(0, 1)$	-0.363 ± 0.059	-0.775 ± 0.062	-1.441 ± 0.039	-1.720 ± 0.029	$\mathbf{-2.226 \pm 0.009}$	$\mathbf{-2.284 \pm 0.005}$
BNN, data-free scale	$\mathbf{-0.603 \pm 0.050}$	$\mathbf{-0.867 \pm 0.060}$	-1.422 ± 0.030	-1.662 ± 0.015	-2.196 ± 0.006	-2.270 ± 0.003
BNN, tempered	-0.419 ± 0.049	$\mathbf{-0.879 \pm 0.050}$	$\mathbf{-1.687 \pm 0.043}$	$\mathbf{-1.870 \pm 0.034}$	–	–
Single network						
Gaussian predictor, 1 net	$\mathbf{+1.650 \pm 7.236}^{\dagger}$	-0.718 ± 0.206	-1.327 ± 0.042	-1.621 ± 0.029	-1.893 ± 0.059	-2.021 ± 0.019
NPE, 1 flow	$+0.578 \pm 1.205$	-0.407 ± 0.348	-1.266 ± 0.065	-1.497 ± 0.127	-2.006 ± 0.025	-2.074 ± 0.017

Table 2: Test NLL (nats), mean ± sd over repeats: 20 repeats at $n \leq 100$, 5 at $n \geq 500$; lower is better; exact posterior -2.390. Bold, per column: the lowest mean and every row not distinguishable from it by a two-sided paired t-test at $p < 0.05$ (Section 5.3). †: at $n \leq 100$ the Wilcoxon signed-rank test disagrees with the bolding. The one dagger: the single network at $n = 50$ is bolded only because one repeat at $+32.1$ inflates its sd (median over repeats -0.34). Gaussian predictor: one converged network (member 0); NPE: one flow (member 0 of the ensemble).

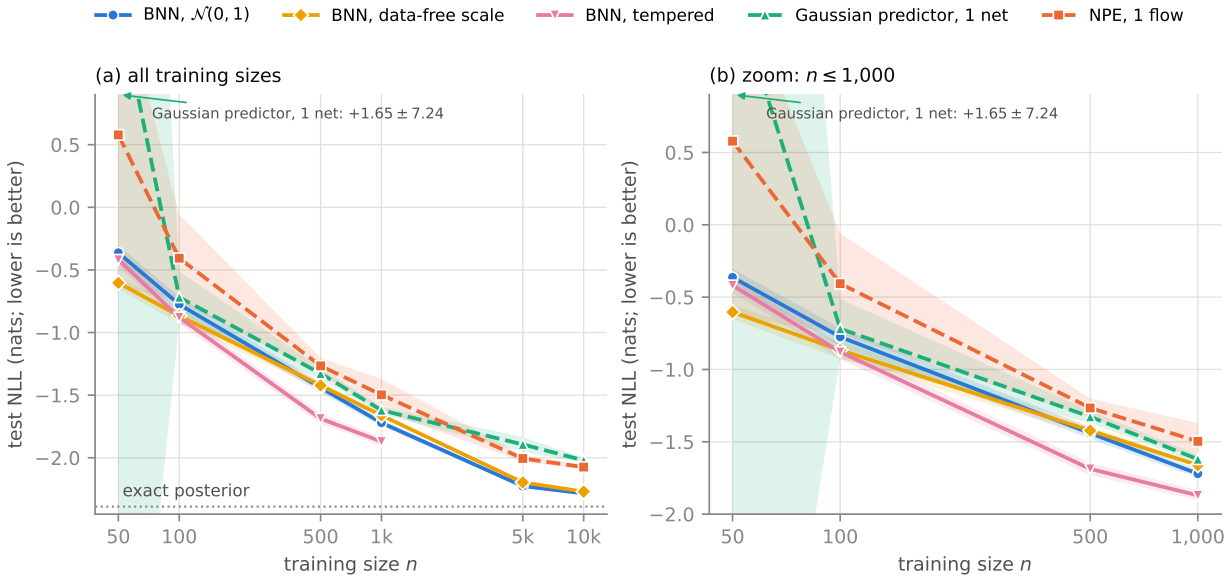


Figure 5: Test NLL against n for the three BNNs and the single networks of Table 2: mean over repeats, band ±1 sd over repeats. Means above the plotted range are written at the top edge.

Against a single network, sampling looks decisive (Table 2, Figure 5). Every BNN is below a single NPE flow at every n, by about 1 nat at $n = 50$ and 0.4 at $n = 100$, and below a single Gaussian-predictor network from $n = 500$, the data-free and tempered BNNs also at $n = 100$. At $n = 50$ the paired test separates no BNN from the single Gaussian-predictor network, because one of its 20 repeats scores $+32$, although the BNNs are lower on most repeats. The larger difference at small n is reliability: over training sets, a single network's NLL has a standard deviation of 0.2–7 nats

at $n \leq 100$, every BNN's 0.05–0.06. A single network fitted to 50 or 100 simulations is a poor and unpredictable estimator. How much of the BNN's advantage over it comes from sampling, and how much from averaging over many networks, is what the comparison with ensembles separates.

6.2 Sampling alone: the BNN against 12-member ensembles

	$n = 50$	$n = 100$	$n = 500$	$n = 1{,}000$	$n = 5{,}000$	$n = 10{,}000$
Sampled BNN (12 chains)						
BNN, $\mathcal{N}(0, 1)$	-0.363 ± 0.059	-0.775 ± 0.062	-1.441 ± 0.039	-1.720 ± 0.029	$\mathbf{-2.226 \pm 0.009}$	$\mathbf{-2.284 \pm 0.005}$
BNN, data-free scale	$\mathbf{-0.603 \pm 0.050}$	-0.867 ± 0.060	-1.422 ± 0.030	-1.662 ± 0.015	-2.196 ± 0.006	-2.270 ± 0.003
BNN, tempered	-0.419 ± 0.049	-0.879 ± 0.050	$\mathbf{-1.687 \pm 0.043}$	$\mathbf{-1.870 \pm 0.034}$	–	–
12-member ensembles						
Gaussian predictor, 12 nets	$\mathbf{-0.551 \pm 0.199}$	-0.946 ± 0.159	-1.498 ± 0.040	-1.744 ± 0.021	-1.991 ± 0.010	-2.076 ± 0.014
NPE, 12 flows	-0.441 ± 0.298	$\mathbf{-1.062 \pm 0.186}$	-1.483 ± 0.114	-1.699 ± 0.045	-2.067 ± 0.015	-2.105 ± 0.012
NPE, 12 flows, long-trained	–	–	–	–	–	-2.159 ± 0.007
Test datasets with $\lvert z \rvert > 3$ (%): median over repeats (worst repeat)						
BNN, $\mathcal{N}(0, 1)$	0.0 (0.0)	0.0 (0.2)	0.2 (0.3)	0.4 (0.5)	1.1 (1.2)	1.2 (1.2)
BNN, data-free scale	0.3 (2.1)	0.3 (1.2)	0.4 (0.7)	0.5 (0.6)	1.0 (1.1)	1.2 (1.2)
BNN, tempered	0.0 (0.1)	0.0 (0.2)	0.5 (0.7)	0.6 (0.8)	–	–
Gaussian predictor, 12 nets	2.2 (6.6)	1.5 (2.8)	1.9 (2.2)	1.5 (1.8)	1.0 (1.1)	0.9 (1.0)
NPE, 12 flows	3.0 (7.7)	1.8 (4.3)	1.4 (1.7)	1.3 (1.8)	1.0 (1.2)	0.8 (0.8)
NPE, 12 flows, long-trained	–	–	–	–	–	0.9 (1.1)

Table 3: Test NLL (nats), mean ± sd over repeats: 20 repeats at $n \leq 100$, 5 at $n \geq 500$; lower is better; exact posterior −2.390. Bold, per column: the lowest mean and every row not distinguishable from it by a two-sided paired t-test at $p < 0.05$ (Section 5.3). †: at $n \leq 100$ the Wilcoxon signed-rank test disagrees with the bolding. Gaussian predictor: the 12 networks trained to convergence; NPE: 12 flows at the default budget, and at $n = 10{,}000$ also the long-trained ensemble (Section 4.3.2). Lower block: percentage of the 1,000 test datasets on which the predictive is badly over-confident, $|z| > 3$ for σ or ρ with $z = (\text{predicted mean} - \theta^*)/\text{predicted sd}$; median over repeats, worst repeat in parentheses; the exact posterior scores 0.9%.

Against these equally ensembled baselines the BNN does not have the lowest average NLL at $n \leq 100$, but it is the most reliable estimator there, and it has the lowest NLL from $n = 500$ with a tempered likelihood and from $n = 5{,}000$ without. Ensembling removes most of the single-network advantage. Twelve flows lower NPE's NLL by about 1 nat at $n = 50$ and 0.7 at $n = 100$, and twelve networks lower the Gaussian predictor's by 0.23 at $n = 100$, so against equally ensembled baselines the BNN no longer wins at small n (Table 3, Figure 6).

Small n: NPE is as good or better on average. At $n = 100$ the 12-flow NPE has the lowest NLL of all models, 0.18–0.29 nats below every BNN. At $n = 50$ the comparison is close: only the data-free BNN is below NPE, by 0.16 ± 0.28 nats on 13 of 20 repeats. All repeats share one test set, whose sampling uncertainty the paired test does not include; for this comparison it is about 0.05 nats, comparable to the repeat-to-repeat standard error of 0.06, so the difference is marginal, and it comes from the prior rather than from sampling, since this BNN ties its own unsampled networks. Neither adjustment rescues $n = 100$. NPE's advantage there is in the predicted mean: the 12-flow ensemble's mean error is 0.10 for σ and 0.09 for ρ, against 0.12–0.13 and 0.10–0.11 for the BNNs (Appendix A.6), and neither adjustment changes the BNN's mean error at $n \leq 100$ by more than 0.006. The advantage in σ, whose posterior is right-skewed, comes from the flow head: with a bivariate Gaussian head NPE's mean error for σ at $n = 100$ rises to the BNNs' level, 0.126 against 0.096 (Appendix A.3).

Small n: the BNN is more reliable. The NLL of every BNN varies 3–4 times less over train-

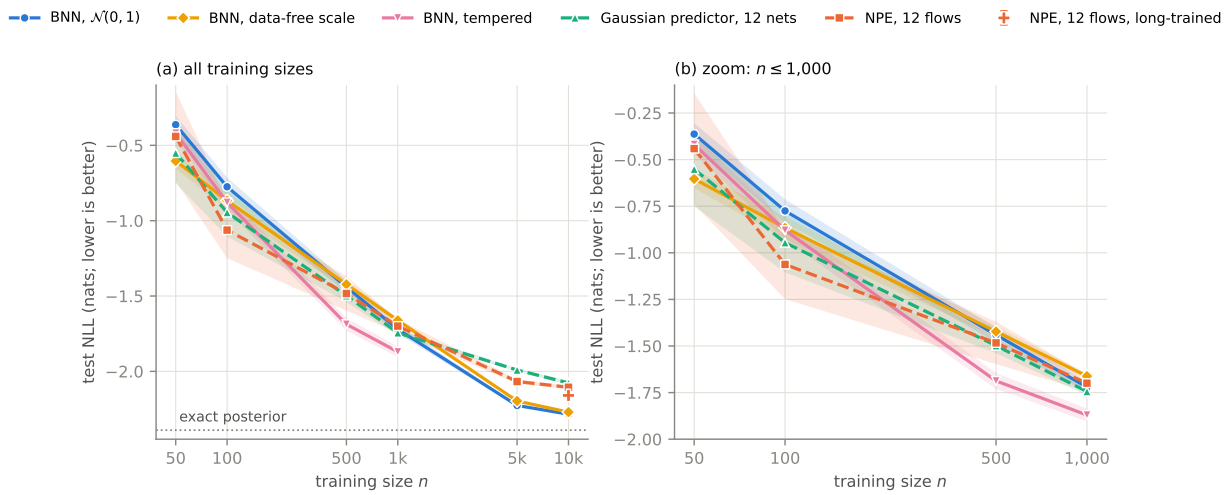


Figure 6: Test NLL against n for the three BNNs and the 12-member ensembles of Table 3: mean over repeats, band ± 1 sd over repeats; the long-trained NPE at $n = 10{,}000$ as a single marker with its sd.

ing sets than that of the Gaussian predictor's ensemble and 3–6 times less than NPE's (standard deviation 0.05–0.06 nats at $n \leq 100$, against 0.16–0.20 for the Gaussian predictor and 0.19–0.30 for NPE), and its worst training set is better: at $n = 50$ every BNN's worst repeat (-0.25 to -0.48) is below both ensembles' (-0.08 and $+0.23$). The ensembles are also badly over-confident on a few percent of test datasets at small n, 2–3% in the median repeat at $n = 50$ and up to 7.7% in the worst, whereas no BNN exceeds 0.3% in the median repeat, below the exact posterior's own 0.9% (lower block of Table 3). The Gaussian predictor is the same 12 networks without sampling, and its capped version, from which the chains start, varies as much over training sets (Section 6.3), so the gain in reliability is due to sampling. It is not the product of a wide predictive: with the data-free scale the BNN at $n = 50$ is calibrated (TARP coverage area $+0.003$, against $+0.014$ for NPE) and its width is 1.15 / 1.20 times the one its own errors warrant (Appendix A.5). Under $\mathcal{N}(0, 1)$ and tempering it is conservative (TARP coverage area $+0.05$ under $\mathcal{N}(0, 1)$; Figure 7).

Large n: the BNN wins on average. From $n = 5{,}000$ both untempered BNNs have the lower NLL (at these sizes the $\mathcal{N}(0, 1)$ BNN is also, approximately, the optimally tempered one; Section 4.4.2): by 0.13–0.18 nats against the 12-flow NPE (5 of 5, $p \leq 2 \times 10^{-5}$), by 0.11–0.13 at $n = 10{,}000$ against the NPE trained to convergence, and by 0.19–0.24 against the Gaussian predictor. With a tempered likelihood the BNN is lower than both from $n = 500$: by 0.20 and 0.17 nats against NPE at $n = 500$ and 1,000, and by 0.19 and 0.13 against the Gaussian predictor. Several differences in budget, tuning and data use favour the BNN at large n; Appendix A.4 lists them and estimates their size, which is below the margins here.

Large n: better networks, not only averaging. These are also the sizes at which MILE improves on the same networks optimized to convergence, and only partly by averaging. At $n = 5{,}000$ and 10,000 a typical single posterior draw of the untempered BNNs is already 0.04–0.14 nats below the converged 12-network ensemble (Table 9), 20–70% of the BNN's lead over it; averaging within and across chains supplies the rest. MCLMC therefore also finds better individual networks than optimization to convergence. At these sizes the 12 chains do not agree with each other (median pooled $\widehat{R}$ on the predicted mean 1.16–1.68), although each chain's predictive is stable (Appendix H).

6.3 Sampling and ensembling inside the BNN

Each BNN is itself sampled and ensembled: 12 chains, each started from one network of the capped Gaussian predictor (Table 8). For the average NLL the two are close substitutes at small n. Mixing the 12 chains lowers the NLL by 0.003–0.026 nats against a single chain at $n \leq 1{,}000$ (0.02–0.07 at $n \geq 5{,}000$), whereas ensembling the 12 starting networks without sampling lowers it by 0.10 at $n = 50$, 0.23 at $n = 100$ and 0.08–0.14 at $n = 500$–1,000 (Appendix A.2): the draws of one chain already do the work of an ensemble. For the spread over training sets they are not substitutes. The 12 starting networks vary as much as the converged Gaussian predictor (standard deviation 0.16–0.18 nats at $n \leq 100$), the BNN started from them 0.05–0.06. A single draw used alone is a poor estimator at small n under $\mathcal{N}(0, 1)$ and tempering (median test NLL +46 and +83 at $n = 50$) because it is badly over-confident on a few test datasets; mixing a chain's draws removes these failures. Under the data-free scale a typical draw scores -0.06, against -0.38 for its starting network and -0.60 for its chain.

6.4 Adjusting the prior and the likelihood

Excess width under the default prior. Under $\mathcal{N}(0, 1)$ the BNN's predictive at $n = 50$ is 2.38 / 2.42 times (σ / ρ) as wide as its own mean errors warrant, and its widths and the spread of its means together claim 3.09 / 3.67 times the variance of θ over the test set, where a coherent predictive claims 1 (coherence ratio, Appendix A.5). At every $n \leq 1{,}000$, 84–98% of the excess variance is in the aleatoric term. The excess shrinks with n and is gone from $n = 5{,}000$.

Data-free scale. The data-free scale reduces the width over warranted width at $n = 50$ to 1.15 / 1.20 and the coherence ratio to 1.00 / 1.09, and lowers the NLL most there, by 0.24 nats against $\mathcal{N}(0, 1)$; its gain shrinks with n, and from $n = 1{,}000$ it is slightly worse. The same scale follows from the coherence condition on the training inputs alone (Section 4.4.1), so it is not an artefact of the simulations used to calibrate it.

Empirical Bayes. Maximizing the marginal likelihood of the training data recovers the data-free scale where the update is stable (final scale 0.37, 0.36 and 0.32 at $n = 100$, 500 and 1,000, against 0.3655). At $n = 50$ it collapses in all 20 repeats to a nearly constant predictor, 0.54 nats above its control; a per-layer version did not rescue $n = 50$ and was worse at $n = 500$ and 1,000 (Appendix A.7).

Tempering. Tempering lowers the NLL at every $n \leq 1{,}000$ but most at $n = 500$ and 1,000, by 0.25 and 0.15 nats, where the tempered BNN is the best model and the only one ahead of both ensembles. At $n = 500$ the data-free scale narrows the predictive as much as tempering does (width over warranted width 1.09 / 1.24, against 1.09 / 1.26) but its predicted mean is worse, whereas the tempered BNN has the lowest mean error of any BNN (Appendices A.5 and A.6; the same holds at $n = 1{,}000$). At $n = 500$ and 1,000 the selected temperature is the coldest at which the chains still mix (Appendix C). The data-free scale is better at $n = 50$, the two tie at $n = 100$, and tempering is better from $n = 500$; combining them was not tried.

Summary. Against single networks every BNN has the lower NLL at every n. Against 12-member ensembles of the same baselines no BNN is lower on average at $n \leq 100$: NPE is lower at $n = 100$ by 0.18–0.29 nats, and at $n = 50$ only the data-free BNN is lower, marginally. At $n \leq 100$ every BNN's NLL varies 3–4 times less over training sets than the Gaussian predictor's ensemble and 3–6 times less than NPE's, its worst training set at $n = 50$ is better, and it is badly over-confident on at

most 0.3% of test datasets in the median repeat, against 1.5–3% for the ensembles and 0.9% for the exact posterior. The BNN is lower than both ensembles from $n = 500$ with a tempered likelihood and from $n = 5{,}000$ without. Under the default prior it is too wide at $n \leq 100$, almost entirely in the aleatoric term, and the data-free scale removes most of the excess.

7 Discussion

On this task, BNNs sampled with MILE do not beat NPE on average in the low-data regime once both are ensembles of the same size, but they are the more reliable estimator, and we argue that reliability is what matters for SBI with little data. A practitioner trains one estimator on one simulation budget and applies it to one observation. They cannot average over training sets, and they cannot tell from their own training set whether it is one on which an optimized estimator happens to do well, since every training set is an equally likely draw from the simulator. How far a single result can be trusted then depends on the spread over training sets as well as on the average. On that criterion the BNN is better, with a spread at $n \leq 100$ that is 3–6 times smaller than NPE's and 3–4 times smaller than the Gaussian predictor's (Section 6.2). We suggest that low-budget SBI comparisons report this spread, and how often the posterior is badly over-confident, alongside the average score. The paragraphs below give the interpretations behind this answer and the other results; the measurements of Section 6 and the appendices support them but do not establish them.

7.1 Why sampling, and not ensembling, buys consistency. With 50 or 100 simulations the main difficulty is not the capacity of the estimator but the noise in the training set. An optimized network, and an ensemble of 12, fits the particular draw it was given. The Gaussian predictor's networks vary as much over training sets at MILE's starting point as after training to convergence, and the same networks sampled with MILE vary 3–4 times less (Section 6.3). Our interpretation is that an ensemble averages over initializations that are all fitted to the same draw, whereas sampling averages over the networks that the draw does not rule out, so its predictive depends less on which draw it was. We have not tested this against an ensemble with explicit regularization or a bootstrap over training rows. The dependence on the draw cuts both ways, which is why it does not lower the average. On favourable draws the optimized ensembles beat the BNN, and on unfavourable ones they lose to it. For the same reason, a comparison with a single network overstates what sampling does. For the average NLL at small n, sampling and ensembling are close substitutes, and deep ensembles can themselves be read as approximate Bayesian model averaging [15]. Here the single-network comparison shows an advantage of about 1 nat at $n = 50$ that equal ensembling removes. Ensembled baselines of the kind that Hermans et al. [6] recommend and Delaunoy et al. [8] include are therefore the necessary control, and without them a low-budget advantage of a BNN is evidence for averaging rather than for sampling.

7.2 At small n the aleatoric term is not aleatoric. Under $\mathcal{N}(0, 1)$, 84–98% of the BNN's excess width at $n \leq 1{,}000$ lies in its aleatoric term, and a smaller prior scale chosen without training data removes most of it (Section 6.4). The width the network reports as irreducible noise is therefore set largely by the prior. The excess is not in the data, whose exact posterior width is known, and it responds to a change of the weight prior. Some excess at $n = 50$ is not specific to the prior. NPE, which has no weight prior, claims 1.44 / 1.38 times the prior variance as a mean over repeats and 1.20 / 1.24 in the median repeat (one repeat at 6.29 for σ dominates the mean), and its excess is gone by $n = 100$. The $\mathcal{N}(0, 1)$ BNN's excess, 3.09 / 3.67, is two to three times NPE's by either

measure and persists to $n = 1{,}000$ (Appendix A.5). We read this as the noise output absorbing what a misspecified prior and a small training set leave unresolved, that is, epistemic uncertainty reported as aleatoric. It is an instance of the finite-n failure described by Bickford Smith et al. [47], for whom the model's expected conditional uncertainty is only an estimate of the irreducible part, and possibly a highly inaccurate one. It also parallels the Gaussian-process result of Wendl et al. [48], in which structure the kernel cannot represent is absorbed into a learned noise level. Neither work varies the training-set size or compares with an exact posterior, and in the latter the mismatch is a too-restrictive kernel rather than a too-wide prior. For SBI the implication is that a BNN's aleatoric/epistemic split at small budgets should not be read as a split into irreducible and reducible posterior uncertainty without an external check, such as the coherence ratio, which needs only the prior and is necessary but not sufficient, or a reference posterior. The conclusions of Section 6 do not depend on the split, since they rest on the NLL of the full predictive. How the prior reaches the noise output is not established. One possibility, that tempering acts on the noise output directly through every likelihood term (Eq. (5)) while a weight prior acts only through the network's function-space prior, is stated in Appendix G.2.

7.3 The scale and the form of the prior. Tempering with $T = 0.6$–0.7 counts each training pair 1.4–1.7 times (Eq. (5)), so the posterior relies less on the prior, and we read it as compensating a misspecified prior. $\mathcal{N}(0, 1)$ on every weight is a default rather than a prior chosen for this task, the untempered BNN is under-confident at every $n \leq 1{,}000$ (Appendix A.5), and work on the cold posterior effect lists an inadequate prior among its causes [22, 23]. Our results separate the scale of the prior from its form. The scale is well specified once it is set to the data-free value, which empirical Bayes recovers from the training data at $n = 100$–$1{,}000$ (Section 6.4), so the error in MILE's default lies in its scale and is corrected. The isotropic Gaussian form is probably not well specified. At the smallest n the scale is the dominant error, and the data-free scale gains as much as tempering or more (more at $n = 50$, a tie at $n = 100$). At intermediate n what remains is the form, which only tempering compensates, and there tempering gains most. At $n = 500$ the data-free scale narrows the predictive as much as tempering does but worsens the predicted mean, whereas tempering gives the best mean of any BNN. This is what one would expect if a single scale for every weight still pulls the fit away from the data while tempering lowers the weight of the prior as a whole, and Wenzel et al. [22] likewise find that "the cold posterior effect cannot be resolved by using the right scaling of the Normal prior". Our reading predicts that a prior fitted to the task's function space, for instance optimized as in Delaunoy et al. [8], would make tempering unnecessary at intermediate n, which was not tested. At $n = 50$ the training data cannot set the scale at all, since empirical Bayes collapses there to a nearly constant predictor, the underfitting that Lotfi et al. [28] describe for marginal-likelihood hyperparameters at small n. For BNN-based SBI at small budgets we therefore recommend setting the prior scale from the prior predictive, or from the coherence condition on the training inputs, rather than by empirical Bayes.

7.4 What MCLMC does at large n, and how to judge convergence. From $n = 5{,}000$ the untempered BNNs beat both ensembles, and a typical single draw is already below the converged 12-network ensemble, by 20–70% of the BNN's lead (Section 6.2). We read this as MCLMC acting in part as a better optimizer, whose trajectories reach networks that the same networks optimized to convergence do not. The chains disagree with each other while each is stable. This is the situation in which Sommer et al. [21] argue that $\widehat{R}$ should not penalize chain-wise differences, and in which, as Papamarkou et al. [10] note for non-converged chains, the predictive remains useful. More generally, the pooled $\widehat{R}$ on the predicted mean is not an acceptance test for these fits. It

passes the collapsed empirical-Bayes fits at $n = 50$ on two of five repeats, does not see disagreement between the members' widths at small n, and fails at $n = 10{,}000$ on fits whose test NLL has a standard deviation of 0.005 nats over repeats (Appendix H.2). For BNN-based SBI we suggest judging fits by chain-wise statistics and by scores against a reference rather than by pooled $\widehat{R}$. MILE's tuning is a related case. Re-estimating its step size and L periodically during sampling lowers the NLL of $\mathcal{N}(0, 1)$ by 0.15 and 0.12 nats at $n = 500$ and 1,000 (Appendix A.7), where the re-tune settles at about a third of the warmup's L (earlier runs, Appendix H.3), whereas Sommer et al. [12] find MILE effectively tuning-free on regression datasets from the UCI machine-learning repository. Whether the gain reflects faster exploration or a biased target is not established.

7.5 Relation to BNNs for simulation-based inference. At small budgets our results agree with Delaunoy et al. [8]. As in their study, the untempered $\mathcal{N}(0, 1)$ BNN is conservative (TARP coverage area +0.05 at $n = 50$) and NPE is over-confident, although the 12-flow ensemble much less so than a single flow. At $n = 50$ the ensemble is badly over-confident ($|z| > 3$) on 3.0% of test datasets in the median repeat, against 6.4% for one flow (1.8% against 6.3% at $n = 100$). At large budgets the results differ. Delaunoy et al. [8] find their BNN slow to reach NPE's accuracy, and Poh et al. [9] find NPE more accurate and better calibrated than an under-confident BNN at 200,000–800,000 simulations. Here the MILE-sampled BNN is more accurate than NPE at our largest size, $n = 10{,}000$, and its under-confidence is confined to small n and largely removed there by the data-free scale. Both studies fit the BNN by variational inference, and neither scores it against a reference posterior. Ours is sampled by MCMC under a scalar prior scale and scored by test NLL with the exact posterior as a reference, so the results need not conflict.

7.6 Limitations. The main limitation is that every result comes from one task, a single simulator with a two-parameter posterior, so whether the BNN's reliability advantage and its aleatoric excess hold on other simulators is untested; the limitations of the task itself are discussed in Section 3.4. At $n \leq 100$ every method's test NLL is 1.3–2.0 nats above the exact posterior's -2.39, while the methods differ from each other by only 0.1–0.3, which is part of why a winner on the average NLL is hard to pick there. The comparisons favour the BNN in budget, tuning effort and data use (Appendix A.4). At $n \leq 100$ this strengthens the conclusion, and at large n each difference is smaller than the margin, although they have not been removed jointly. All repeats share one test set, and at $n \geq 5{,}000$ they also share most of their training rows, so their spread there is mostly seed variance. The BNN's learning rate, weight decay and temperatures were chosen on the training draws of repeats 0–2, which are also evaluated, and the best temperature at $n \geq 5{,}000$ is inferred rather than searched (Section 4.4.2). Finally, MCLMC is used without a Metropolis correction, and at $n \geq 5{,}000$ the chains do not agree, so the large-n results describe MILE's output rather than a converged posterior.

7.7 Next steps. (i) Other simulators, such as standard SBI benchmark tasks [16] and astrophysical simulators such as weak-lensing maps [49] or the CAMELS suite [5]. Since the test NLL needs no exact posterior, the main comparison carries over directly. (ii) Controls for the consistency mechanism, namely ensembles with explicit regularization and bootstrap ensembles over training rows. (iii) Separating MCLMC's optimization effect from averaging at large n, for instance by optimizing from the chains' final states to convergence. (iv) A Bayesian NPE, with NPE's flow head sampled by MILE. NPE's remaining small-n advantage comes from its flow head, which can represent the skew of σ's posterior (Appendix A.3), and sampling it would combine that flexibility with

the consistency of sampling; Delaunoy et al. [8] fit such a flow by variational inference. (v) Priors that act where the excess width is, such as a separate scale for the noise output or a prior fitted to the task's function space. (vi) Larger fields, with the same simulator on two- and three-dimensional grids, toward cosmological field-level inference (Section 3.4).

References

[1] Kyle Cranmer, Johann Brehmer, and Gilles Louppe. The frontier of simulation-based inference. *Proceedings of the National Academy of Sciences*, 117(48):30055–30062, 2020.

[2] George Papamakarios and Iain Murray. Fast ϵ-free inference of simulation models with Bayesian conditional density estimation. In *Advances in Neural Information Processing Systems 29*, 2016.

[3] George Papamakarios, Theo Pavlakou, and Iain Murray. Masked autoregressive flow for density estimation. In *Advances in Neural Information Processing Systems 30*, 2017.

[4] T. Lucas Makinen, Tom Charnock, Justin Alsing, and Benjamin D. Wandelt. Lossless, scalable implicit likelihood inference for cosmological fields. *Journal of Cosmology and Astroparticle Physics*, 2021(11):049, 2021. arXiv:2107.07405.

[5] Francisco Villaescusa-Navarro, Daniel Anglés-Alcázar, Shy Genel, David N. Spergel, Rachel S. Somerville, Romeel Dave, Annalisa Pillepich, Lars Hernquist, Dylan Nelson, Paul Torrey, et al. The CAMELS project: Cosmology and astrophysics with machine-learning simulations. *The Astrophysical Journal*, 915(1):71, 2021. doi: 10.3847/1538-4357/abf7ba.

[6] Joeri Hermans, Arnaud Delaunoy, François Rozet, Antoine Wehenkel, Volodimir Begy, and Gilles Louppe. A trust crisis in simulation-based inference? Your posterior approximations can be unfaithful. *Transactions on Machine Learning Research*, 2022.

[7] Alex Kendall and Yarin Gal. What uncertainties do we need in Bayesian deep learning for computer vision? In *Advances in Neural Information Processing Systems 30*, 2017.

[8] Arnaud Delaunoy, Maxence de la Brassinne Bonardeaux, Siddharth Mishra-Sharma, and Gilles Louppe. Low-budget simulation-based inference with Bayesian neural networks. *arXiv preprint arXiv:2408.15136v1*, 2024.

[9] Jason Poh, Ashwin Samudre, Aleksandra Ćiprijanović, Joshua Frieman, Gourav Khullar, and Brian D. Nord. Deep inference of simulated strong lenses in ground-based surveys. *Journal of Cosmology and Astroparticle Physics*, 2025(05):053, 2025. doi: 10.1088/1475-7516/2025/05/053.

[10] Theodore Papamarkou, Jacob Hinkle, M. Todd Young, and David Womble. Challenges in Markov chain Monte Carlo for Bayesian neural networks. *Statistical Science*, 37(3):425–442, 2022.

[11] Pavel Izmailov, Sharad Vikram, Matthew D. Hoffman, and Andrew Gordon Wilson. What are Bayesian neural network posteriors really like? In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021.

[12] Emanuel Sommer, Jakob Robnik, Giorgi Nozadze, Uroš Seljak, and David Rügamer. Microcanonical Langevin ensembles: Advancing the sampling of Bayesian neural networks. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2502.06335.

[13] Jakob Robnik and Uroš Seljak. Fluctuation without dissipation: Microcanonical Langevin Monte Carlo. In *Proceedings of the 6th Symposium on Advances in Approximate Bayesian Inference*, volume 253 of *PMLR*, pages 111–126, 2024.

[14] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems 30*, 2017.

[15] Andrew Gordon Wilson and Pavel Izmailov. Bayesian deep learning and a probabilistic perspective of generalization. In *Advances in Neural Information Processing Systems 33*, 2020. arXiv:2002.08791.

[16] Jan-Matthis Lueckmann, Jan Boelts, David S. Greenberg, Pedro J. Gonçalves, and Jakob H. Macke. Benchmarking simulation-based inference. In *Proceedings of the 24th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2021.

[17] Sean Talts, Michael Betancourt, Daniel Simpson, Aki Vehtari, and Andrew Gelman. Validating Bayesian inference algorithms with simulation-based calibration. *arXiv preprint arXiv:1804.06788v2*, 2018.

[18] Pablo Lemos, Adam Coogan, Yashar Hezaveh, and Laurence Perreault-Levasseur. Sampling-based accuracy testing of posterior estimators for general inference. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.

[19] Radford M. Neal. *Bayesian Learning for Neural Networks*, volume 118 of *Lecture Notes in Statistics*. Springer, 1996.

[20] David J. C. MacKay. A practical Bayesian framework for backpropagation networks. *Neural Computation*, 4(3):448–472, 1992.

[21] Emanuel Sommer, Lisa Wimmer, Theodore Papamarkou, Ludwig Bothmann, Bernd Bischl, and David Rügamer. Connecting the dots: Is mode-connectedness the key to feasible sample-based inference in Bayesian neural networks? In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, pages 45988–46018, 2024.

[22] Florian Wenzel, Kevin Roth, Bastiaan S. Veeling, Jakub Świątkowski, Linh Tran, Stephan Mandt, Jasper Snoek, Tim Salimans, Rodolphe Jenatton, and Sebastian Nowozin. How good is the Bayes posterior in deep neural networks really? In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.

[23] Vincent Fortuin, Adrià Garriga-Alonso, Sebastian W. Ober, Florian Wenzel, Gunnar Rätsch, Richard E. Turner, Mark van der Wilk, and Laurence Aitchison. Bayesian neural network priors revisited. In *International Conference on Learning Representations (ICLR)*, 2022.

[24] Lorenzo Noci, Kevin Roth, Gregor Bachmann, Sebastian Nowozin, and Thomas Hofmann. Disentangling the roles of curation, data-augmentation and the prior in the cold posterior effect. In *Advances in Neural Information Processing Systems 34*, 2021.

[25] David J. C. MacKay. *Bayesian Methods for Adaptive Models*. PhD thesis, California Institute of Technology, 1992.

[26] Wei Deng, Xiao Zhang, Faming Liang, and Guang Lin. An adaptive empirical Bayesian method for sparse deep learning. In *Advances in Neural Information Processing Systems 32*, 2019. arXiv:1910.10791.

[27] Javier Antorán, Shreyas Padhy, Riccardo Barbano, Eric Nalisnick, David Janz, and José Miguel Hernández-Lobato. Sampling-based inference for large linear models, with application to linearised Laplace. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.04994.

[28] Sanae Lotfi, Pavel Izmailov, Gregory Benton, Micah Goldblum, and Andrew Gordon Wilson. Bayesian model selection, the marginal likelihood, and generalization. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, volume 162 of *PMLR*, 2022.

[29] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, 2006.

[30] Matthew Sainsbury-Dale, Andrew Zammit-Mangion, and Raphaël Huser. Likelihood-free parameter estimation with neural Bayes estimators. *The American Statistician*, 78(1):1–14, 2024. doi: 10.1080/00031305.2023.2249522.

[31] Christopher K. I. Williams and Carl Edward Rasmussen. Gaussian processes for regression. In *Advances in Neural Information Processing Systems 8*, pages 514–520, 1996.

[32] Radford M. Neal. Monte Carlo implementation of Gaussian process models for Bayesian regression and classification. Technical Report 9702, Department of Statistics, University of Toronto, 1997.

[33] Iain Murray and Ryan Prescott Adams. Slice sampling covariance hyperparameters of latent Gaussian models. In *Advances in Neural Information Processing Systems 23*, 2010. arXiv:1006.0868.

[34] Mark S. Handcock and Michael L. Stein. A Bayesian analysis of kriging. *Technometrics*, 35 (4):403–410, 1993.

[35] Jean-Michel Marin, Pierre Pudlo, Christian P. Robert, and Robin J. Ryder. Approximate Bayesian computational methods. *Statistics and Computing*, 22(6):1167–1180, 2012.

[36] Yanzhi Chen, Dinghuai Zhang, Michael U. Gutmann, Aaron Courville, and Zhanxing Zhu. Neural approximate sufficient statistics for implicit models. In *International Conference on Learning Representations (ICLR)*, 2021.

[37] Samuel Müller, Noah Hollmann, Sebastian Pineda, Josif Grabocka, and Frank Hutter. Transformers can do Bayesian inference. In *International Conference on Learning Representations (ICLR)*, 2022.

[38] Paul E. Chang, Nasrulloh Loka, Daolang Huang, Ulpu Remes, Samuel Kaski, and Luigi Acerbi. Amortized probabilistic conditioning for optimization, simulation and inference. In *Proceedings of the 28th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2025. arXiv:2410.15320.

[39] Matthias Bitzer, Mona Meister, and Christoph Zimmer. Amortized inference for Gaussian process hyperparameters of structured kernels. In *Proceedings of the 39th Conference on Uncertainty in Artificial Intelligence (UAI)*, 2023.

[40] Andrew Zammit-Mangion, Matthew Sainsbury-Dale, and Raphaël Huser. Neural methods for amortized inference. *Annual Review of Statistics and Its Application*, 12:311–335, 2025. doi: 10.1146/annurev-statistics-112723-034123.

[41] Yajing Huang, Graeme E. Addison, and Charles L. Bennett. Accounting for correlations when fitting extra cosmological parameters. *The Astrophysical Journal*, 882(2):124, 2019. doi: 10.3847/1538-4357/ab3654. arXiv:1904.10521.

[42] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

[43] Conor Durkan, Artur Bekasov, Iain Murray, and George Papamakarios. nflows: normalizing flows in PyTorch. Zenodo, version 0.14, 2020. doi:10.5281/zenodo.4296287.

[44] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

[45] Jasper Snoek, Hugo Larochelle, and Ryan P. Adams. Practical Bayesian optimization of machine learning algorithms. In *Advances in Neural Information Processing Systems 25*, pages 2951–2959, 2012.

[46] Andrew Gelman and Donald B. Rubin. Inference from iterative simulation using multiple sequences. *Statistical Science*, 7(4):457–472, 1992.

[47] Freddie Bickford Smith, Jannik Kossen, Eleanor Trollope, Mark van der Wilk, Adam Foster, and Tom Rainforth. Rethinking aleatoric and epistemic uncertainty. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.

[48] Manuel Wendl, Erik Englesson, Andreas Krause, and Carl Henrik Ek. Forgetting to improve: Principled data removal in active learning. *arXiv preprint arXiv:2611.04054v1*, 2026.

[49] Niall Jeffrey, Justin Alsing, and François Lanusse. Likelihood-free inference with neural compression of DES SV weak lensing map statistics. *Monthly Notices of the Royal Astronomical Society*, 501(1):954–969, 2021. doi: 10.1093/mnras/staa3594.

[50] Alberto Cabezas, Adrien Corenflos, Junpeng Lao, Rémi Louf, et al. BlackJAX: Composable Bayesian inference in JAX. *arXiv:2402.10797*, 2024.

[51] Jakob Robnik, G. Bruno De Luca, Eva Silverstein, and Uroš Seljak. Microcanonical Hamiltonian Monte Carlo. *Journal of Machine Learning Research*, 24(311):1–34, 2023.

[52] Carl Edward Rasmussen. A practical Monte Carlo implementation of Bayesian learning. In *Advances in Neural Information Processing Systems 8*, pages 598–604, 1996.

[53] Aki Vehtari, Andrew Gelman, Daniel Simpson, Bob Carpenter, and Paul-Christian Bürkner. Rank-normalization, folding, and localization: An improved $\widehat{R}$ for assessing convergence of MCMC. *Bayesian Analysis*, 16(2):667–718, 2021.

A Further results on the final runs

Same fits, repeats and tests as Section 6. Table 4 gives each metric of this appendix for the exact posterior, the best a method can do.

metric	σ	ρ
mean error (vs exact mean); RMS error vs θ^*	0; 0.156	0; 0.050
relative first-moment error vs θ^*	0.102	0.060
coherence ratio	1.00	1.01
std(z); fraction $\lvert z\rvert < 1$; median $\lvert z\rvert$	1.02; 0.69; 0.71	0.97; 0.71; 0.67
test NLL (joint)	-2.390	
TARP coverage area; calibration error	$+0.005$; 0.005	
test datasets with $\lvert z\rvert > 3$ (%)	0.9	

Table 4: Each metric evaluated for the exact posterior (1,000 test datasets). Mean error, width error and the warranted-width ratio are 0, 0 and 1 by construction. The TARP values (1,000 grid-posterior draws) are the Monte Carlo floor of the test.

Reproduce: tables of this appendix: `python paper/scripts/appendix_tables.py` (paired differences, NPE variants, moment and TARP tables) and `paper/scripts/section6.py` (verdict, decomposition and width tables, TARP figure); both read `results/raw/final` and fit nothing (Appendix B).

A.1 Paired differences

Tables 6 and 7 give every paired comparison cited in Section 6. Table 5 condenses the comparisons of each BNN with each 12-member ensemble into a verdict.

BNN	$n = 50$	$n = 100$	$n = 500$	$n = 1{,}000$	$n = 5{,}000$	$n = 10{,}000$
BNN against NPE, 12 flows						
$\mathcal{N}(0, 1)$	tie $+0.08$	worse $+0.29$	tie $+0.04$	tie -0.02	**better** -0.16	**better** -0.18
data-free scale	**better** -0.16	worse $+0.20$	tie $+0.06$	tie $+0.04$	**better** -0.13	**better** -0.16
tempered	tie $+0.02$	worse $+0.18$	**better** -0.20	**better** -0.17	–	–
BNN against Gaussian predictor, 12 nets						
$\mathcal{N}(0, 1)$	worse $+0.19$	worse $+0.17$	tie $+0.06$	tie $+0.02$	**better** -0.24	**better** -0.21
data-free scale	tie -0.05	worse $+0.08$	tie $+0.08$	worse $+0.08$	**better** -0.21	**better** -0.19
tempered	worse $+0.13$	tie $+0.07$	**better** -0.19	**better** -0.13	–	–

Table 5: Each BNN against each 12-member ensemble: **better**, tie or worse by a two-sided paired t-test at $p < 0.05$ on the per-repeat test NLL (20 repeats at $n \leq 100$, 5 above; Section 5.3), with the mean difference BNN − ensemble in nats (negative: the BNN is lower). The tempered BNN is not run at $n \geq 5{,}000$. Against the long-trained NPE at $n = 10{,}000$ the $\mathcal{N}(0, 1)$ and data-free BNNs are also better, by 0.13 and 0.11 nats. Same fits as Table 3.

A.2 Inside the BNN: chains, draws and starting networks

The BNN is itself both sampled and ensembled, and Table 8 splits its NLL accordingly. Mixing the 12 chains adds little: the 12-chain NLL is 0.003–0.026 nats below a single chain's at $n \leq 1{,}000$, and 0.02–0.07 below it at $n \geq 5{,}000$. Ensembling the same 12 starting networks without sampling

	$n = 50$	$n = 100$	$n = 500$	$n = 1{,}000$	$n = 5{,}000$	$n = 10{,}000$
BNN vs NPE ensemble (thesis)						
BNN $\mathcal{N}(0,1)$ − NPE 12	$+0.078 \pm 0.299$ 9/20, t .257, W .245	$\mathbf{+0.288 \pm 0.193}$ 2/20, t <.001, W <.001	$+0.043 \pm 0.136$ 2/5, t .522	-0.021 ± 0.023 3/5, t .113	$\mathbf{-0.159 \pm 0.012}$ 5/5, t <.001	$\mathbf{-0.179 \pm 0.016}$ 5/5, t <.001
BNN data-free − NPE 12	$\mathbf{-0.162 \pm 0.281}$ 13/20, t .018, W .021	$\mathbf{+0.196 \pm 0.179}$ 3/20, t <.001, W <.001	$+0.061 \pm 0.129$ 2/5, t .349	$+0.038 \pm 0.034$ 1/5, t .066	$\mathbf{-0.129 \pm 0.012}$ 5/5, t <.001	$\mathbf{-0.165 \pm 0.016}$ 5/5, t <.001
BNN tempered − NPE 12	$+0.022 \pm 0.299$ 10/20, t .744, W .596	$\mathbf{+0.184 \pm 0.185}$ 5/20, t <.001, W <.001	$\mathbf{-0.204 \pm 0.115}$ 5/5, t .017	$\mathbf{-0.171 \pm 0.029}$ 5/5, t <.001	–	–
BNN $\mathcal{N}(0,1)$+retune − NPE 12	$+0.039 \pm 0.304$ 10/20, t .572, W .475	$\mathbf{+0.265 \pm 0.185}$ 1/20, t <.001, W <.001	-0.107 ± 0.109 4/5, t .093	$\mathbf{-0.143 \pm 0.029}$ 5/5, t <.001	$\mathbf{-0.159 \pm 0.014}$ 5/5, t <.001	$\mathbf{-0.177 \pm 0.014}$ 5/5, t <.001
BNN emp. Bayes − NPE 12	$\mathbf{+0.576 \pm 0.353}$ 2/20, t <.001, W <.001	$\mathbf{+0.185 \pm 0.171}$ 4/20, t <.001, W <.001	-0.087 ± 0.117 4/5, t .171	$\mathbf{-0.099 \pm 0.043}$ 5/5, t .006	$\mathbf{-0.111 \pm 0.018}$ 5/5, t <.001	$\mathbf{-0.141 \pm 0.011}$ 5/5, t <.001
BNN vs long-trained NPE ensemble ($n = 5{,}000$ and $10{,}000$ only)						
BNN $\mathcal{N}(0,1)$ − NPE 12 long	–	–	–	–	$\mathbf{-0.139 \pm 0.019}$ 5/5, t <.001	$\mathbf{-0.125 \pm 0.010}$ 5/5, t <.001
BNN data-free − NPE 12 long	–	–	–	–	$\mathbf{-0.109 \pm 0.017}$ 5/5, t <.001	$\mathbf{-0.111 \pm 0.007}$ 5/5, t <.001
BNN $\mathcal{N}(0,1)$+retune − NPE 12 long	–	–	–	–	$\mathbf{-0.138 \pm 0.020}$ 5/5, t <.001	$\mathbf{-0.123 \pm 0.007}$ 5/5, t <.001
BNN emp. Bayes − NPE 12 long	–	–	–	–	$\mathbf{-0.090 \pm 0.023}$ 5/5, t <.001	$\mathbf{-0.087 \pm 0.009}$ 5/5, t <.001
Sampling vs finished optimization: BNN vs converged Gaussian predictor (12 nets)						
BNN $\mathcal{N}(0,1)$ − Gaussian predictor 12 conv.	$\mathbf{+0.188 \pm 0.212}$ 4/20, t <.001, W .001	$\mathbf{+0.171 \pm 0.163}$ 3/20, t <.001, W <.001	$+0.058 \pm 0.074$ 1/5, t .156	$+0.024 \pm 0.030$ 1/5, t .150	$\mathbf{-0.235 \pm 0.011}$ 5/5, t <.001	$\mathbf{-0.208 \pm 0.014}$ 5/5, t <.001
BNN data-free − Gaussian predictor 12 conv.	-0.052 ± 0.192 12/20, t .238, W .231	$\mathbf{+0.079 \pm 0.134}$ 5/20, t .016, W .015	$+0.076 \pm 0.065$ 0/5, t .058	$\mathbf{+0.083 \pm 0.018}$ 0/5, t <.001	$\mathbf{-0.206 \pm 0.007}$ 5/5, t <.001	$\mathbf{-0.194 \pm 0.014}$ 5/5, t <.001
BNN tempered − Gaussian predictor 12 conv.	$\mathbf{+0.132 \pm 0.213}$ 6/20, t .012, W .012	$+0.067 \pm 0.149$ 6/20, t .060, W .070	$\mathbf{-0.189 \pm 0.066}$ 5/5, t .003	$\mathbf{-0.126 \pm 0.031}$ 5/5, t <.001	–	–
BNN $\mathcal{N}(0,1)$+retune − Gaussian predictor 12 conv.	$\mathbf{+0.149 \pm 0.226}$ 6/20, t .008, W .009	$\mathbf{+0.148 \pm 0.162}$ 4/20, t <.001, W <.001	$\mathbf{-0.092 \pm 0.058}$ 5/5, t .024	$\mathbf{-0.099 \pm 0.024}$ 5/5, t <.001	$\mathbf{-0.235 \pm 0.009}$ 5/5, t <.001	$\mathbf{-0.206 \pm 0.013}$ 5/5, t <.001
BNN emp. Bayes − Gaussian predictor 12 conv.	$\mathbf{+0.685 \pm 0.222}$ 0/20, t <.001, W <.001	$\mathbf{+0.068 \pm 0.136}$ 5/20, t .036, W .033	$\mathbf{-0.072 \pm 0.055}$ 5/5, t .044	$\mathbf{-0.055 \pm 0.008}$ 5/5, t <.001	$\mathbf{-0.187 \pm 0.009}$ 5/5, t <.001	$\mathbf{-0.170 \pm 0.013}$ 5/5, t <.001

Table 6: Paired differences in test NLL (nats), $A - B$ per repeat, mean ± sd over repeats (20 at $n \leq 100$, 5 above); negative: A is better. Below each difference: repeats in which A is better (k/R), the two-sided paired t-test p, and at $n \leq 100$ the two-sided Wilcoxon signed-rank p (W; with 5 repeats its smallest attainable p is 0.0625, so it is not shown). Bold: t-test $p < 0.05$. †: the Wilcoxon test disagrees about $p < 0.05$. Gaussian predictor: conv. = trained to convergence, capped = MILE's pretraining budget, the BNN's own start; 12 / 1: ensemble / member 0. Part 1: the BNN against NPE and against its converged unsampled networks; the long-trained NPE exists at $n = 5{,}000$ and 10,000 only, the tempered BNN is not run at $n \geq 5{,}000$.

	$n = 50$	$n = 100$	$n = 500$	$n = 1{,}000$	$n = 5{,}000$	$n = 10{,}000$
Sampling vs the BNN's own start: BNN vs capped Gaussian predictor (12 nets)						
BNN $\mathcal{N}(0,1)$ − Gaussian predictor 12 capped	$\mathbf{+0.112 \pm 0.198}$ 5/20, t .020, W .024	$\mathbf{+0.170 \pm 0.164}$ 3/20, t <.001, W <.001	$+0.026 \pm 0.079$ 2/5, t .498	$\mathbf{-0.075 \pm 0.019}$ 5/5, t <.001	$\mathbf{-0.424 \pm 0.020}$ 5/5, t <.001	$\mathbf{-0.470 \pm 0.016}$ 5/5, t <.001
BNN data-free − Gaussian predictor 12 capped	$\mathbf{-0.128 \pm 0.163}$ 15/20, t .002, W .002	$\mathbf{+0.079 \pm 0.135}$ 5/20, t .017, W .017	$+0.045 \pm 0.064$ 1/5, t .196	$\mathbf{-0.016 \pm 0.005}$ 5/5, t .003	$\mathbf{-0.394 \pm 0.019}$ 5/5, t <.001	$\mathbf{-0.456 \pm 0.015}$ 5/5, t <.001
BNN tempered − Gaussian predictor 12 capped	$+0.056 \pm 0.197$ 6/20, t .215, W .143	$+0.067 \pm 0.149$ 6/20, t .061, W .070	$\mathbf{-0.220 \pm 0.063}$ 5/5, t .001	$\mathbf{-0.225 \pm 0.025}$ 5/5, t <.001	–	–
Ensembling: 12 members vs member 0						
NPE 12 − NPE 1	$\mathbf{-1.019 \pm 1.255}$ 20/20, t .002, W <.001	$\mathbf{-0.656 \pm 0.349}$ 20/20, t <.001, W <.001	$\mathbf{-0.217 \pm 0.114}$ 5/5, t .013	$\mathbf{-0.202 \pm 0.086}$ 5/5, t .006	$\mathbf{-0.062 \pm 0.038}$ 5/5, t .022	$\mathbf{-0.031 \pm 0.014}$ 5/5, t .008
Gaussian predictor 12 conv. − Gaussian predictor 1 conv.	$-2.201 \pm 7.247^{\dagger}$ 17/20, t .190, W <.001	$\mathbf{-0.227 \pm 0.146}$ 19/20, t <.001, W <.001	$\mathbf{-0.171 \pm 0.009}$ 5/5, t <.001	$\mathbf{-0.124 \pm 0.043}$ 5/5, t .003	$\mathbf{-0.097 \pm 0.055}$ 5/5, t .016	$\mathbf{-0.056 \pm 0.011}$ 5/5, t <.001
Gaussian predictor 12 capped − Gaussian predictor 1 capped	$\mathbf{-0.100 \pm 0.157}$ 13/20, t .011, W .017	$\mathbf{-0.227 \pm 0.145}$ 19/20, t <.001, W <.001	$\mathbf{-0.135 \pm 0.029}$ 5/5, t <.001	$\mathbf{-0.078 \pm 0.012}$ 5/5, t <.001	-0.050 ± 0.060 4/5, t .136	-0.001 ± 0.024 3/5, t .936
Between the unsampled baselines						
Gaussian predictor 12 conv. − NPE 12	-0.110 ± 0.261 13/20, t .074, W .090	$\mathbf{+0.117 \pm 0.218}$ 5/20, t .027, W .030	-0.015 ± 0.083 3/5, t .709	-0.045 ± 0.049 4/5, t .111	$\mathbf{+0.076 \pm 0.016}$ 0/5, t <.001	$\mathbf{+0.029 \pm 0.018}$ 0/5, t .025
Gaussian predictor 12 conv. − Gaussian predictor 12 capped	$\mathbf{-0.076 \pm 0.085}$ 18/20, t <.001, W <.001	-0.000 ± 0.004 3/20, t .888, W .866	$\mathbf{-0.031 \pm 0.024}$ 5/5, t .045	$\mathbf{-0.098 \pm 0.018}$ 5/5, t <.001	$\mathbf{-0.188 \pm 0.023}$ 5/5, t <.001	$\mathbf{-0.262 \pm 0.022}$ 5/5, t <.001
Prior, likelihood and sampler variants						
BNN data-free − BNN $\mathcal{N}(0,1)$	$\mathbf{-0.240 \pm 0.072}$ 20/20, t <.001, W <.001	$\mathbf{-0.092 \pm 0.069}$ 17/20, t <.001, W <.001	$+0.018 \pm 0.020$ 1/5, t .111	$\mathbf{+0.059 \pm 0.015}$ 0/5, t <.001	$\mathbf{+0.030 \pm 0.005}$ 0/5, t <.001	$\mathbf{+0.014 \pm 0.005}$ 0/5, t .003
BNN tempered − BNN $\mathcal{N}(0,1)$	$\mathbf{-0.056 \pm 0.030}$ 20/20, t <.001, W <.001	$\mathbf{-0.104 \pm 0.044}$ 20/20, t <.001, W <.001	$\mathbf{-0.247 \pm 0.037}$ 5/5, t <.001	$\mathbf{-0.150 \pm 0.010}$ 5/5, t <.001	–	–
BNN $\mathcal{N}(0,1)$+retune − BNN $\mathcal{N}(0,1)$	$\mathbf{-0.039 \pm 0.037}$ 17/20, t <.001, W <.001	$-0.023 \pm 0.049^{\dagger}$ 14/20, t .052, W .027	$\mathbf{-0.150 \pm 0.042}$ 5/5, t .001	$\mathbf{-0.123 \pm 0.009}$ 5/5, t <.001	$+0.000 \pm 0.003$ 3/5, t .807	$+0.002 \pm 0.005$ 1/5, t .470
BNN emp. Bayes − BNN $\mathcal{N}(0,1)$+retune	$\mathbf{+0.536 \pm 0.242}$ 0/20, t <.001, W <.001	$\mathbf{-0.080 \pm 0.063}$ 18/20, t <.001, W <.001	$+0.020 \pm 0.018$ 1/5, t .069	$\mathbf{+0.044 \pm 0.018}$ 0/5, t .005	$\mathbf{+0.048 \pm 0.009}$ 0/5, t <.001	$\mathbf{+0.036 \pm 0.003}$ 0/5, t <.001

Table 7: Paired differences in test NLL (nats), $A - B$ per repeat, mean ± sd over repeats (20 at $n \leq 100$, 5 above); negative: A is better. Below each difference: repeats in which A is better (k/R), the two-sided paired t-test p, and at $n \leq 100$ the two-sided Wilcoxon signed-rank p (W; with 5 repeats its smallest attainable p is 0.0625, so it is not shown). Bold: t-test $p < 0.05$. †: the Wilcoxon test disagrees about $p < 0.05$. Gaussian predictor: conv. = trained to convergence, capped = MILE's pretraining budget, the BNN's own start; 12 / 1: ensemble / member 0. Part 2: the BNN against its own capped starting networks, ensembling, the unsampled baselines against each other, and the BNN variants; empirical Bayes is compared with its fixed-scale control on the same re-tuning sampler.

gains more at small n: 0.10 nats at $n = 50$, 0.23 at $n = 100$ and 0.08–0.14 at $n = 500$–1,000. The starting networks vary over training sets as much as the converged Gaussian predictor (standard deviation 0.16–0.18 at $n \leq 100$ for the 12-network mixture). Within one chain, averaging over the draws carries the predictive. Table 9 scores a single posterior draw used alone. Under $\mathcal{N}(0, 1)$ and tempering a typical draw has a test NLL of +46 and +83 at $n = 50$ (+7 and +8 at $n = 100$), because it is badly over-confident on a few test datasets: in the median draw of the $\mathcal{N}(0, 1)$ BNN at $n = 50$ (repeat 0) the median per-dataset NLL is +0.36, but 10 of the 1,000 test datasets carry 72% of the total, and on the worst the draw predicts a standard deviation of 0.04 for σ with the truth 18 standard deviations away and a correlation of 0.995. Mixing a chain's draws covers these failures. Under the data-free scale a typical draw scores −0.06, against −0.38 for its starting network and −0.60 for its chain. At $n = 5{,}000$ and 10,000 a typical draw of the untempered BNNs is below the converged 12-network ensemble, the observation behind the large-n paragraph of Section 6.2.

	$n = 50$	$n = 100$	$n = 500$	$n = 1{,}000$	$n = 5{,}000$	$n = 10{,}000$
The BNN's starting networks (capped Gaussian predictor)						
1 network (member 0)	-0.376 ± 0.215	-0.718 ± 0.205	-1.332 ± 0.031	-1.567 ± 0.016	-1.753 ± 0.065	-1.813 ± 0.021
12 networks	-0.475 ± 0.177	$\mathbf{-0.945 \pm 0.159}$	-1.467 ± 0.046	-1.646 ± 0.010	-1.803 ± 0.014	-1.814 ± 0.012
BNN, $\mathcal{N}(0, 1)$						
one chain	-0.346 ± 0.058	-0.749 ± 0.060	-1.429 ± 0.038	-1.716 ± 0.029	-2.187 ± 0.012	-2.211 ± 0.011
12 chains (the BNN)	-0.363 ± 0.059	-0.775 ± 0.062	-1.441 ± 0.039	-1.720 ± 0.029	$\mathbf{-2.226 \pm 0.009}$	$\mathbf{-2.284 \pm 0.005}$
BNN, data-free scale						
one chain	-0.599 ± 0.050	-0.862 ± 0.059	-1.420 ± 0.030	-1.659 ± 0.015	-2.177 ± 0.007	-2.231 ± 0.007
12 chains (the BNN)	$\mathbf{-0.603 \pm 0.050}$	-0.867 ± 0.060	-1.422 ± 0.030	-1.662 ± 0.015	-2.196 ± 0.006	-2.270 ± 0.003
BNN, tempered						
one chain	-0.400 ± 0.049	-0.854 ± 0.053	-1.674 ± 0.040	-1.857 ± 0.033	–	–
12 chains (the BNN)	-0.419 ± 0.049	$\mathbf{-0.879 \pm 0.050}$	$\mathbf{-1.687 \pm 0.043}$	$\mathbf{-1.870 \pm 0.034}$	–	–

Table 8: Test NLL of the BNN's components (nats; mean ± sd over repeats, as in Table 2). Starting networks: the capped Gaussian predictor, the networks MILE's chains start from (single network = member 0). One chain: the mixture of a chain's 1,000 draws, averaged over the 12 chains. 12 chains: the BNN as reported (recomputed from the per-draw outputs; matches the saved test NLL). Bold, per column: the lowest mean and every row not distinguishable from it by a two-sided paired t-test at $p < 0.05$.

	$n = 50$	$n = 100$	$n = 500$	$n = 1{,}000$	$n = 5{,}000$	$n = 10{,}000$
Optimized networks (Gaussian predictor)						
capped, 1 network (MILE's start)	-0.376 ± 0.215	-0.718 ± 0.205	-1.332 ± 0.031	-1.567 ± 0.016	-1.753 ± 0.065	-1.813 ± 0.021
converged, 1 network	$\mathbf{+1.650 \pm 7.236}^{\dagger}$	-0.718 ± 0.206	-1.327 ± 0.042	-1.621 ± 0.029	-1.893 ± 0.059	-2.021 ± 0.019
converged, 12 networks	$\mathbf{-0.551 \pm 0.199}$	$\mathbf{-0.946 \pm 0.159}$	$\mathbf{-1.498 \pm 0.040}$	$\mathbf{-1.744 \pm 0.021}$	-1.991 ± 0.010	-2.076 ± 0.014
A single posterior draw, used alone (median over the $12 \times 1{,}000$ draws)						
BNN, $\mathcal{N}(0, 1)$	$+45.855 \pm 19.644$	$+7.067 \pm 4.575$	-0.990 ± 0.043	-1.428 ± 0.031	-2.033 ± 0.036	-2.172 ± 0.015
BNN, data-free scale	-0.064 ± 0.143	-0.419 ± 0.099	-1.159 ± 0.027	-1.461 ± 0.019	$\mathbf{-2.086 \pm 0.009}$	$\mathbf{-2.212 \pm 0.005}$
BNN, tempered	$+83.272 \pm 34.708$	$+8.399 \pm 6.595$	-0.977 ± 0.162	-1.458 ± 0.053	–	–

Table 9: A single sampled network against optimized networks: test NLL (nats; mean ± sd over repeats, 20 at $n \leq 100$, 5 above). Optimized: the Gaussian predictor, capped (the network MILE's chains start from, member 0) and converged (member 0, and the 12-network ensemble). Single posterior draw: for each of the $12 \times 1{,}000$ draws its own test NLL, the median over draws (the mean over draws is dominated by a few draws with near-zero width at small n). Bold, per column: the lowest mean and every row not distinguishable from it by a two-sided paired t-test at $p < 0.05$; †: at $n \leq 100$ the Wilcoxon signed-rank test disagrees (the converged single network at $n = 50$, bolded only because one repeat at +32.1 inflates its sd).

A.3 NPE head and encoder width

Two one-change variants of NPE separate the contributions of its encoder and of its flow head. Both are 12-flow ensembles on the same training rows, split, seeds, learning rate and weight decay as NPE, at the default budget at $n \leq 100$ and the long budget at $n = 10{,}000$, where every NPE row is the long-trained fit. *Flow, encoder 16* widens the encoder output from 8 to 16, giving the BNN's hidden widths ($100 \to 16 \to 16 \to 16$); *Gaussian head, encoder 16* replaces that variant's flow by a bivariate Gaussian head. Encoder capacity does not matter: the wider encoder is 0.09 nats worse at $n = 50$ and no different at $n = 100$ or 10,000, and its mean errors are within 0.001 of NPE's. The head does. With a Gaussian head the error of the predicted mean of σ rises to the BNNs' level at $n = 100$ (0.126 against 0.096 for the flow, worse on 16 of 20 repeats; the Wilcoxon test separates them, the t-test does not, because of a few extreme repeats) and above it at $n = 10{,}000$ (0.048 against 0.037, 5 of 5), and the NLL is worse by 0.16 nats at $n = 10{,}000$ (5 of 5). At $n = 50$ the Gaussian-head NPE is unstable (median NLL over repeats +1.30, one repeat at +4,987). NPE's advantage in the predicted mean of σ at small n therefore comes from its flow head, which can represent the skew of σ's posterior (Section 3.4), not from its encoder.

	$n = 50$	$n = 100$	$n = 10{,}000$
Sampled BNN			
BNN, $\mathcal{N}(0, 1)$	-0.363 ± 0.059	-0.775 ± 0.062	$\mathbf{-2.284 \pm 0.005}$
BNN, data-free scale	$\mathbf{-0.603 \pm 0.050}$	-0.867 ± 0.060	-2.270 ± 0.003
BNN, tempered	-0.419 ± 0.049	-0.879 ± 0.050	–
BNN without sampling			
Gaussian predictor, 12 nets, capped	-0.475 ± 0.177	-0.945 ± 0.159	-1.814 ± 0.012
Gaussian predictor, 12 nets, converged	$\mathbf{-0.551 \pm 0.199}$	-0.946 ± 0.159	-2.076 ± 0.014
NPE-style, 12 networks			
Flow, encoder 8 (= NPE)	-0.441 ± 0.298	$\mathbf{-1.062 \pm 0.186}$	-2.159 ± 0.007
Flow, encoder 16	-0.351 ± 0.249	$\mathbf{-1.077 \pm 0.161}$	-2.147 ± 0.019
Gaussian head, encoder 16	$\mathbf{+252.274 \pm 1114.377}^{\dagger}$	$\mathbf{-0.997 \pm 0.216}^{\dagger}$	-1.984 ± 0.050
NPE-style, 1 network (member 0)			
Flow, encoder 8 (= NPE), 1 net	$+0.578 \pm 1.205$	-0.407 ± 0.348	-2.077 ± 0.044
Flow, encoder 16, 1 net	$\mathbf{+1.500 \pm 4.738}^{\dagger}$	-0.154 ± 0.919	-2.061 ± 0.047
Gaussian head, encoder 16, 1 net	$\mathbf{+1874259.521 \pm 8371757.331}^{\dagger}$	$+8.119 \pm 19.538$	-1.910 ± 0.146

Table 10: NPE variants: test NLL (nats), mean ± sd over repeats (20 at $n \leq 100$, 5 at $n = 10{,}000$); exact posterior −2.390. Bold: the lowest mean and every row not distinguishable from it by a two-sided paired t-test at $p < 0.05$; †: the Wilcoxon signed-rank test disagrees. The Gaussian-head rows at $n = 50$ have extreme repeats (12 networks: median +1.30, maximum +4,987; one network: median +59); their bolding there is an artefact of the inflated sd (each loses to the best row on 20 of 20 repeats). `python paper/scripts/appendix_tables.py`.

A.4 Fairness of the comparison

The comparisons of Section 6 are paired on training rows and test set, but the methods differ in budget, tuning and data use. Each difference is listed below with its direction and, where possible, its size.

- **Training budget of NPE.** NPE runs at its default budget (at most 20,000 steps, patience 20 evaluations) except at $n = 10{,}000$, where the long-trained ensemble is used. At $n = 10{,}000$

the default budget cost 0.054 nats (Section 4.3.2). At $n = 5{,}000$ a long-trained ensemble, run after the tables of Section 6 were made, improves on the default budget by 0.02 nats and leaves the untempered BNNs 0.11–0.14 nats ahead (Section 7.6). Favours the BNN at $n = 5{,}000$ by about 0.02 nats.

- **Compute.** A BNN member uses at most 2,000 pretraining steps, 50,000 warmup steps and 10,000 sampling steps, each a full-batch gradient: about three times the gradient evaluations of a default-budget flow. The converged Gaussian predictor (up to 67,500 steps) and the long-trained NPE (up to 84,400) remove the optimization-budget part of this difference where they are used. Favours the BNN where the comparator is at its default budget.

- **Hyperparameter search.** NPE's learning rate and weight decay come from a Bayesian-optimization search of 3–20 trials per size that acted as a random search of 3–10 points (Appendix C) on one 80/20 split of the repeat-0 training rows; the BNN's and the Gaussian predictor's from a 255-point grid scored on three repeats (Section 4.6). The converged Gaussian predictor keeps values chosen under the 2,000-step budget, and at $n = 50$ the grid's argmin is noise-driven. Favours the BNN over NPE in tuning effort; disfavours the converged Gaussian predictor.

- **Data use.** The BNN's sampling stage conditions on all n training rows; the Gaussian predictor and NPE fit 80% of them and use the rest for early stopping (Section 4.1). A rough estimate, extrapolating each ensemble's improvement from $n = 5{,}000$ to 10,000 log-linearly, puts the value of 25% more rows at about 0.01 nats for NPE and 0.03 for the Gaussian predictor at large n, against margins of 0.11–0.24. Favours the BNN at every n.

- **Architecture.** NPE's encoder narrows to 8 dimensions where the BNN's narrowest layer has 16; an encoder of the BNN's width did not improve NPE (worse by 0.09 nats at $n = 50$, no difference at $n = 100$ or 10,000; Appendix A.3). NPE's flow head, in turn, can represent the skew of σ's posterior, which the Gaussian heads cannot (Section 3.4).

- **Test set.** All repeats share one test set of 1,000 datasets, whose sampling uncertainty the paired tests do not include. It is comparable to the repeat-to-repeat uncertainty at $n \leq 100$ (0.05 against 0.06 nats for the data-free BNN against NPE at $n = 50$) and larger than it at $n = 10{,}000$ (0.014 against 0.004), where the margins are about 0.12.

At $n \leq 100$, where NPE matches or beats the BNN, every difference that favours a method favours the BNN, so it strengthens that conclusion. At large n the differences favouring the BNN are each smaller than its margin, but they have not been removed jointly. The long-trained NPE at $n = 5{,}000$ has closed one of the two largest (0.02 nats); an NPE fitted on all n rows would close the other.

A.5 Width and coherence

The aleatoric/epistemic split. With a heteroskedastic likelihood, in which the network outputs a mean and a noise scale per input, the predictive variance splits by the law of total variance into an aleatoric part (the expected predicted noise, irreducible) and an epistemic part (the variance of the predicted mean over the posterior, reducible with data), as in Eq. (10) [7]. In their experiments the aleatoric term stays roughly constant across training-set sizes while the epistemic term decreases as data grow. Bickford Smith et al. [47] list this behaviour with varying n (Table 3 of Kendall and Gal [7]) among the "misleading implications" of the aleatoric/epistemic view: for finite n the

model's expected conditional uncertainty "is only an estimator" of the uncertainty that would remain after infinite data, "and it can be highly inaccurate", and the irreducible/reducible split holds only for a well-specified model with exact updating, "with any model misspecification or error in belief updating reintroducing the dependency". Their statements are for entropy; under squared loss their uncertainty measure is the variance (their Example 1), and the expected predicted noise variance of Eq. (10) takes the same role here. The BNN's aleatoric term is therefore an estimate of the irreducible width, to be checked rather than assumed. In SBI the role of (x, y) is played by (S, θ): the exact posterior width given S is the irreducible part, and the epistemic part should vanish as $n \to \infty$ and grow as n shrinks. Because the exact posterior is known, whether the BNN's widths are right, and in which term an excess lies, can be measured directly.

Metrics. With $\hat{\mu}_j$, $\hat{s}_j$ the mean and total standard deviation of a method's predictive and $\mu_j^\star$, $s_j^\star$ those of the exact posterior, per parameter: the *width relative to the warranted width* is $\big(\sum_j \hat{s}_j^2 / \sum_j \big[(s_j^\star)^2 + (\hat{\mu}_j - \mu_j^\star)^2\big]\big)^{1/2}$, the RMS width over the RMS width a calibrated predictive needs given its own mean error; 1 is ideal. (A mean of per-dataset ratios would divide by each dataset's realized error and is biased upward: a calibrated predictive at the BNN's error level scores 1.1–1.7 on it at $n \le 100$.) The *coherence ratio* $\big(\frac{1}{M}\sum_j \hat{s}_j^2 + \widehat{\mathrm{Var}}_j(\hat{\mu}_j)\big) / \widehat{\mathrm{Var}}_j(\theta_j^*)$ is the law-of-total-variance condition of Section 2.4 applied to the predictive: 1 for a coherent predictive, above 1 when the widths and the spread of the means together claim more uncertainty than the prior has; it uses only the prior and is necessary, not sufficient (the prior itself scores 1). The *aleatoric share of the excess* is the part of the variance in excess of the exact posterior's and of the mean error that lies in the aleatoric term of Eq. (10).

Results. Under $\mathcal{N}(0, 1)$ the sampled BNN's predictive at $n \le 100$ is wider than its own errors warrant and claims more uncertainty than the prior has (Table 12); most of the excess is in the aleatoric term, an instance of the finite-n estimation error of Bickford Smith et al. [47], measured here against a known irreducible width. Tempering does not narrow it at $n = 50$ and narrows it from $n = 100$; the data-free scale removes most of it at every size. The Gaussian predictor's ensemble is slightly too narrow and claims less variance than the prior at small n, and the NPE ensemble is close to warranted except at $n = 50$.

A.6 Moments and calibration

Metrics. All metrics are computed per parameter, with $\hat{\mu}_j$, $\hat{s}_j$ the mean and total standard deviation of a method's predictive and $\mu_j^\star$, $s_j^\star$ those of the exact posterior. *Mean error*: $\big(\frac{1}{M}\sum_j (\hat{\mu}_j - \mu_j^\star)^2\big)^{1/2}$, 0 for the exact posterior; against the true parameter the RMS error is bounded below, in expectation, by that of the exact posterior mean (0.156 for σ and 0.050 for ρ), and the relative error against the true parameter, $\frac{1}{M}\sum_j |\hat{\mu}_j - \theta_j^*| / |\theta_j^*|$ (denominator floored at 10^{-3}), appears in Appendix G. *Width error*: $\frac{1}{M}\sum_j |\hat{s}_j - s_j^\star| / s_j^\star$. *Spread of z*: $\mathrm{std}(z)$ with $z_j = (\hat{\mu}_j - \theta_j^*)/\hat{s}_j$, above 1 for an over-confident method and below 1 for an under-confident one; the fraction with $|z_j| < 1$ is 0.68 for a calibrated Gaussian. *TARP* [18] is a joint coverage test from samples, necessary and sufficient for an accurate posterior estimator. Parameters are rescaled to the unit box spanned by the reference grid; for each test dataset a reference point θ_j^{r} is drawn uniformly in the box, and f_j is the fraction of 1,000 predictive draws closer to θ_j^{r} than θ_j^* is. The expected coverage $\mathrm{ECP}(\alpha) = \frac{1}{M}\#\{j : f_j \le \alpha\}$ equals α for a calibrated estimator and is averaged over 20 draws of the reference points. It is reported as the coverage area $\int_0^1 (\mathrm{ECP}(\alpha) - \alpha)\, \mathrm{d}\alpha$ (positive = conservative, negative = over-confident) and the calibration error $\frac{1}{51}\sum_\alpha |\mathrm{ECP}(\alpha) - \alpha|$ on a 51-point grid.

Results. Tables 13–15 give the error of the predicted mean against the exact posterior mean, the

relative error of the predictive standard deviation, and the TARP calibration error, for every model including the capped Gaussian predictor and the appendix variants. Figure 7 shows the TARP coverage curves of the five ensembled models.

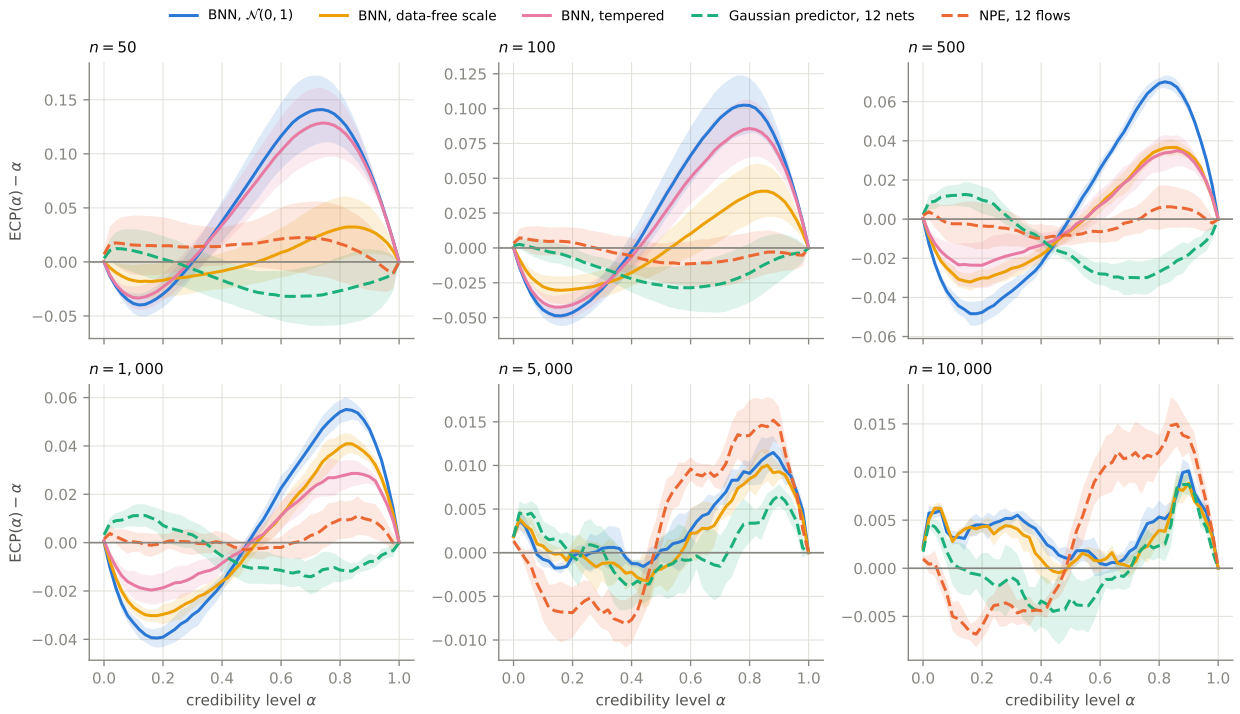


Figure 7: TARP calibration of the three BNNs and the two 12-member ensembles: expected coverage minus credibility level, $\mathrm{ECP}(\alpha) - \alpha$, against α. 0: credible regions contain the truth as often as they claim; above 0: conservative (too wide); below 0: over-confident. Line: pooled over repeats (20 at $n \leq 100$, 5 above); band: ± 1 sd over the per-repeat curves. The coverage area $\int(\mathrm{ECP}(\alpha) - \alpha)\,\mathrm{d}\alpha$ at $n = 50$ is $+0.052$, $+0.003$ and $+0.049$ for the $\mathcal{N}(0,1)$, data-free and tempered BNNs, -0.015 for the Gaussian predictor and $+0.014$ for NPE.

A.7 Empirical Bayes and the re-tuning sampler

Empirical Bayes (Section 4.4.3) runs on the re-tuning sampler, and its fixed-scale control is $\mathcal{N}(0,1)$ with the same re-tuning (rows in Tables 7 and 13–15). **Re-tuning alone** lowers the NLL of $\mathcal{N}(0,1)$ by 0.15 and 0.12 nats at $n = 500$ and 1,000 ($p \leq 0.002$) and changes nothing at $n \geq 5{,}000$. **Empirical Bayes** recovers the data-free scale where it is stable: its final scale is 0.37, 0.36 and 0.32 at $n = 100$, 500 and 1,000, against the data-free 0.3655, and 0.32 and 0.28 at $n = 5{,}000$ and 10,000. Against its control it is better at $n = 100$ (by 0.08 nats, $p = 2 \times 10^{-5}$) and worse from $n = 1{,}000$ (by 0.04–0.05). At $n = 50$ it collapses in all 20 repeats: the scale falls to the floor of 10^{-3} and the network becomes a nearly constant predictor: across test datasets its predicted width varies by at most 3.3% (coefficient of variation) and its predicted mean has a standard deviation of at most 12% of the prior's. It then returns the training mean with a width of about 0.75 times the training standard deviation of θ, a value set by softplus(0) because the output biases are shrunk with every other weight, and its NLL is 0.54 nats above its control. This is the underfitting the marginal likelihood is prone to at small n [28], and the sample-based update of the same form is

reported to diverge to very small regularizers [27]. A per-layer version (exploratory 3-member runs, original learning rates) did not rescue $n = 50$ and was worse than the isotropic one at $n = 500$ and 1,000 (by 0.09 and 0.03 nats, 0 of 5 repeats), although MacKay [25] argues that one scale for all weight classes is inconsistent. Fits that dropped a failed member without replacing it: re-tuning 10 of 20 at $n = 50$, 7 of 20 at $n = 100$ and 2 of the 20 at $n \geq 500$; empirical Bayes 1, 6 and 1; every fit of the other models used all 12 members.

	$n = 50$	$n = 100$	$n = 10{,}000$
σ			
Sampled BNN			
BNN, $\mathcal{N}(0, 1)$	0.143 ± 0.013	0.124 ± 0.010	$\mathbf{0.025 \pm 0.001}$
BNN, data-free scale	0.145 ± 0.012	0.127 ± 0.008	0.026 ± 0.000
BNN, tempered	0.142 ± 0.015	0.119 ± 0.011	–
BNN without sampling			
Gaussian predictor, 12 nets, capped	0.167 ± 0.021	0.127 ± 0.011	0.055 ± 0.001
Gaussian predictor, 12 nets, converged	0.159 ± 0.023	0.127 ± 0.011	0.037 ± 0.001
NPE-style, 12 networks			
Flow, encoder 8 (= NPE)	$\mathbf{0.120 \pm 0.038}$	$\mathbf{0.097 \pm 0.008}$	0.036 ± 0.001
Flow, encoder 16	$\mathbf{0.119 \pm 0.029}$	$\mathbf{0.096 \pm 0.010}$	0.037 ± 0.001
Gaussian head, encoder 16	0.251 ± 0.146	$\mathbf{0.126 \pm 0.069}^\dagger$	0.048 ± 0.004
NPE-style, 1 network (member 0)			
Flow, encoder 8 (= NPE), 1 net	$\mathbf{0.127 \pm 0.032}$	0.107 ± 0.013	0.043 ± 0.002
Flow, encoder 16, 1 net	$\mathbf{0.129 \pm 0.034}^\dagger$	0.105 ± 0.010	0.044 ± 0.006
Gaussian head, encoder 16, 1 net	0.251 ± 0.143	0.128 ± 0.021	0.052 ± 0.008
ρ			
Sampled BNN			
BNN, $\mathcal{N}(0, 1)$	0.137 ± 0.009	0.108 ± 0.007	$\mathbf{0.012 \pm 0.000}$
BNN, data-free scale	0.140 ± 0.008	0.114 ± 0.006	0.014 ± 0.000
BNN, tempered	0.133 ± 0.009	0.102 ± 0.006	–
BNN without sampling			
Gaussian predictor, 12 nets, capped	0.144 ± 0.012	0.109 ± 0.015	0.050 ± 0.001
Gaussian predictor, 12 nets, converged	0.134 ± 0.018	0.109 ± 0.015	0.032 ± 0.001
NPE-style, 12 networks			
Flow, encoder 8 (= NPE)	$\mathbf{0.124 \pm 0.018}$	$\mathbf{0.093 \pm 0.015}$	0.024 ± 0.001
Flow, encoder 16	$\mathbf{0.124 \pm 0.019}$	$\mathbf{0.092 \pm 0.015}$	0.025 ± 0.001
Gaussian head, encoder 16	0.194 ± 0.059	$\mathbf{0.107 \pm 0.039}^\dagger$	0.036 ± 0.004
NPE-style, 1 network (member 0)			
Flow, encoder 8 (= NPE), 1 net	0.144 ± 0.034	$\mathbf{0.095 \pm 0.016}^\dagger$	0.027 ± 0.002
Flow, encoder 16, 1 net	$0.136 \pm 0.032^\dagger$	$\mathbf{0.099 \pm 0.021}^\dagger$	0.027 ± 0.003
Gaussian head, encoder 16, 1 net	0.192 ± 0.066	0.119 ± 0.022	0.038 ± 0.008

Table 11: NPE variants: error of the predicted mean against the exact posterior mean (Appendix A.6), σ (top) and ρ (bottom), bolded separately as in Table 10.

	$n = 50$	$n = 100$	$n = 500$	$n = 1{,}000$	$n = 5{,}000$	$n = 10{,}000$
Width over warranted width (1: width explained by the method's own mean error)						
BNN, $\mathcal{N}(0, 1)$	2.38 / 2.42	1.76 / 2.10	1.23 / 1.45	1.16 / 1.34	1.01 / 1.05	0.99 / 1.01
BNN, data-free scale	1.15 / 1.20	1.12 / 1.29	1.09 / 1.24	1.09 / 1.26	1.00 / 1.05	1.00 / 1.01
BNN, tempered	2.43 / 2.53	1.56 / 2.03	1.09 / 1.26	1.07 / 1.21	–	–
Gaussian predictor, 12 nets	0.92 / 0.98	0.96 / 1.04	0.95 / 0.96	0.97 / 0.99	0.99 / 1.04	1.00 / 1.04
NPE, 12 flows	1.19 / 1.37	0.97 / 1.14	1.00 / 1.10	0.99 / 1.08	1.01 / 1.04	1.01 / 1.05
Coherence ratio (1: coherent; exact posterior 1.00 / 1.01)						
BNN, $\mathcal{N}(0, 1)$	3.09 / 3.67	1.77 / 2.14	1.03 / 1.08	1.02 / 1.06	0.99 / 1.01	0.99 / 1.01
BNN, data-free scale	1.00 / 1.09	0.98 / 1.08	0.95 / 0.98	0.99 / 1.01	0.99 / 1.00	0.99 / 1.00
BNN, tempered	3.21 / 3.86	1.51 / 1.95	0.99 / 1.03	1.00 / 1.03	–	–
Gaussian predictor, 12 nets	0.74 / 0.81	0.82 / 0.84	0.87 / 0.85	0.94 / 0.94	0.98 / 0.98	0.97 / 0.99
NPE, 12 flows	1.44 / 1.38	0.94 / 0.97	0.96 / 0.95	0.99 / 0.99	0.99 / 1.01	0.99 / 1.01
Aleatoric share of the excess variance						
BNN, $\mathcal{N}(0, 1)$	0.85 / 0.92	0.84 / 0.92	0.90 / 0.98	0.85 / 0.98	–	–
BNN, tempered	0.86 / 0.93	0.81 / 0.92	0.55 / 0.96	0.62 / 1.05	–	–

Table 12: Width metrics defined above, mean over repeats, σ / ρ. Width over warranted width: RMS width over the RMS width a calibrated predictive needs given its own mean error. Coherence ratio: (mean predicted variance + variance of the predicted mean) / variance of θ^* over the test set. Aleatoric share: of the variance in excess of the exact posterior's and of the mean error, the part in the aleatoric term, for the two BNNs with an excess (undefined where width / warranted ≈ 1, so not shown at $n \geq 5{,}000$).

	$n = 50$	$n = 100$	$n = 500$	$n = 1{,}000$	$n = 5{,}000$	$n = 10{,}000$
σ						
Sampled BNN						
BNN, $\mathcal{N}(0, 1)$	0.143 ± 0.013	0.124 ± 0.010	0.084 ± 0.006	0.067 ± 0.002	$\mathbf{0.032 \pm 0.002}$	$\mathbf{0.025 \pm 0.001}$
BNN, data-free scale	0.145 ± 0.012	0.127 ± 0.008	0.088 ± 0.004	0.070 ± 0.001	0.035 ± 0.002	0.026 ± 0.000
BNN, tempered	0.142 ± 0.015	0.119 ± 0.011	$\mathbf{0.074 \pm 0.005}$	$\mathbf{0.062 \pm 0.005}$	–	–
BNN without sampling						
Gaussian predictor, 12 nets, capped	0.167 ± 0.021	0.127 ± 0.011	0.090 ± 0.003	0.081 ± 0.002	0.055 ± 0.001	0.055 ± 0.001
Gaussian predictor, 12 nets, converged	0.159 ± 0.023	0.127 ± 0.011	0.087 ± 0.002	0.072 ± 0.003	0.043 ± 0.001	0.037 ± 0.001
Gaussian predictor, 1 net, capped	0.166 ± 0.022	0.134 ± 0.017	0.094 ± 0.005	0.082 ± 0.005	0.062 ± 0.005	0.055 ± 0.002
Gaussian predictor, 1 net, converged	0.160 ± 0.024	0.134 ± 0.017	0.092 ± 0.005	0.077 ± 0.004	0.050 ± 0.003	0.043 ± 0.001
NPE						
NPE, 12 flows	$\mathbf{0.120 \pm 0.038}$	$\mathbf{0.097 \pm 0.008}$	$\mathbf{0.080 \pm 0.005}$	$\mathbf{0.062 \pm 0.003}$	0.054 ± 0.003	0.041 ± 0.001
NPE, 1 flow	$\mathbf{0.127 \pm 0.032}^{\dagger}$	0.107 ± 0.013	0.087 ± 0.007	0.070 ± 0.003	0.055 ± 0.006	0.044 ± 0.002
Appendix (retune sampler)						
BNN, $\mathcal{N}(0, 1)$ + retune	0.143 ± 0.015	0.126 ± 0.018	$\mathbf{0.076 \pm 0.004}$	$\mathbf{0.061 \pm 0.003}$	$\mathbf{0.032 \pm 0.002}$	$\mathbf{0.025 \pm 0.000}$
BNN, empirical Bayes	0.256 ± 0.009	0.128 ± 0.009	0.081 ± 0.004	0.065 ± 0.002	0.037 ± 0.001	0.029 ± 0.000
ρ						
Sampled BNN						
BNN, $\mathcal{N}(0, 1)$	0.137 ± 0.009	0.108 ± 0.007	0.062 ± 0.003	0.046 ± 0.002	$\mathbf{0.017 \pm 0.001}$	$\mathbf{0.012 \pm 0.000}$
BNN, data-free scale	0.140 ± 0.008	0.114 ± 0.006	0.073 ± 0.002	0.056 ± 0.001	0.020 ± 0.001	0.014 ± 0.000
BNN, tempered	0.133 ± 0.009	0.102 ± 0.006	$\mathbf{0.050 \pm 0.002}$	$\mathbf{0.040 \pm 0.001}$	–	–
BNN without sampling						
Gaussian predictor, 12 nets, capped	0.144 ± 0.012	0.109 ± 0.015	0.072 ± 0.006	0.060 ± 0.001	0.050 ± 0.001	0.050 ± 0.001
Gaussian predictor, 12 nets, converged	0.134 ± 0.018	0.109 ± 0.015	0.069 ± 0.005	0.051 ± 0.002	0.037 ± 0.000	0.032 ± 0.001
Gaussian predictor, 1 net, capped	0.146 ± 0.015	0.117 ± 0.018	0.071 ± 0.003	0.062 ± 0.001	0.054 ± 0.005	0.050 ± 0.001
Gaussian predictor, 1 net, converged	$\mathbf{0.135 \pm 0.022}$	0.117 ± 0.018	0.069 ± 0.004	0.053 ± 0.002	0.039 ± 0.001	0.034 ± 0.001
NPE						
NPE, 12 flows	$\mathbf{0.124 \pm 0.018}$	$\mathbf{0.093 \pm 0.015}$	0.066 ± 0.009	0.052 ± 0.004	0.030 ± 0.001	0.028 ± 0.000
NPE, 1 flow	0.144 ± 0.034	$\mathbf{0.095 \pm 0.016}$	0.065 ± 0.008	0.060 ± 0.010	0.029 ± 0.002	0.028 ± 0.001
Appendix (retune sampler)						
BNN, $\mathcal{N}(0, 1)$ + retune	0.136 ± 0.008	0.107 ± 0.006	0.055 ± 0.002	0.041 ± 0.001	$\mathbf{0.017 \pm 0.001}$	0.012 ± 0.000
BNN, empirical Bayes	0.185 ± 0.004	0.115 ± 0.006	0.064 ± 0.001	0.050 ± 0.001	0.023 ± 0.000	0.017 ± 0.000

Table 13: Mean error: RMS over the test datasets of (predicted mean − exact posterior mean), σ (top) and ρ (bottom), bolded separately; 0 for the exact posterior. Mean ± sd over repeats (20 at $n \leq 100$, 5 above), same fits as Section 6; bold: lowest mean and every row not distinguishable from it by a two-sided paired t-test at $p < 0.05$; †: the Wilcoxon test disagrees.

	$n = 50$	$n = 100$	$n = 500$	$n = 1{,}000$	$n = 5{,}000$	$n = 10{,}000$
σ						
Sampled BNN						
BNN, $\mathcal{N}(0, 1)$	2.234 ± 0.304	1.269 ± 0.191	0.451 ± 0.035	0.298 ± 0.013	$\mathbf{0.058 \pm 0.003}$	$\mathbf{0.042 \pm 0.001}$
BNN, data-free scale	0.624 ± 0.157	0.493 ± 0.107	0.291 ± 0.017	0.231 ± 0.016	$\mathbf{0.057 \pm 0.003}$	$\mathbf{0.042 \pm 0.001}$
BNN, tempered	2.264 ± 0.393	0.994 ± 0.164	0.249 ± 0.035	0.179 ± 0.008	–	–
BNN without sampling						
Gaussian predictor, 12 nets, capped	0.396 ± 0.105	0.306 ± 0.086	$\mathbf{0.172 \pm 0.027}$	0.161 ± 0.011	0.115 ± 0.012	0.119 ± 0.004
Gaussian predictor, 12 nets, converged	$\mathbf{0.358 \pm 0.111}$	0.306 ± 0.086	$\mathbf{0.165 \pm 0.021}$	0.137 ± 0.004	0.069 ± 0.005	0.058 ± 0.002
Gaussian predictor, 1 net, capped	$\mathbf{0.361 \pm 0.086}$	0.328 ± 0.079	$\mathbf{0.163 \pm 0.026}$	0.150 ± 0.010	0.102 ± 0.020	0.096 ± 0.005
Gaussian predictor, 1 net, converged	$\mathbf{0.349 \pm 0.096}$	0.332 ± 0.082	$\mathbf{0.160 \pm 0.025}$	0.127 ± 0.014	0.071 ± 0.008	0.063 ± 0.004
NPE						
NPE, 12 flows	0.599 ± 0.367	$\mathbf{0.264 \pm 0.078}$	$\mathbf{0.213 \pm 0.045}$	$\mathbf{0.114 \pm 0.024}$	0.106 ± 0.005	0.077 ± 0.003
NPE, 1 flow	0.516 ± 0.257	$\mathbf{0.257 \pm 0.063}$	$\mathbf{0.203 \pm 0.066}$	$\mathbf{0.099 \pm 0.009}$	0.091 ± 0.011	0.071 ± 0.006
Appendix (retune sampler)						
BNN, $\mathcal{N}(0, 1)$ + retune	1.995 ± 0.485	1.310 ± 0.536	0.320 ± 0.014	0.205 ± 0.017	$\mathbf{0.057 \pm 0.004}$	$\mathbf{0.043 \pm 0.001}$
BNN, empirical Bayes	0.502 ± 0.137	0.481 ± 0.110	0.214 ± 0.013	0.145 ± 0.013	0.062 ± 0.003	0.046 ± 0.000
ρ						
Sampled BNN						
BNN, $\mathcal{N}(0, 1)$	8.916 ± 1.246	6.023 ± 0.822	2.134 ± 0.094	1.308 ± 0.082	$\mathbf{0.175 \pm 0.005}$	$\mathbf{0.091 \pm 0.003}$
BNN, data-free scale	3.965 ± 0.538	3.402 ± 0.360	1.754 ± 0.058	1.265 ± 0.044	0.187 ± 0.007	0.100 ± 0.002
BNN, tempered	9.067 ± 1.139	5.431 ± 0.769	1.280 ± 0.123	0.846 ± 0.090	–	–
BNN without sampling						
Gaussian predictor, 12 nets, capped	3.052 ± 0.487	2.312 ± 0.498	1.030 ± 0.288	0.781 ± 0.030	0.720 ± 0.030	0.735 ± 0.034
Gaussian predictor, 12 nets, converged	2.865 ± 0.567	2.317 ± 0.499	0.948 ± 0.202	0.594 ± 0.057	0.410 ± 0.016	0.310 ± 0.021
Gaussian predictor, 1 net, capped	2.895 ± 0.488	2.261 ± 0.520	$\mathbf{0.727 \pm 0.036}$	0.692 ± 0.060	0.663 ± 0.096	0.645 ± 0.042
Gaussian predictor, 1 net, converged	$\mathbf{2.559 \pm 0.763}$	2.281 ± 0.496	$\mathbf{0.659 \pm 0.057}$	$\mathbf{0.468 \pm 0.058}$	0.348 ± 0.013	0.294 ± 0.040
NPE						
NPE, 12 flows	3.615 ± 1.455	1.944 ± 0.534	1.185 ± 0.363	0.809 ± 0.131	0.375 ± 0.025	0.345 ± 0.019
NPE, 1 flow	$\mathbf{3.490 \pm 2.022}$	$\mathbf{1.276 \pm 0.476}$	$\mathbf{0.826 \pm 0.279}$	$\mathbf{0.746 \pm 0.222}$	0.242 ± 0.039	0.266 ± 0.015
Appendix (retune sampler)						
BNN, $\mathcal{N}(0, 1)$ + retune	8.167 ± 1.045	6.052 ± 1.323	1.655 ± 0.080	0.992 ± 0.061	$\mathbf{0.178 \pm 0.007}$	0.107 ± 0.002
BNN, empirical Bayes	3.230 ± 0.549	3.350 ± 0.334	1.397 ± 0.066	0.864 ± 0.044	0.208 ± 0.004	0.138 ± 0.002

Table 14: Width error: mean over the test datasets of $|\hat{s} - s^\star|/s^\star$, σ (top) and ρ (bottom), bolded separately. Too narrow scores at most 1, too wide is unbounded. Mean $\pm$ sd over repeats (20 at $n \leq 100$, 5 above), same fits as Section 6; bold: lowest mean and every row not distinguishable from it by a two-sided paired t-test at $p < 0.05$; †: the Wilcoxon test disagrees.

	$n = 50$	$n = 100$	$n = 500$	$n = 1{,}000$	$n = 5{,}000$	$n = 10{,}000$
Sampled BNN						
BNN, $\mathcal{N}(0, 1)$	0.067 ± 0.014	0.050 ± 0.009	0.036 ± 0.003	0.029 ± 0.002	$\mathbf{0.004 \pm 0.001}$	0.004 ± 0.000
BNN, data-free scale	$\mathbf{0.021 \pm 0.006}$	0.024 ± 0.006	0.021 ± 0.002	0.022 ± 0.002	0.004 ± 0.000	$\mathbf{0.003 \pm 0.000}$
BNN, tempered	0.061 ± 0.014	0.042 ± 0.009	0.018 ± 0.006	0.015 ± 0.004	–	–
BNN without sampling						
Gaussian predictor, 12 nets, capped	$\mathbf{0.023 \pm 0.013}$	0.017 ± 0.009	$\mathbf{0.016 \pm 0.005}$	0.009 ± 0.003	0.007 ± 0.002	0.007 ± 0.001
Gaussian predictor, 12 nets, converged	$\mathbf{0.023 \pm 0.013}$	0.017 ± 0.009	0.016 ± 0.004	0.009 ± 0.002	$\mathbf{0.003 \pm 0.001}$	$\mathbf{0.003 \pm 0.000}$
Gaussian predictor, 1 net, capped	$\mathbf{0.025 \pm 0.014}$	0.018 ± 0.011	0.024 ± 0.004	0.014 ± 0.003	0.009 ± 0.005	$\mathbf{0.005 \pm 0.002}$
Gaussian predictor, 1 net, converged	$\mathbf{0.032 \pm 0.020}$	0.018 ± 0.011	0.025 ± 0.003	0.017 ± 0.003	0.006 ± 0.001	$\mathbf{0.005 \pm 0.002}$
NPE						
NPE, 12 flows	$\mathbf{0.023 \pm 0.010}$	$\mathbf{0.011 \pm 0.006}$	$\mathbf{0.007 \pm 0.003}$	$\mathbf{0.005 \pm 0.002}$	0.008 ± 0.001	0.007 ± 0.001
NPE, 1 flow	0.034 ± 0.018	0.028 ± 0.008	$\mathbf{0.015 \pm 0.007}$	$\mathbf{0.010 \pm 0.004}$	$\mathbf{0.004 \pm 0.001}$	$\mathbf{0.003 \pm 0.001}$
Appendix (retune sampler)						
BNN, $\mathcal{N}(0, 1)$ + retune	0.061 ± 0.016	0.047 ± 0.010	0.027 ± 0.003	0.019 ± 0.003	0.004 ± 0.000	0.004 ± 0.000
BNN, empirical Bayes	0.044 ± 0.016	0.023 ± 0.005	0.015 ± 0.002	0.011 ± 0.002	0.004 ± 0.000	$\mathbf{0.003 \pm 0.000}$

Table 15: TARP calibration error, mean over credibility levels of $|\mathrm{ECP}(\alpha) - \alpha|$; 0 = calibrated, exact posterior 0.005. Mean ± sd over repeats (20 at $n \leq 100$, 5 above), same fits as Section 6; bold: lowest mean and every row not distinguishable from it by a two-sided paired t-test at $p < 0.05$; †: the Wilcoxon test disagrees.

B Reproducibility

Code. The code of this report is at https://github.com/sam-dowd/rbf-sbi-bnn; all paths below are relative to it. It holds the library (`src/rbf_sbi/`: simulator and exact posterior, the BNN, NPE, evaluation and diagnostics), the scripts that build the data, tune, fit and post-process (`scripts/data/`, `scripts/tuning/`, `scripts/fit/`, `scripts/analysis/`), the scripts that regenerate every table and figure (`paper/scripts/`, run in order by `paper/make_paper.sh`), Slurm templates for NERSC Perlmutter (`slurm/`), the canonical dataset, and the small result files the tables need (selected hyperparameters, the NLL decomposition of Appendix A.2 and the convergence diagnostics of Appendix H.2). The result files of the final runs (9.2 GB) are to be deposited separately. Each script's docstring gives its arguments.

Environment. Python 3.11: `conda create -n rbf-sbi python=3.11 pip`, `pip install -r requirements.txt`, which pins the versions used: `sklearn-contrib-bde` 1.0.2, `blackjax` 1.2.2 (pinned by `bde`), `jax`/`jaxlib` 0.10.2, `flax` 0.12.8, `optax` 0.2.8, `numpy` 2.4.6, `scipy` 1.17.1, `torch` 2.13.0, `nflows` 0.14, `scikit-optimize` 0.10.2. Hardware: Section 4.7. Data, result and scratch folders can be redirected with the environment variables `RBF_SBI_DATA`, `RBF_SBI_RESULTS` and `RBF_SBI_SCRATCH`.

Code version and reproduction. Runs do not record a commit. The final runs were made between 2026-10-30 and 2026-11-05 with the project's research code; the repository is a curated copy of it whose model code differs only in imports and paths and in the removal of a baseline that is not part of this report. Every table of this report regenerates byte for byte from the repository's scripts and result files. GPU runs are not bit-reproducible: a re-fit of the $\mathcal{N}(0,1)$ BNN at $n = 100$, repeat 0, with the repository's code reproduces its pretraining exactly up to floating-point differences (the same stopping step for every network; test NLL of the pretrained networks -0.669 in both runs), but its chains differ, and the sampled BNN's test NLL is -0.753 against -0.828 in the reported run.

Data. `python scripts/data/build_dataset.py` writes `data/dataset.npz` (seed 0; 10,000-row pool, 1,000 test rows; also included in the repository). Training subsamples and fit seeds: Section 3.3. `python scripts/data/build_reference.py` computes and caches the exact test-set posteriors in `data/reference_posterior/test_n100_ntest1000_seed0.npz`.

Runs. The BNNs, the capped Gaussian predictor and the default-budget NPE come from `python scripts/fit/run_experiment.py <key> <n> rbf_kernel <repeats>`, submitted by `slurm/submit_final_sweep.sh` (template `slurm/final_sweep.sbatch`), which writes `results/raw/final/<key>_n<n>_r<r>.npz`; the BNNs also write their per-draw outputs to `sbi_final_draws/` in the scratch folder. The converged Gaussian predictor, the long-trained NPE and the NPE variants are refits with their own scripts (Table 16), which read and assert the seeds, subsamples and hyperparameters of the corresponding default run.

Hyperparameters. BNN and Gaussian predictor: `scripts/tuning/optimize_pretrain_grid.py` → `results/hp_search/pretrain_grid/pretrain_grid_n<n>.npz`. NPE: `scripts/tuning/optimize_npe.py` → `results/npe_params.npz`. Temperatures: `scripts/tuning/optimize_temperature_grid.py` (with `PRETUNED_HP=1`) → `results/temperature_search_n<n>_stdprior_pretuned_grid.npz`. Data-free scale: `scripts/tuning/optimize_prior_width.py` → `results/gaussian_width_prior_phi.npz`; its simulation-free check `scripts/tuning/coherence_width_check.py`. All of these result files are included.

model	result key	fitting call
BNN, $\mathcal{N}(0,1)$	`final_bnn_stdnormal_pretuned`	`bnn.fit_correlated`; also writes the capped Gaussian predictor
BNN, data-free scale	`final_bnn_gaussprior_pretuned`	`fit_correlated`, prior $\mathcal{N}(0, 0.3655^2)$
BNN, tempered	`final_bnn_temp_pretuned`	`fit_correlated(temperature=T)`, T from the search of Appendix C
$\mathcal{N}(0,1)$ + re-tune	`final_bnn_stdnormal_retune_pretuned`	`bnn.fit_rasmussen_gibbs_hierarchical( resample_tau=False)`
empirical Bayes	`final_bnn_ebblock_pretuned`	`fit_rasmussen_gibbs_hierarchical( resample_tau=False, eb_block="iso")`
Gaussian predictor, capped	`final_bnn_pretrained{_ens12,}_pretuned`	the $\mathcal{N}(0,1)$ BNN's pretrained networks, scored unsampled
Gaussian predictor, converged	`final_bnn_pretrained{_ens12,}_pretuned_long`	`scripts/fit/gaussian_predictor_long.py` (`bnn.fit_pretrained_ensemble`)
NPE, 12 flows / 1 flow	`final_npe_ens12` / `final_npe`	`npe.fit`, 12 member seeds
NPE, long-trained	`final_npe_ens12_long`	`scripts/fit/npe_long_members.py`, merged by `scripts/fit/npe_long_merge.py`
NPE variants	`final_npe{_ctx16,_gauss_ctx16}{_ens12,}`	`scripts/fit/npe_variant_members.py`, merged by `scripts/fit/npe_variant_merge.py`

Table 16: One row per reported model of the final runs; functions are in `src/rbf_sbi/`. Hyperparameters: Section 4.6 and Appendix C. The earlier runs of Appendix G use the sweep keys listed in `scripts/fit/run_experiment.py`'s module docstring; their result files are not released, and the code behind their tables is in `legacy/`.

Tables and figures, each from a script that reads the result files and fits nothing (CPU, minutes; all run by `bash paper/make_paper.sh`):

- Section 3: `paper/scripts/rbf_figures.py`; grid check `scripts/data/refine_reference_posterior.py`.
- Section 6 (Tables 2 and 3, Figures 5 and 6), Tables 5, 8, 9 and 12, and Figure 7: `paper/scripts/section6.py`, after `scripts/analysis/lppd_decomposition.py` has written `results/lppd_decomposition.npz` from the per-draw outputs.
- Tables 6, 7, 10, 11 and 13–15: `paper/scripts/appendix_tables.py`.
- Figure 8: `paper/scripts/temperature_search_plot.py`.
- Table 21: `scripts/analysis/chainwise_diagnostics.py` on the per-draw outputs (one shard per model, `slurm/chainwise_shards.sbatch`, merged by `scripts/analysis/merge_chainwise_shards.py`) → `results/rbf_chainwise_diagnostics.npz`; then `paper/scripts/diagnostics_table.py`.
- Appendix G (earlier runs): `legacy/results4_{scale,shape,side}.py`.

C Hyperparameters and compute

Hyperparameter search (summary in Section 4.6). Apart from the learning rate and weight decay, all settings are fixed as in Sections 4.2–4.5. The test set is never used.

- **BNN and Gaussian predictor** (`scripts/tuning/optimize_pretrain_grid.py`): one network of the BNN's architecture, pretrained exactly as in Section 4.5 and not sampled, on the training sets of repeats 0–2, scored by its NLL on the 1,000 pool rows that follow the training rows (at $n = 10{,}000$, where the pool has no rows left, on an inner 80/20 split of the training rows). Grid of 17 learning rates (10^{-5} to 10^{-1}, quarter decades) by 15 weight decays (10^{-7} to 1, half decades); the argmin of the mean over the three training sets is used at every size. One pair per size serves every BNN variant and both versions of the Gaussian predictor.
- **NPE** (`scripts/tuning/optimize_npe.py`): Bayesian optimization (Section 4.6) of the early-stopping validation NLL on an 80/20 split (seed 123) of the repeat-0 training rows; 200 s per size, 3–20 trials (3, 5, 9, 14, 20 and 19 at $n = 10{,}000$, 5,000, 1,000, 500, 100 and 50).

Rationale. Only learning rate and weight decay are searched, with the architecture fixed and identical across methods, so that a difference between methods cannot come from one of them receiving a larger or better-tuned network. MILE's sampler settings are not searched: in the MILE ablations (two UCI regression datasets, bikesharing and protein) predictive performance changes little with the energy-variance schedule or the two parameters of the step-size adaptation, and only marginally with a much longer warmup, which its authors judge not worth the linear cost; they call the method "effectively tuning-free" [12]. In MILE the two hyperparameters act only on the pretraining that initializes the chains and on the initial step size, so the BNN's network is scored as a standalone predictor, on a grid so that the whole landscape is visible. The grid supersedes an earlier search that scored complete 3-member MILE fits by held-out NLL (`legacy/optimize_bde.py`; 12–15 trials per size at $n \leq 1{,}000$, and 29–30 with a single-split objective at $n \geq 5{,}000$), whose values the earlier runs of Appendix G use. Moving from those values to the grid's left the sampled BNNs essentially unchanged (paired test-NLL differences of $+0.001$ to $+0.021$ nats for $\mathcal{N}(0, 1)$ and the data-free scale at every n, each smaller than its sd) and improved the Gaussian predictor where the earlier values had under-trained it. NPE keeps its own search; the baselines were not re-tuned with the BNN's objective. The reasons for the following were not recorded: (a) NPE's search ranges; (b) its random state and 10 initial points (the `scikit-optimize` default); (c) its 200 s budget; (d) the choice of 3 training draws and 1,000 held-out rows for the grid, taken over from the earlier BNN search.

Differences between the searches. (i) NPE's search was in effect a random search of 3–10 points. `scikit-optimize` draws 10 random points before its Gaussian-process phase. At $n \geq 1{,}000$ the 200 s budget ended within them (3, 5 and 9 trials); at $n \leq 500$ the 4–10 guided trials that followed did not improve on them, so every selected pair is one of the random initial points (checked by regenerating them from the random state). (ii) The objectives differ: the BNN's network is scored on 1,000 held-out points over three training draws, NPE on 20% of one draw (10 points at $n = 50$).

Shared by all BNNs. $100 \to 16 \to 16 \to 16 \to 5$, soft-sign, $d = 2{,}245$. Pretraining: AdamW at the per-size learning rate and weight decay of Table 1, full batch, 80/20 split, validation NLL every 100 steps, patience 5 evaluations ($\Delta > 10^{-4}$), at least 500 and at most 2,000 steps, best checkpoint kept. MILE: 50,000 warmup steps (phases I–III 40,000/5,000/5,000), EEVPD target $0.5 \to 0.1$, ESS target 100, initial step size equal to the learning rate; 10,000 sampling steps thinned by 10 to

1,000 kept draws per member; $K = 12$ members; under stock MILE, at most three attempts in all (the original seed and two re-seeds) to replace non-finite members. Gaussian predictor and NPE: Sections 4.2 and 4.3.

model	prior	T	sampling loop	steps / kept per member	wall time per fit
$\mathcal{N}(0,1)$	$\mathcal{N}(0,1)$	1	stock MILE	10,000 / 1,000	37–71 s
tempered	$\mathcal{N}(0,1)$	0.7 / 0.7 / 0.6 / 0.7	stock MILE, tempered warmup	10,000 / 1,000	36–55 s
data-free scale	$\mathcal{N}(0, 0.3655^2)$	1	stock MILE	10,000 / 1,000	36–72 s
$\mathcal{N}(0,1)$ + re-tune	$\mathcal{N}(0,1)$	1	re-tune every 1,000 steps	12,500 / 1,000	422–534 s
empirical Bayes	$\mathcal{N}(0,\tau^2)$, block update	1	re-tune loop	12,500 / 1,000	411–527 s
Gaussian predictor, converged	–	–	AdamW, $\leq$ 200,000 steps	–	24–82 s
NPE, 12 flows	–	–	Adam, $\leq$ 20,000 steps	–	157–238 s ($n \leq$ 1,000); 663–907 s ($n \geq$ 5,000)

Table 17: Per-model settings and wall time per fit (one training set, all 12 members, fitting and prediction; median per size, range over sizes) of the final runs, on one NVIDIA A100 (Section 4.7). BNN and Gaussian-predictor times are recorded in each result file; NPE times are job wall times from the run log (not included in the repository) divided by the repeats per job. Re-tune: 200 step-size and 50 L steps every 100 kept draws, 2,500 extra steps per member. Empirical-Bayes update: Section 4.4.3.

Temperature search. The temperatures of the tempered BNN ($T = 0.7$, 0.7, 0.6, 0.7 at $n = 50$–1,000; Section 4.4.2) are chosen as follows. The MCLMC target is $\log p(\mathbf{w}) + \frac{1}{T} \log p(\mathcal{D} \mid \mathbf{w})$ under the $\mathcal{N}(0,1)$ prior, with the pretraining-tuned learning rate and weight decay. T is searched per size on the grid $0.2, 0.3, \ldots, 1.0$ by bisection on the sign of the local slope, always including $T = 1$, so 5 of the 9 grid points are evaluated. Each evaluated T is a 3-member fit on the training sets of repeats 0–2, scored by the NLL on the 1,000 pool rows that follow each repeat's training rows; the chosen T minimizes the mean over the three repeats, over every evaluated point, with no filter on mixing.

The gain over $T = 1$ at the chosen T is 0.08, 0.20, 0.21 and 0.11 nats on validation at $n = 50$, 100, 500 and 1,000; in the final 12-member runs on the test set it is 0.06, 0.10, 0.25 and 0.15 nats. At $n \leq 100$ the validation NLL is flat to within the repeat spread between $T = 0.6$ and 0.8, and almost every evaluated T mixes (one repeat at $n = 100$, $T = 0.9$, has a maximum split-$\widehat{R}$ of 1.53). At $n = 500$ and 1,000 the minimum lies next to a mixing failure: at $n = 500$, $T = 0.5$ has $\widehat{R}$ 2.9–10.3 and $T = 0.4$ diverges (validation NLL +598, $\widehat{R} \approx 100$); at $n = 1{,}000$, $T = 0.6$ has $\widehat{R}$ 1.9–6.8 and a higher mean NLL than $T = 0.7$ (-1.67 against -1.91): its two worst-mixed repeats ($\widehat{R}$ 5.2, 6.8) score -1.52 and -1.53, while the third ($\widehat{R}$ 1.85) scores -1.95, the lowest single value at that size. The chosen T is therefore the coldest temperature at which the chains still mix, not an interior optimum of a smooth curve, at the two sizes where tempering helps most (Figure 8).

Reproduce: search `scripts/tuning/optimize_temperature_grid.py` → `results/temperature_search_n<n>_stdprior_pretuned_grid.npz` (n = 50, 100, 500, 1,000); figure `python`

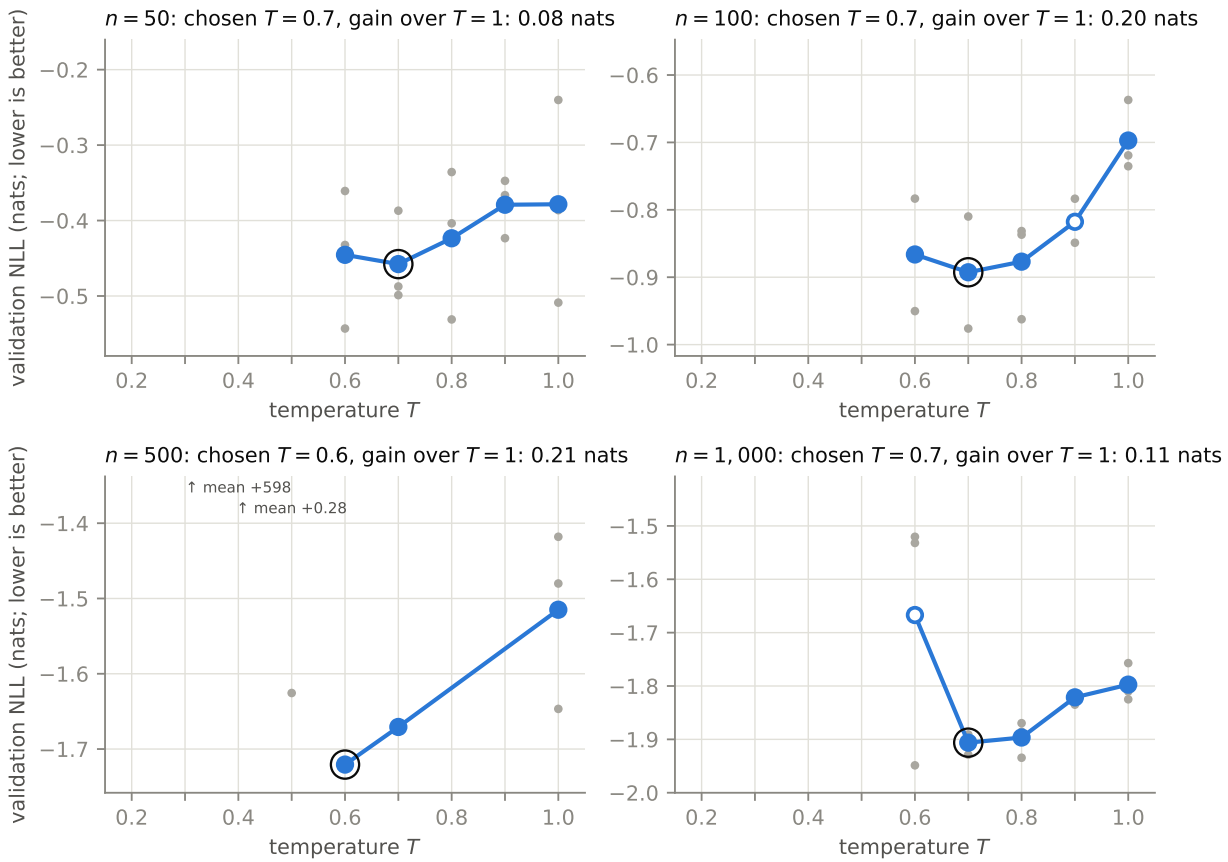


Figure 8: Validation NLL against temperature from the search behind the tempered BNN. Line and filled markers: mean over repeats 0–2 (3-member fits, $\mathcal{N}(0, 1)$ prior, pretraining-tuned lr/wd); grey dots: each repeat; ring: the chosen T. Open markers: max split-$\widehat{R}$ on the predicted mean > 1.5 in at least one repeat (poorly mixed; at $n = 100$, $T = 0.9$ is flagged by one repeat at 1.53). Means above the panel are written at its top edge; repeats above it are not drawn (all three at $n = 500$, $T = 0.4$; two of three at $n = 500$, $T = 0.5$); every other value is in range. Only the evaluated temperatures are shown (bisection, 5 of 9 grid points per size). Each panel has its own vertical scale. Each panel's title gives the chosen T and its mean gain over $T = 1$ on validation.

`paper/scripts/temperature_search_plot.py`.

D Implementation of the correlated `bde` model

Departures from stock `bde`, all subclass overrides in `src/rbf_sbi/bnn.py` (installed package unmodified):

- **Correlated two-target head.** `bde` is single-output: its output width is fixed at 2 and its sampling likelihood is a univariate Gaussian whatever loss is passed. A joint (σ, ρ) likelihood with a fitted correlation needs four overrides (output width, log-posterior, pretraining loss, and `fit` itself to bypass a check on the loss class) plus an sklearn-tag override; details below.
- **Pretraining schedule.** `bde` checks the validation loss at every step, so its patience counts steps; at $n = 100$ pretraining stopped almost immediately. We evaluate every 100 steps, stop after 5 evaluations without improvement ($> 10^{-4}$), never before step 500, and cap at 2,000 steps; full-batch, 80/20 split of the training set. AdamW replaces `bde`'s Adam, which ignored weight decay. Learning rate and weight decay are tuned per size (Section 4.6).
- **Member retry** (stock MILE). Members whose chains become non-finite are discarded; the whole ensemble is refitted with a new seed on the same training rows, and finite members are pooled across attempts until K exist, for at most three attempts in all; the failure rate is recorded. The re-tuning loop has no retry: a non-finite member is dropped (Section 5.2).
- **Tempering** (Section 4.4.2): $\log p(\mathbf{w}) + \log p(\mathcal{D} \mid \mathbf{w})/T$ in warmup and sampling; pretraining is untouched.
- **Periodic re-tuning** (optional; Appendix H.3): after every 100 kept draws (1,000 steps) the chain re-estimates ϵ (200 steps) and L (50 steps) at the final EEVPD target 0.1, starting from the current values and carrying the step-size adaptation's running averages across re-tunes; 2,500 extra steps per chain. This re-implements `bde`'s phase I/II loop, which otherwise resets its accumulators on every call.

The two-target head. `bde`'s `BdeRegressor` is single-output: its sklearn tags declare `single_output=True`, its network has two outputs (mean and scale), and its sampling likelihood slices a univariate Gaussian from them whatever loss is passed. `CorrelatedBdeRegressor` (`src/rbf_sbi/bnn.py`) subclasses it; the installed package is not modified.

1. **Output width.** `_CorrelatedBdeBuilder._determine_output_dim` returns 5 $(\mu_\sigma, \mu_\rho, a_\sigma, a_\rho, c)$ instead of 2.
2. **Log-posterior.** `_build_log_post` returns `_BivariateProbabilisticModel`, whose `log_likelihood` is the bivariate normal of Eq. (9) summed over the training set (`softplus`$+10^{-6}$ scales, tanh correlation) and whose `log_unnormalized_posterior` is $\log p(\mathbf{w}) + \log p(\mathcal{D} \mid \mathbf{w})/T$.
3. **Pretraining loss.** `BivariateGaussianNLL`, the negative of the same log density, summed (not averaged) over the batch so that pretraining and sampling share one scale.
4. **`fit`.** `Bde._fit` calls `task.validate_loss`, which accepts only the loss classes `GaussianNLL` and `Rmse` for regression, by class identity. `CorrelatedBdeRegressor.fit` is a copy of `Bde._fit` without that check, plus the pretraining schedule of Section 4.5 (evaluation every 100 steps, minimum 500 steps via `_MinStepsEarlyStoppingCallback`, `optax.adamw`).

5. **sklearn tags.** `__sklearn_tags__` declares `multi_output=True`, `single_output=False`, `two_d_labels=True`; otherwise `bde.data.utils.validate_fit_data` treats a two-column target as a column vector and keeps only the first column, without an error.

The method-resolution-order (MRO) pitfall. Overriding `_fit` instead of `fit` does nothing: `BdeRegressor.fit` calls `super()._fit`, and that `super()` is bound to `BdeRegressor`, so it resolves to `Bde._fit` whatever a subclass defines. The first implementation did this and failed with "`BivariateGaussianNLL` not allowed for task regression".

Read-out. Scales are read out through the same softplus$(\cdot) + 10^{-6}$ map as in the likelihood, then multiplied by the training-set target standard deviation; means are destandardized. The correlation needs no transformation: $\mathrm{Corr}(aX + b,\, cY + d) = \mathrm{Corr}(X, Y)$ for $a, c > 0$.

Later additions (same file): `_draw_samples` (a kept-sample count independent of the thinning factor), member-by-member prediction (`_predict_raw_per_member`, identical output at a third of the GPU memory), and the re-tune loop `fit_rasmussen_gibbs_hierarchical`, which runs its own MCLMC chain on the same log-posterior. All rely on undocumented `bde` internals and are tied to version 1.0.2.

E Background details

E.1 Microcanonical Langevin Monte Carlo

MCLMC [13] samples a target $p(\mathbf{w} \mid \mathcal{D})$ on $\mathbb{R}^d$ with a unit-speed velocity $\mathbf{u}$ ($\|\mathbf{u}\| = 1$) and the dynamics, in the form used by Sommer et al. [12],

$$\mathrm{d}\mathbf{w} = \mathbf{u}\,\mathrm{d}t, \qquad \mathrm{d}\mathbf{u} = \big(I - \mathbf{u}\mathbf{u}^\top\big)\Big(\tfrac{1}{d-1}\nabla \log p(\mathbf{w} \mid \mathcal{D})\,\mathrm{d}t + \xi\,\mathrm{d}\mathbf{W}\Big). \tag{14}$$

The stationary distribution of $\mathbf{w}$ is the target for any noise level ξ, including the deterministic dynamics $\xi = 0$ [13]; the noise only decorrelates the velocity, over a length L. In practice the deterministic part is integrated with the minimal-norm integrator (two gradient evaluations per step of size ϵ), and the noise is applied as a partial refresh of the velocity [13],

$$\mathbf{u} \leftarrow \frac{\mathbf{u} + \omega\mathbf{z}}{\|\mathbf{u} + \omega\mathbf{z}\|}, \qquad \mathbf{z} \sim \mathcal{N}(0, I), \qquad \omega = \sqrt{\big(e^{2\epsilon/L} - 1\big)/d}, \tag{15}$$

with ω as implemented in BlackJAX [50]. There is no Metropolis–Hastings correction, so a finite ϵ biases the stationary distribution. The bias is controlled by tuning ϵ to a target energy-error variance per dimension (EEVPD), $\mathrm{Var}[\Delta E]/d$ [51]; the BlackJAX update uses $\mathrm{Var}[\Delta E] \propto \epsilon^6$. Tuning proceeds in three phases [12]: (I) adapt ϵ to the target EEVPD while burning in; (II) set $L = \big(\sum_i \mathrm{Var}[w_i]\big)^{1/2}$ from the chain's own variance; (III) refine $L = c\,\epsilon\,N/\widehat{\mathrm{ESS}}$ from the effective sample size of N further steps ($c = 0.4$ in the implementation used here).

E.2 Tempering

Wenzel et al. [22, App. E] also show that the KL-weighted evidence lower bound used in variational BNNs, $\lambda\,\mathrm{KL}\big(q(\mathbf{w}) \,\|\, p(\mathbf{w})\big) - \sum_i \mathbb{E}_q \log p(y_i \mid x_i, \mathbf{w})$, has the same minimizer as $\mathrm{KL}(q \,\|\, p_P)$ with $T = \lambda$: down-weighting the KL term ($\lambda < 1$) is variational inference for the partially tempered posterior. The temperature-weighted objective of Delaunoy et al. [8] is of this kind. This project uses partial tempering, Eq. (4).

Tempering versus changing the prior. The prior factor of full tempering is itself a prior: for a Gaussian prior, $\mathcal{N}(\mathbf{w}; 0, s^2 I)^{1/T} \propto \mathcal{N}(\mathbf{w}; 0, Ts^2 I)$ [22, Sec. 2.2], so full tempering at T with prior $\mathcal{N}(0, s^2)$ is exactly partial tempering at T with prior $\mathcal{N}(0, Ts^2)$; the two schemes differ only by a rescaling of the prior. The likelihood factor $p(\mathcal{D} \mid \mathbf{w})^{1/T}$, which both contain and which is all of partial tempering, cannot be obtained by changing the prior. Writing the partially tempered posterior as a $T = 1$ posterior, $p(\mathcal{D} \mid \mathbf{w})\, \tilde{p}(\mathbf{w})$, requires $\tilde{p}(\mathbf{w}) \propto p(\mathbf{w})\, p(\mathcal{D} \mid \mathbf{w})^{1/T-1}$, which depends on the training data and is therefore not a prior. Any prior, of any scale or shape, can at most approximate the effect of likelihood tempering, never reproduce it; Eq. (5) shows the effect is a change of the likelihood instead.

Where the temperature acts. The temperature acts only in the MCLMC target, during warmup and sampling. Pretraining (the deep-ensemble optimization that initializes the chains) uses the untempered likelihood. Prediction and evaluation are untempered: each posterior draw $\mathbf{w}_{ks}$ predicts with its own Gaussian $\mathcal{N}_2(\theta; \boldsymbol{\mu}_{ks}(S), \Sigma_{ks}(S))$, and the predictive is the mixture of Section 5.1, exactly as at $T = 1$. Tempering changes which weights are drawn, not how a drawn network predicts, so the test NLL of a tempered BNN is a density of the same form as that of the untempered BNN and of NPE, and the three are directly comparable.

E.3 Empirical Bayes and data-free calibration of the prior scale

No prior reproduces likelihood tempering (Section 2.4). The alternative considered here keeps $T = 1$ and sets the scale of an isotropic Gaussian prior, either from the training data or from the simulator alone.

Empirical Bayes (type-II maximum likelihood). Make the prior scale a hyperparameter, $p(\mathbf{w} \mid \tau) = \mathcal{N}(\mathbf{w}; 0, \tau^2 I_d)$, and choose it to maximize the evidence

$$p(\mathcal{D} \mid \tau) = \int p(\mathcal{D} \mid \mathbf{w})\, p(\mathbf{w} \mid \tau)\, \mathrm{d}\mathbf{w}, \qquad \hat{\tau} = \arg\max_{\tau} p(\mathcal{D} \mid \tau), \tag{16}$$

then use the posterior $p(\mathbf{w} \mid \mathcal{D}, \hat{\tau})$ [20]. Since $\partial_\tau \log p(\mathcal{D} \mid \tau) = \mathbb{E}_{p(\mathbf{w}\mid\mathcal{D},\tau)}\big[\partial_\tau \log p(\mathbf{w} \mid \tau)\big]$ and $\partial_\tau \log p(\mathbf{w} \mid \tau) = -d/\tau + \|\mathbf{w}\|^2/\tau^3$, the evidence is stationary where

$$\hat{\tau}^2 = \frac{\mathbb{E}_{p(\mathbf{w}\mid\mathcal{D},\hat{\tau})}\|\mathbf{w}\|^2}{d} = \frac{\|\mathbf{m}\|^2}{\gamma}, \qquad \gamma = d - \frac{\operatorname{tr}\operatorname{Cov}[\mathbf{w} \mid \mathcal{D}, \hat{\tau}]}{\hat{\tau}^2}, \tag{17}$$

with $\mathbf{m}$ the posterior mean. The second form is MacKay's evidence condition $2\alpha E_W^{\mathrm{MP}} = k - \alpha \operatorname{Tr} A^{-1}$ with $\alpha = 1/\tau^2$ [25, Eqs. 2.22–2.23], where $\gamma \in [0, d]$ is "the number of good parameter measurements", the effective number of weights determined by the data. MacKay evaluates it at the posterior mode with the Gaussian (Laplace) covariance A^{-1}, which is exact only for quadratic energies; with posterior samples, as here, Eq. (17) can be used with sampled moments and iterated as a fixed point. Two caveats: the data are used twice, once to choose τ and once in the posterior; and if γ is small (little data relative to d), the fixed point is poorly determined.

Data-free calibration. Alternatively, the prior can be chosen before any training data are seen, so that the network's prior predictive is consistent with the simulator. Delaunoy et al. [8] optimize a per-weight prior so that the BNN's prior predictive matches a calibrated target; the simplest version, used here, chooses the single scale τ for which the prior predictive's coverage of the simulator's parameters is calibrated (Section 4.4.1). Both routes fit the prior predictive to the simulator's joint distribution of (θ, S), one through the evidence of the training sample and one through coverage without data. In a well-specified model they should therefore select the same scale at every n, and

a scale that drifts with n indicates that no single prior of this family reproduces the simulator. Agreement at some n therefore shows that the scale is right for this family, not that the family is right: an isotropic Gaussian with the correct scale can still be the wrong form of prior. This is our expectation rather than a published result; Appendices A.7 and G.1 test it.

F Reference posterior and NLL details

Grid adequacy. On the global grid two test datasets (true $\rho = 0.049$, the smallest in the test set, and 0.062) put 1.5% and 4.7% of their mass in the lowest ρ cell, and 40 have a ρ posterior narrower than two cells. Re-evaluating every test posterior on a per-dataset 300×300 grid over ± 10 posterior standard deviations (float64, ρ down to 10^{-3}, exact density at θ^*) leaves the summaries unchanged: the ratio of fine to coarse posterior standard deviation has median 1.0000 for both parameters, the posterior correlation and calibration agree to three decimals, and the mean reference test NLL moves by 0.002 nats. Only the two boundary datasets change materially: 3.8% and 11% of their mass lies below $\rho = 0.02$, and their ρ standard deviation is 24% and 30% larger. The tail is genuine. With grid spacing 1/9 the correlation between neighbouring points, $\exp(-1/(162\rho^2))$, is below 10^{-3} for $\rho < 0.03$, so the likelihood is flat in ρ there and the posterior follows the prior towards $\rho = 0$. The global grid is used throughout; quantities that depend on it are insensitive to this choice (e.g. the per-dataset ratio of the BNN's width to its warranted width, Section A.5, is unchanged to three decimals).

Per-method densities. Each method's q is a normalized density on θ in original units, evaluated exactly at θ^*_j:

- *Gaussian predictor*: the equal-weight mixture of its 12 networks' Gaussians (one Gaussian for a single network), each from Eq. (9) after mapping the network's outputs back to original units as for the BNN below; for one Gaussian, $-\log q = \log(2\pi) + \log s_\sigma + \log s_\rho + \frac{1}{2}\log(1 - r^2) + \frac{1}{2(1-r^2)}\big(z_\sigma^2 + z_\rho^2 - 2r z_\sigma z_\rho\big)$ with $z_p = (\theta_p - \mu_p)/s_p$.
- *BNN*: the equal-weight mixture over all KS posterior draws,

$$q(\theta \mid S) = \frac{1}{KS} \sum_{k=1}^{K} \sum_{s=1}^{S} \mathcal{N}_2\big(\theta;\, \boldsymbol{\mu}_{ks}(S),\, \Sigma_{ks}(S)\big), \tag{18}$$

  each component from Eq. (9) after mapping the draw's outputs back to original units ($\boldsymbol{\mu} \mapsto \boldsymbol{\mu} \odot \mathbf{s}_y + \mathbf{m}_y$, $s_\bullet \mapsto s_\bullet s_{y,\bullet}$, r unchanged), so no Jacobian term arises. It is evaluated as logsumexp over components minus $\log(KS)$.
- *NPE*: by change of variables, $\log q_\phi(\theta \mid S) = \log \mathcal{N}\big(\Psi(\theta); 0, I_2\big) + \log|\det \partial\Psi/\partial\theta|$, where Ψ maps θ to the base variable through two masked affine autoregressive layers, each followed by an order reversal (Jacobian determinant 1). Each layer sets $y_i = a_i x_i + b_i$, with $a_i = \mathrm{softplus}(\cdot) + 10^{-3}$ and b_i functions of $x_{<i}$ and $g(S)$, contributing $\sum_i \log a_i$.
- *Ensembles* of the baselines: $\log \frac{1}{K} \sum_k q_k(\theta^* \mid S)$ over the members.
- *Exact posterior*: the grid density at θ^* (grid mass per cell divided by the cell area, bilinearly interpolated).

Because all five are densities on the same space in the same units, their NLLs are directly comparable.

G Further prior and sampler variants

All results in this appendix come from earlier 3-member runs: 5 repeats per size, the earlier BNN hyperparameters of Appendix C, NPE as a single flow, and temperatures 0.2 / 0.3 / 0.6 / 0.8 for the tempered BNN. They are not comparable number for number with Section 6, and are kept for the variants not run in the final runs and for two checks of the prior scale.

G.1 Prior scale: data-free width and empirical Bayes

The final results of both are in Section 6.2 and Appendix A.7; this appendix gives the methods and, from the earlier runs, two checks of the evidence's fixed point.

Both models replace $\mathcal{N}(0,1)$ by an isotropic $\mathcal{N}(0,\tau^2)$ (Section 2.4).

Data-free width. The scale $\tau = 0.3655$ is chosen before any training by Gaussian-process Bayesian optimization of |coverage area| of the prior predictive: weights drawn from $\mathcal{N}(0,\tau^2)$, the untrained network's predictive evaluated by TARP (Appendix A.6) on fresh simulator draws, with inputs standardized by a simulator-drawn measurement set and θ by the prior moments. The width minimizing |coverage area| gives +0.0037, against +0.200 for $\mathcal{N}(0,1)$. It is a continuous version of the fixed-width sweep of Delaunoy et al. [8, Fig. 1]. It is run on stock MILE, paired with $\mathcal{N}(0,1)$. At fit time the BNN standardizes with training-set statistics, so at small n the width acts in slightly different units from those it was calibrated in.

Empirical Bayes. One scale for all weights is updated between blocks inside the re-tune loop (Section 4.4.3). Each member starts at the RMS of its pretrained weights; the scale is fixed for the one-time warmup and within each 1,000-step block. After a block, the target $\tau_t^2 = \|\mathbf{m}\|^2/\gamma$ of Eq. (17) is computed from the block's per-weight mean $\mathbf{m}$ and variance, with γ clamped to $[1, d]$; $\log\tau$ moves half-way towards $\log\tau_t$, by at most a factor 2, with a floor of 10^{-3}, and (L, ϵ) are re-tuned. All kept draws are retained, including those of early blocks drawn under a scale still moving. It is paired with $\mathcal{N}(0,1)$ plus the same re-tune.

- **Empirical Bayes recovers the data-free width at $n = 100$–1,000.** The median final scale is 0.378, 0.350 and 0.324 (ranges 0.32–0.39, 0.34–0.36, 0.31–0.33), within 11% of 0.3655. Two independent checks place the evidence's fixed point within ∼10% of 0.3655 at these sizes: the zero-data one-point evidence peaks at 0.3655 in 5/5 repeats, and fixed-width chains satisfy the stationarity condition of Eq. (17) at 0.382, 0.373 and 0.400. The data-chosen and the data-free scales therefore agree, so the scale is well specified: for an isotropic Gaussian prior on this task the right scale is about 0.37, not MILE's default of 1 (Section 2.4). This does not make the isotropic Gaussian form well specified. The empirical-Bayes scale drifts at larger n (0.32 and 0.28 at $n = 5{,}000$ and 10,000 in the final runs, Appendix A.7), and at intermediate n, where the form rather than the scale dominates, tempering gains more than the correct scale does (Section 7.3).

G.2 Prior shape, and our claim on priors versus tempering

$\mathcal{N}(0,1)$ is replaced by a Student-t with $\nu = 4$ and scale $s = 1/\sqrt{2}$ on every weight, on stock MILE with the same chain budget, seeds, subsamples and hyperparameters. Its variance $s^2\nu/(\nu-2) = 1$ equals that of $\mathcal{N}(0,1)$, so the pair differs in tail shape only. It is the marginal of $w \mid \tau^2 \sim \mathcal{N}(0,\tau^2)$, $\tau^2 \sim \text{Inv-Gamma}(2,1)$, the weight prior of the Gibbs variants (Appendix G.4). Heavy tails are also

(a) data-free width vs. $\mathcal{N}(0,1)$ (stock MILE)

n	test NLL	paired difference	width / warranted σ/ρ	RMS aleatoric σ/ρ
50	$-0.369 \to \mathbf{-0.607}$	-0.238 ± 0.090 (5/5)	$2.20/2.34 \to \mathbf{1.08/1.12}$	$0.42/0.31 \to \mathbf{0.21/0.15}$
100	$\mathbf{-0.806} \to \mathbf{-0.833}$	-0.027 ± 0.103 (4/5)	$1.65/1.97 \to \mathbf{1.11/1.22}$	$0.28/0.21 \to \mathbf{0.21/0.14}$
500	$\mathbf{-1.439} \to \mathbf{-1.435}$	$+0.003 \pm 0.072$ (4/5)	$1.23/1.42 \to \mathbf{1.09/1.24}$	$0.19/\mathbf{0.10} \to \mathbf{0.18/0.10}$
1,000	$\mathbf{-1.734} \to -1.667$	$+0.067 \pm 0.042$ (0/5)	$1.15/1.33 \to \mathbf{1.09/1.26}$	$\mathbf{0.18/0.08} \to \mathbf{0.17}/0.09$
5,000	$\mathbf{-2.213} \to \mathbf{-2.192}$	$+0.022 \pm 0.027$ (1/5)	$\mathbf{1.00/1.04} \to \mathbf{1.00/1.04}$	$\mathbf{0.15/0.06} \to 0.15/0.06$
10,000	$\mathbf{-2.272} \to -2.262$	$+0.011 \pm 0.006$ (0/5)	$\mathbf{0.99/1.01} \to \mathbf{0.99/1.01}$	$0.15/\mathbf{0.05} \to \mathbf{0.15}/0.06$

(b) empirical Bayes vs. $\mathcal{N}(0,1)$ + re-tune

n	test NLL	paired difference	width / warranted σ/ρ	RMS aleatoric σ/ρ	empirical-Bayes scale median (at floor)
50	$\mathbf{-0.424} \to +0.354$	$+0.778 \pm 0.313$ (0/5)	$2.10/2.16 \to \mathbf{0.67/0.73}$	$0.40/0.28 \to \mathbf{0.20/0.14}$	0.001 (14/15)
100	$\mathbf{-0.801} \to \mathbf{-0.857}$	-0.055 ± 0.069 (4/5)	$1.64/1.79 \to \mathbf{1.07/1.18}$	$0.28/0.18 \to \mathbf{0.20/0.13}$	0.378 (0/15)
500	$\mathbf{-1.598} \to -1.547$	$+0.051 \pm 0.019$ (0/5)	$1.15/1.35 \to \mathbf{1.04/1.19}$	$0.18/\mathbf{0.09} \to \mathbf{0.17/0.09}$	0.350 (0/15)
1,000	$\mathbf{-1.850} \to -1.805$	$+0.045 \pm 0.021$ (0/5)	$1.09/1.25 \to \mathbf{1.02/1.15}$	$0.16/\mathbf{0.07} \to \mathbf{0.16}/0.08$	0.324 (0/15)
5,000	$\mathbf{-2.216} \to -2.195$	$+0.021 \pm 0.011$ (0/5)	$\mathbf{1.00/1.05} \to \mathbf{1.00/1.05}$	$\mathbf{0.15/0.06} \to \mathbf{0.15}/0.06$	0.425 (0/15)
10,000	$\mathbf{-2.271} \to -2.241$	$+0.030 \pm 0.006$ (0/5)	$\mathbf{0.99/1.04} \to \mathbf{1.00/1.03}$	$\mathbf{0.15/0.06} \to 0.15/0.06$	0.266 (0/15)

Table 18: Prior-scale models against their controls ($\to$: control, then model; means over 5 repeats). Paired difference: model minus control ($k/5$: model better). Bold: the better of each pair (both if within the sd of their paired difference); width / warranted closer to 1, RMS aleatoric closer to the exact sd (0.153 / 0.054). Empirical-Bayes scale: median final scale over 15 members (5 repeats $\times$ 3), with the number at the 10^{-3} floor (< 0.01).

what the pretrained weights show: by the Akaike information criterion (AIC) a Student-t fits the AdamW-pretrained weights better than a Laplace or a Gaussian at $n = 100$ and $n = 10{,}000$ (fitted $\nu = 3.4$ and 3.0).

n	test NLL $\mathcal{N}(0,1) \to$ Student-t	paired difference	width / warranted σ/ρ	RMS aleatoric σ/ρ	RMS epistemic σ/ρ	$\max \widehat{R}$ range
50	$-0.369 \to \mathbf{-0.466}$	-0.098 ± 0.035 (5/5)	$\mathbf{2.20/2.34} \to \mathbf{2.27/2.24}$	$\mathbf{0.42/0.31} \to \mathbf{0.44/0.29}$	$\mathbf{0.21/0.16} \to \mathbf{0.21/0.14}$	1.01–1.06 $\to$ 1.01–1.03
100	$\mathbf{-0.806} \to \mathbf{-0.773}$	$+0.034 \pm 0.111$ (1/5)	$\mathbf{1.65/1.97} \to \mathbf{1.56/1.79}$	$\mathbf{0.28/0.21} \to \mathbf{0.27/0.20}$	$\mathbf{0.16/0.12} \to \mathbf{0.14/0.11}$	1.04–1.46 $\to$ 1.02–1.22
500	$\mathbf{-1.439} \to \mathbf{-1.413}$	$+0.026 \pm 0.088$ (1/5)	$\mathbf{1.23/1.42} \to \mathbf{1.25}/1.49$	$\mathbf{0.19/0.10} \to \mathbf{0.20/0.11}$	$\mathbf{0.09/0.06} \to \mathbf{0.09/0.06}$	1.02–1.11 $\to$ 1.05–1.13
1,000	$-1.734 \to \mathbf{-1.802}$	-0.067 ± 0.054 (5/5)	$\mathbf{1.15/1.33} \to \mathbf{1.13/1.31}$	$\mathbf{0.18/0.08} \to \mathbf{0.17/0.08}$	$\mathbf{0.08/0.05} \to \mathbf{0.07}/0.04$	1.06–1.13 $\to$ 1.19–1.65
5,000	$\mathbf{-2.213} \to \mathbf{-2.218}$	-0.004 ± 0.024 (2/5)	$\mathbf{1.00/1.04} \to \mathbf{1.00/1.04}$	$\mathbf{0.15/0.06} \to \mathbf{0.15}/0.06$	$\mathbf{0.03/0.02} \to \mathbf{0.03}/0.02$	3.14–4.05 $\to$ 2.28–2.49
10,000	$\mathbf{-2.272} \to \mathbf{-2.275}$	-0.003 ± 0.013 (2/5)	$\mathbf{0.99/1.01} \to \mathbf{0.99/1.01}$	$0.15/\mathbf{0.05} \to \mathbf{0.15/0.05}$	$\mathbf{0.02/0.01} \to \mathbf{0.02/0.01}$	7.53–20.94 $\to$ 5.79–12.76

Table 19: Student-$t(4, 1/\sqrt{2})$ against $\mathcal{N}(0,1)$, both on stock MILE ($\to$: $\mathcal{N}(0,1)$ value, then Student-t; means over 5 repeats). Paired difference: Student-t minus $\mathcal{N}(0,1)$ ($k/5$: Student-t better). Bold: paired difference with |mean| > sd and the same sign on 5 of 5 repeats.

- **NLL.** Student-t is better at $n = 50$ (-0.098 ± 0.035, 5/5) and $n = 1{,}000$ (-0.067 ± 0.054, 5/5); there is no difference at $n = 100$ and 500 ($+0.034 \pm 0.111$, $+0.026 \pm 0.088$; 1/5 each) or from $n = 5{,}000$ ($|\Delta| \le 0.004$).
- **Widths.** Width over warranted width and RMS aleatoric width change by less than their paired sd at every $n \le 1{,}000$, and by at most 0.005 above (at $n = 50$, 2.20 / 2.34 to 2.27 / 2.24). The RMS epistemic width falls by 4–10% at $n \le 100$, about one paired sd. The shape of the prior does not touch the excess width of Section A.5.
- **Where the gains sit.** With a Gaussian summary of each predictive (its first two moments and correlation): at $n = 50$ the summary NLL changes by -0.032 ± 0.072 (3/5) against

-0.098 ± 0.035 for the actual NLL, so the gain is in the shape of the predictive mixture; at $n = 1{,}000$ the summary carries all of it (-0.066 ± 0.064), i.e. the first two moments and the correlation.

- **Against the other remedies** (paired, all stock MILE). At $n = 50$ the data-free width is better than Student-t by 0.140 ± 0.062 (5/5); tempering and Student-t are indistinguishable (0.039 ± 0.278). Tempering is better at $n = 100$ (0.166 ± 0.154, 4/5) and $n = 500$ (0.275 ± 0.050, 5/5); at $n = 1{,}000$ the two are indistinguishable (Student-t minus tempered $+0.027 \pm 0.049$, 2/5).
- **Cost.** At $n = 1{,}000$, $\max \widehat{R}$ rises from 1.06–1.13 to 1.19–1.65.
- **Other shapes.** Single $n = 100$ runs on earlier hyperparameters (learning rate 10^{-3}, no weight decay; repeat 0): Laplace and Student-$t(\nu = 3)$ at scale 1, and Student-$t(\nu = 3)$ at scale 0.3655, are 0.02–0.08 worse in NLL than the Gaussian of the same scale, within the $n = 100$ repeat sd (0.11); the Laplace run lost 2 of 6 member attempts. All runs of this comparison, including the Gaussian references, used learning rate 10^{-3} and no weight decay (for the scale-1 Gaussian, which predates logging of the learning rate, the script's constant at the time). Optimized per-weight Gaussian priors [8] are in Appendix G.4.

Claim. On this task, changing the shape of the weight prior at fixed variance leaves the predictive widths unchanged at every n. It lowers the NLL at $n = 50$ and $n = 1{,}000$ only, by less than the data-free width at $n = 50$ and by as much as tempering at $n = 1{,}000$. The only prior change that reduces the excess width is a smaller scale, and it improves the NLL only at $n = 50$ (Appendix G.1). In fully connected classifiers on MNIST and FashionMNIST, heavy-tailed priors remove the cold posterior effect ("removed—or even inverted", with $T \approx 1$ "indeed ideal"), though not in convolutional or residual networks, and "we do not expect there to be one universal prior" [23]; those experiments are classification at full dataset size, with no heteroskedastic regression and no small-n study. Hypothesis, untested: the excess sits in the aleatoric output, which tempering scales directly in every likelihood term (Eq. (5)) and a weight prior reaches only through the network's function-space prior.

G.3 Calibration of all variants

The table covers all models of this section and NPE, paired by repeat. Empirical Bayes runs on the re-tune loop; its control, $\mathcal{N}(0, 1)$ with the re-tune, is listed as its own column, since the re-tune alone changes the NLL.

- **Best NLL by size** (earlier runs; not tabulated). $n = 50$: data-free width (-0.607; tempering not distinguishable). $n = 100$: tempering (-0.938; $\mathcal{N}(0, 1)$ not distinguishable). $n = 500$: tempering (-1.688). $n = 1{,}000$: $\mathcal{N}(0, 1)$ with the re-tune (-1.850). From $n = 5{,}000$: $\mathcal{N}(0, 1)$, Student-t and $\mathcal{N}(0, 1)$ + re-tune, indistinguishable.
- **The re-tune alone** lowers the NLL by 0.160 ± 0.046 at $n = 500$ and 0.115 ± 0.028 at $n = 1{,}000$ (5/5 each), comparable to tempering (0.249, 0.094), and by 0.055 ± 0.062 at $n = 50$ (4/5, within noise); no difference at $n = 100$ or from $n = 5{,}000$. It is a sampler change, not a change of target; Appendix H.3 treats it. Empirical Bayes is 0.02–0.05 behind this control at every $n \geq 500$, and on its own column ahead of stock-MILE $\mathcal{N}(0, 1)$ only because of it.

n	$\mathcal{N}(0,1)$	tempered	data-free width	Student-t	$\mathcal{N}(0,1)$ + re-tune	empirical Bayes	NPE
width / warranted (σ/ρ)							
50	2.20/2.34	1.69/2.11	**1.08**/**1.12**	2.27/2.24	2.10/2.16	0.67/**0.73**	**0.94**/1.22
100	1.65/1.97	**1.07**/**1.35**	**1.11**/**1.22**	1.56/1.79	1.64/1.79	**1.07**/**1.18**	**0.89**/**0.93**
500	1.23/1.42	1.09/1.24	1.09/1.24	1.25/1.49	1.15/1.35	**1.04**/1.19	**0.95**/**0.98**
1,000	1.15/1.33	1.10/1.26	1.09/1.26	1.13/1.31	1.09/1.25	**1.02**/1.15	0.95/**0.99**
5,000	**1.00**/1.04	–	**1.00**/1.04	**1.00**/1.04	**1.00**/1.05	**1.00**/1.05	0.98/**0.99**
10,000	0.99/**1.01**	–	0.99/**1.01**	0.99/**1.01**	**0.99**/1.04	**1.00**/1.03	**0.99**/**1.02**
z-spread (σ/ρ)							
50	0.53/0.48	0.67/0.54	**0.95**/**0.88**	0.51/0.48	0.54/0.52	1.50/**1.39**	**1.51**/**1.19**
100	0.64/0.52	**1.01**/**0.81**	**0.91**/0.81	0.66/0.57	0.66/0.59	**0.94**/**0.83**	1.34/**1.29**
500	0.83/0.68	**0.94**/**0.80**	0.93/**0.78**	0.81/0.65	0.89/0.72	**0.97**/**0.81**	1.13/**1.16**
1,000	0.89/0.73	0.93/0.78	0.93/0.78	0.91/0.76	0.94/0.78	**0.99**/**0.86**	1.10/**1.10**
5,000	**1.04**/**0.95**	–	1.03/**0.95**	**1.03**/**0.94**	**1.03**/**0.94**	**1.03**/**0.94**	1.05/**1.03**
10,000	1.04/**0.97**	–	1.04/**0.97**	1.04/**0.97**	1.04/0.95	**1.03**/0.95	1.04/**0.98**

Table 20: Width over warranted width and z-spread, σ/ρ, means over 5 repeats (exact posterior: 1 / 1 and 1.02 / 0.97). Bold, per parameter: closest to 1 on either side, and every model whose paired difference in distance from 1 is within its sd. Values below 1 are over-confident.

- **Every BNN has a lower NLL than a single NPE flow** on at least 4 of 5 repeats at every n, except empirical Bayes at $n = 50$ (-0.128 ± 0.551, 2/5), where both are over-confident (width over warranted 0.67 / 0.73 and 0.94 / 1.22, z-spread 1.50 / 1.39 and 1.51 / 1.19).
- **Calibration** (Table 20). At $n \leq 1{,}000$ every BNN except empirical Bayes at $n = 50$ is wider than warranted, mostly in ρ. Among the BNNs the data-free width is closest to 1 at $n = 50$ (1.08 / 1.12), and empirical Bayes at $n = 100$–1,000 (1.07 / 1.18, 1.04 / 1.19, 1.02 / 1.15), with tempering and the data-free width next. The single NPE flow is within 0.11 of 1 at $n = 100$–1,000 but over-confident in z-spread at every $n \leq 1{,}000$ (1.10–1.51). From $n = 5{,}000$ all models are within 0.06 of 1.
- **First moments.** Against $\mathcal{N}(0,1)$, relative first-moment error moves by at most 0.01 / 0.02 for every model except empirical Bayes at $n = 50$ (0.16 / 0.26 to 0.23 / 0.36) and tempering, which lowers the ρ error at $n \leq 500$ (at $n = 100$, 0.208 to 0.165).

G.4 Other variants

All runs here are 3-member fits. Unless stated, they use 5 repeats, are paired with $\mathcal{N}(0,1)$ as in Section 5.3 and use the earlier BNN hyperparameters (`results/bde_params_n<n>.npz` at every size); a negative difference means the variant is better.

Rasmussen-style Gibbs (`rasmussen_gibbs_retune`). Each weight has its own scale, $w_j \mid \tau_j^2 \sim \mathcal{N}(0, \tau_j^2)$ with $\tau_j^2 \sim$ Inv-Gamma$(2,1)$ [52], and each step draws every τ_j^2 from its conjugate conditional and moves $\mathbf{w}$ by MCLMC, with the re-tune. Against $\mathcal{N}(0,1)$ at the same 125,000 steps and 10,000 kept draws: test NLL -0.149 ± 0.056 ($n = 50$, 5/5), -0.025 ± 0.106 ($n = 100$, 2/5), -0.253 ± 0.096 ($n = 500$, 5/5), -0.200 ± 0.045 ($n = 1{,}000$, 5/5), with smaller widths at every size. Decomposed at 10,000 steps: at $n = 500$ and 1,000 $\mathcal{N}(0,1)$ with the re-tune and no τ sampling reproduces the gain (Gibbs minus it: $+0.007 \pm 0.025$, -0.004 ± 0.015), and Gibbs without the re-tune equals $\mathcal{N}(0,1)$ ($+0.007 \pm 0.154$, $+0.002 \pm 0.046$); at $n = 50$ both Gibbs variants match the fixed Student-$t(4, 0.7071)$, the marginal of the Gibbs prior, and beat $\mathcal{N}(0,1)$ by 0.16 (5/5) regardless of

the re-tune. So the gain is the re-tune at $n \geq 500$ and the marginal prior shape at $n = 50$.

Block Gibbs (`bnn.fit_gibbs_hierarchical`, single runs). The prior is centred, $w_j \sim \mathcal{N}(0, \tau_j^2)$ with $\tau_j^2 \sim$ Inv-Gamma$(2, 1)$, and the sampler alternates conjugate τ sweeps with ordinary MCLMC on $\mathbf{w}$. $n = 100$: $\max \widehat{R}$ 1.03, test NLL -0.905. $n = 10{,}000$: τ settles within 3–4 sweeps, yet $\max \widehat{R} = 8.25$, so the large-n mixing failure persists with no coupling between weights and scales inside an MCMC step.

Joint MCMC over per-weight scales (`prior_family="hierarchical"`, single runs at $n = 100$). The prior is non-centred, $w_j = \tau_j z_j$ with $\tau_j = g(u_j)$ and $u_j \sim \mathcal{N}(0, 1)$, and MCLMC samples all of them. $g = \exp$ makes τ log-normal and some τ_j diverge; $g =$ softplus (light-tailed) was stable and the best-mixing of the four; $g(u) = |u|$ diverged again, more slowly: in one chain two of 2,245 weights produced an aleatoric width of 48.9 / 34.3 (about 150 times every other run) while $\max \widehat{R}$ was 1.012; the bounded link $\tau_{\max}\Phi(u)$ was first evaluated on wrongly reconstructed weights (Appendix I).

Student-t and Laplace. Appendix G.2.

Optimized per-weight prior [8] (`legacy/optimize_prior.py`). Each weight has its own Gaussian (μ_j, σ_j), optimized for zero-data coverage. Each estimator fix improved the objective and worsened the fit (single runs, $n = 100$): mean prior scale 1.00 / 0.150 / 0.141, coverage area 0.0328 / 0.0126 / 0.0109, test NLL -0.772 / -0.615 / -0.503; the best coverage area never reached the isotropic data-free width's 0.0037. Deviations from the paper: its kernel-amplitude rule assumes a uniform parameter prior (ours is a truncated normal), and its kernel has a per-feature length scale with $1/N$ averaging (ours does not).

Per-step empirical Bayes (`bnn_ebstep_10k`, `bnn_ebstep_retune_10k`). The scale is reset to its fixed point after every chain step. Against $\mathcal{N}(0, 1)$ + re-tune: $n = 50$ NLL $+0.84 \pm 0.41$ (0/5, members blow up); $n = 100$ $+0.009 \pm 0.109$ with smaller widths; $n = 500$–$10{,}000$ $+0.016$ to $+0.028$ (at most 1/5).

Per-layer empirical Bayes (`bnn_ebblock_layer_10k`). The block update of Appendix G.1 with one scale per layer; final scales about 0.1 (input layer), 0.35–0.45 (hidden) and 0.44–0.69 (output). Against the isotropic version: $n = 50$ -0.68 ± 0.10 (5/5) but still $+0.28 \pm 0.24$ behind the data-free width (0/5); $n = 100$ $+0.006 \pm 0.017$; $n = 500$ $+0.092 \pm 0.021$ (0/5); $n = 1{,}000$ $+0.030 \pm 0.009$ (0/5).

H Convergence diagnostics in detail

Appendix H.2 uses the final runs (repeats 0–4 at every n, from their per-draw outputs); Appendix H.3 uses them where stated and the earlier 3-member runs of Appendix G otherwise.

H.1 Role of convergence diagnostics for BNNs

In general MCMC practice, convergence diagnostics gate the use of a sample: $\widehat{R}$ "is computed for each scalar quantity of interest" and a sample is used only if $\widehat{R} < 1.01$ [53]. For BNNs they carry much less weight, and are not the criterion by which samplers are judged. Weight-space posteriors are non-identifiable (permutation and scaling symmetries) and multimodal, so chains are not expected to agree in weight space [11]. Even in function space, chains started in different modes need not reach a common function: $\widehat{R}$ "assumes identifiable parameters, which is not given in BNNs", and a function-space metric "should not penalize these chain-wise differences" [21, Sec. 6.2]. Papamarkou et al. [10] show that a non-converged chain can still yield "a valuable posterior predictive

distribution". Accordingly, Izmailov et al. [11] judge HMC by $\widehat{R}$ on test-set predictions with a 1.1 threshold (met by 91% / 98% of predictions on the CIFAR-10 image / IMDB, while some weights have very large $\widehat{R}$), and MILE itself claims no convergence, reports a chain-wise $\widehat{R}$ on the weights above both the 1.1 and 1.01 cutoffs, and is justified by predictive performance [12, App. A.3].

We follow this practice: diagnostics are reported, never used to accept or discard a fit (only members with non-finite draws are removed, Section 5.2). Fits are judged mainly by their test NLL at θ^* (Section 5.1) and, in Appendices A.5 and A.6, by the agreement of the predictive with the exact posterior, a direct check of what the diagnostics only proxy, available here because the reference is exact. Diagnostics are computed on predictive quantities only, never on raw weights.

What is computed.

- *Pipeline diagnostic* (every BNN fit): the classic potential scale reduction factor $\widehat{R}$ [46], without splitting or rank normalization, and the effective sample size (Geyer's initial monotone sequence, FFT autocovariances), both as implemented in BlackJAX 1.2.2. They are evaluated on the predicted mean $\hat{\mu}$ (standardized units) for every test dataset and parameter, with the K members as the chains. Reported as maximum and mean over the 2,000 (test dataset, parameter) pairs; the flag $\max \widehat{R} \leq 1.01$ is recorded but has no effect.

- *BNN-oriented diagnostics* (computed from saved per-draw outputs): the rank-normalized $\widehat{R}$ of Vehtari et al. [53], (a) pooled across members, each member split in two, and (b) chain-wise $c\widehat{R}^{(4)}$, each member's chain split into 4 sub-chains [21, App. B.1], applied to three quantities per test dataset: the predicted mean, the predicted width s, and the per-draw log density at θ^*, summarized by the median, 95th percentile and fraction ≤ 1.01 over test datasets; and the per-member expanding-window LPPD [21, App. B.2, Eq. 8], $\mathrm{LPPD}_l = \frac{1}{M}\sum_j \log\left(\frac{1}{l}\sum_{l' \leq l} q(\theta_j^* \mid S_j, \mathbf{w}^{(l')})\right)$, with its change over the last 10% of the chain and its spread across members.

H.2 Pooled R-hat is not an acceptance test for BNNs

$\widehat{R}$ "is computed for each scalar quantity of interest" [53]. The pipeline diagnostic pools the members as chains and looks only at the predicted mean (Section 5.2). Table 21 adds the predicted width s and the per-draw log density at θ^*, the chain-wise $c\widehat{R}^{(4)}$ and the per-member LPPD paths of Sommer et al. [21, App. B], computed from the per-draw outputs of the final runs, which are stored in chain order.

- **It passes broken fits.** Empirical Bayes at $n = 50$, which collapses to a nearly constant predictor in every repeat (Appendix A.7), has a pipeline $\max \widehat{R}$ of 1.00 on two of five repeats (1.04–2.27 on the others). A hierarchical-prior run with two diverged weights in one chain, and an aleatoric width about 150 times every other run's, had $\max \widehat{R} = 1.012$ (earlier runs, Appendix G.4).

- **It does not see the width.** Pooled $\widehat{R}$ on s is at least that on μ in every row. Data-free scale at $n = 50$ / 100: μ passes at 87% / 83% of pairs pooled, s at 35% / 38%, while chain-wise both pass at $\geq 99\%$. Each member is stationary; the members settle on different widths, and their LPPDs differ by 0.15 / 0.18 nats. $\mathcal{N}(0,1)$ at $n = 500$: pooled median 1.008 on μ, s passes at 1%, member LPPD spread 0.34 nats.

- **At large n it fails, and the members do disagree.** $\mathcal{N}(0,1)$ at $n = 10{,}000$: pipeline $\max \widehat{R}$ 4.90–6.57, pooled median 1.68 on μ, chain-wise 1.16 (3% of pairs ≤ 1.01). Each chain is nearly

model	n	fits	pipeline max $\widehat{R}$	pooled $\widehat{R}$ median μ / s / $\log p$	pooled frac. ≤ 1.01 μ / s	chain-wise median μ / s	member LPPD spread	ensemble gain	max LPPD drift
$\mathcal{N}(0,1)$	50	5	1.02–1.05	1.007 / 1.024 / 1.008	0.78 / 0.04	1.008 / 1.012	0.285	+0.013	0.021
	100	5	1.04–1.19	1.011 / 1.072 / 1.015	0.46 / 0.01	1.010 / 1.013	0.458	+0.025	0.026
	500	5	1.08–1.23	1.008 / 1.055 / 1.022	0.64 / 0.01	1.006 / 1.006	0.338	+0.012	0.008
	1,000	5	1.04–1.07	1.009 / 1.020 / 1.013	0.55 / 0.21	1.011 / 1.009	0.156	+0.004	0.009
	5,000	5	1.99–2.63	1.235 / 1.300 / 1.200	0.00 / 0.00	1.098 / 1.146	0.045	+0.039	0.018
	10,000	5	4.90–6.57	1.683 / 1.879 / 1.592	0.00 / 0.00	1.158 / 1.242	0.126	+0.073	0.004
tempered	50	5	1.02–1.07	1.011 / 1.025 / 1.016	0.52 / 0.16	1.015 / 1.018	0.196	+0.019	0.026
	100	5	1.06–1.13	1.012 / 1.062 / 1.016	0.43 / 0.00	1.011 / 1.014	0.366	+0.021	0.025
	500	5	1.07–1.27	1.027 / 1.058 / 1.027	0.13 / 0.02	1.027 / 1.028	0.121	+0.013	0.011
	1,000	5	1.10–1.23	1.036 / 1.067 / 1.037	0.08 / 0.01	1.030 / 1.032	0.076	+0.014	0.009
data-free width	50	5	1.02–1.06	1.002 / 1.015 / 1.004	0.87 / 0.35	1.000 / 1.000	0.145	+0.004	0.003
	100	5	1.02–1.08	1.003 / 1.017 / 1.005	0.83 / 0.38	1.000 / 1.000	0.179	+0.004	0.007
	500	5	1.02–1.07	1.003 / 1.012 / 1.007	0.91 / 0.42	1.002 / 1.002	0.154	+0.003	0.006
	1,000	5	1.05–1.15	1.004 / 1.017 / 1.009	0.88 / 0.31	1.003 / 1.004	0.143	+0.003	0.006
	5,000	5	1.49–1.93	1.159 / 1.224 / 1.148	0.00 / 0.00	1.098 / 1.153	0.041	+0.020	0.004
	10,000	5	4.58–6.69	1.662 / 1.905 / 1.591	0.00 / 0.00	1.174 / 1.273	0.072	+0.039	0.005
$\mathcal{N}(0,1)$ + re-tune	50	5	1.02–1.04	1.011 / 1.017 / 1.008	0.46 / 0.16	1.016 / 1.025	0.136	+0.011	0.018
	100	5	1.01–1.05	1.011 / 1.017 / 1.009	0.50 / 0.15	1.016 / 1.030	0.143	+0.008	0.031
	500	5	1.03–1.05	1.013 / 1.022 / 1.015	0.30 / 0.08	1.019 / 1.035	0.106	+0.005	0.024
	1,000	5	1.06–1.11	1.028 / 1.035 / 1.028	0.07 / 0.01	1.033 / 1.050	0.068	+0.006	0.022
	5,000	5	1.86–2.27	1.192 / 1.248 / 1.166	0.00 / 0.00	1.105 / 1.164	0.057	+0.033	0.011
	10,000	5	2.67–4.43	1.326 / 1.483 / 1.279	0.00 / 0.00	1.129 / 1.252	0.068	+0.047	0.005
empirical Bayes	50	5	1.00–2.27	1.038 / 1.306 / 1.147	0.54 / 0.00	1.009 / 1.563	0.457	+0.045	0.032
	100	5	1.01–1.03	1.013 / 1.017 / 1.008	0.43 / 0.28	1.041 / 1.053	0.081	+0.001	0.026
	500	5	1.02–1.03	1.010 / 1.022 / 1.012	0.48 / 0.14	1.022 / 1.052	0.068	+0.002	0.016
	1,000	5	1.03–1.06	1.026 / 1.074 / 1.032	0.12 / 0.01	1.045 / 1.148	0.041	+0.003	0.016
	5,000	5	1.36–1.68	1.144 / 1.181 / 1.128	0.00 / 0.00	1.123 / 1.206	0.027	+0.009	0.006
	10,000	5	2.19–2.68	1.285 / 1.429 / 1.239	0.00 / 0.00	1.171 / 1.306	0.033	+0.013	0.003

Table 21: Convergence diagnostics of the final 12-member BNNs, mean over fits (repeats 0–4). Pooled: rank-normalized $\widehat{R}$ [53], each member split in two (24 sub-chains); median over the 2,000 (test dataset, parameter) pairs for μ and s and the 1,000 test datasets for $\log p$. Chain-wise: $c\widehat{R}^{(4)}$ per member [21, App. B.1], mean over members. Member LPPD spread: max minus min over members of the final LPPD (nats); ensemble gain: pooled LPPD minus mean member LPPD; drift: largest |change of a member's LPPD over the last 10% of its chain|. Pipeline max $\widehat{R}$: Section 5.2. Nine fits kept 11 of 12 members ($\mathcal{N}(0,1)$ + re-tune at $n = 50$, repeats 2 and 3, at $n = 100$, repeats 0–2, at $n = 500$, repeat 4, and at $n = 5{,}000$, repeat 2; empirical Bayes at $n = 100$, repeat 1, and $n = 5{,}000$, repeat 2); their per-member statistics are over 11 members.

stationary in LPPD (largest change over its last 10%: 0.004 nats), but the 12 members differ by 0.13 nats and pooling them gains 0.07: members settle in different predictive solutions, and single chains do not mix within them. The test NLL is nonetheless stable across repeats (-2.284 ± 0.005).

- **Tempering.** At the selected temperatures the tempered BNN's members differ by 0.20 and 0.37 nats at $n = 50$ and 100 ($\mathcal{N}(0,1)$: 0.29 and 0.46), with pipeline max $\widehat{R}$ 1.02–1.13. At $n = 500$ and 1,000, where its temperature sits next to the mixing failure of Appendix C, its chain-wise $\widehat{R}$ on μ is higher than that of $\mathcal{N}(0,1)$ (1.027 and 1.030 against 1.006 and 1.011), with 26% and 23% of test datasets at or below 1.01 against 70% and 50%.

A pooled $\widehat{R}$ on the mean of at most 1.01 therefore neither certifies a fit nor, above it, rejects one. We report the diagnostics of Table 21 alongside each result and judge fits by their test NLL and their agreement with the exact posterior (Section 5.2).

H.3 Periodic re-tuning of MILE's step size and L

MILE tunes (L, ϵ) once, in its warmup, and samples with "a fixed budget of 60k steps per chain"; its ablations change one warmup hyperparameter at a time on two UCI regression datasets and support "the claim that MILE is effectively tuning-free" [12, Sec. 4.3, App. A.2]. They do not consider adapting during sampling. The re-tune of Section 4.5 re-estimates both every 1,000 steps.

- **The warmup's values are far from where the re-tune settles.** $\mathcal{N}(0, 1)$, median over members: L falls to 0.27–0.36 of its warmup value at $n \leq 1{,}000$ and rises 1.7-fold and 6.1-fold at $n = 5{,}000$ and 10,000; ϵ falls to 0.68–0.96 of it (earlier runs).
- **Fit.** Test NLL -0.150 ± 0.042 at $n = 500$ and -0.123 ± 0.009 at $n = 1{,}000$ (5/5 each), the size of the tempering gain; -0.039 ± 0.037 at $n = 50$ (17/20), a tie at $n = 100$ and no difference from $n = 5{,}000$ (Table 7).
- **Mixing: mixed.** In the final runs (Table 21, five fits per size) the re-tune lowers the members' LPPD spread on every fit at $n \leq 1{,}000$ (0.05–0.22 against 0.11–0.69 nats) but raises the chain-wise $\widehat{R}$ on s on every fit there (1.020–1.059 against 1.004–1.017). The pipeline's max $\widehat{R}$ is lower on every fit at $n = 100$, 500, 5,000 and 10,000 but not at $n = 1{,}000$ (1 of 5), and at $n = 10{,}000$ the pooled $\widehat{R}$ on μ is lower on every fit (1.32–1.33 against 1.60–1.77). On the Gibbs sampler of Appendix G.4 (earlier runs) the re-tune meets $\max \widehat{R} \leq 1.01$ on 5/5 repeats at $n = 50$ and 100, against 2/5 and 0/5 without it and 1/5 and 0/5 with 30,000 extra plain steps (not all seed-paired); that sampler warms up at $\tau = 1$ and then samples τ, a mismatch stock MILE does not have.
- **Faster exploration or bias.** Whether the re-tune's gain comes from faster exploration or from a biased target is not established: the adaptation continues through the kept draws without a Metropolis correction, and each re-tune's 250 steps also move the chain (2,500 extra steps per member, with no step-matched control on stock MILE).

I Negative results and pipeline bugs

- **Step-count mismatch.** `fit_correlated` reads its budget as raw steps, the re-tune loop as kept draws: comparisons between the two paths before 2026-10-01 used chains 10–12.5× apart; all were redone step-matched.
- **Seed indexing.** Seeds were indexed by position in the job's own size list, so single-size jobs ran with seed = repeat; three early files are not seed-paired (fixed 2026-10-01).
- **One test set per script.** Before 2026-08-25 each script simulated its own data, giving different test sets; replaced by one canonical dataset file.
- **Retracted claims.** A between-chain variance fraction at $n = 10{,}000$ came from a fit with $\max \widehat{R}$ 14.8, ESS 1.5; an apparent advantage of $\mathcal{N}(0, 1)$ over tuned priors (margin 0.034–0.104) was smaller than its own chain spread (0.127); "coverage-calibrated priors worsen the fit" compared runs with different learning rates.
- **Hierarchical links.** The divergences of Appendix G; the bounded link's first runs used a stale $|u|$ reconstruction of the weights (files deleted).

- **Optimized prior.** Its objective improved while the fit worsened (Appendix G).
- `bde` **pretraining patience** counted single steps (evaluation every step, $\Delta > 10^{-9}$), stopping pretraining almost at once at $n = 100$; replaced by the schedule of Section 4.5.
- `bde` **ignores weight decay**: `Bde._fit` passes `optax.adam(lr)`, overriding its builder's AdamW.
- $n = 50$ **learning rate.** The single-split search chose 0.095, under which pretraining diverged and the best checkpoint was the untrained network; the re-tuned value changed the test NLL by 0.006.
- **Loss scale.** The pretraining loss was a mean while the sampling likelihood is a sum; changed to a sum (2026-08-25).
- **float32 bivariate density.** `metrics.bivariate_normal_logpdf` returns impossible values once a width sits at its floor and $|r|$ rounds to 1; diagnostics compute it in float64.
- **R-hat description.** The pipeline's $\widehat{R}$ was described as split, rank-normalized; it is the classic PSRF (Section 5.2).